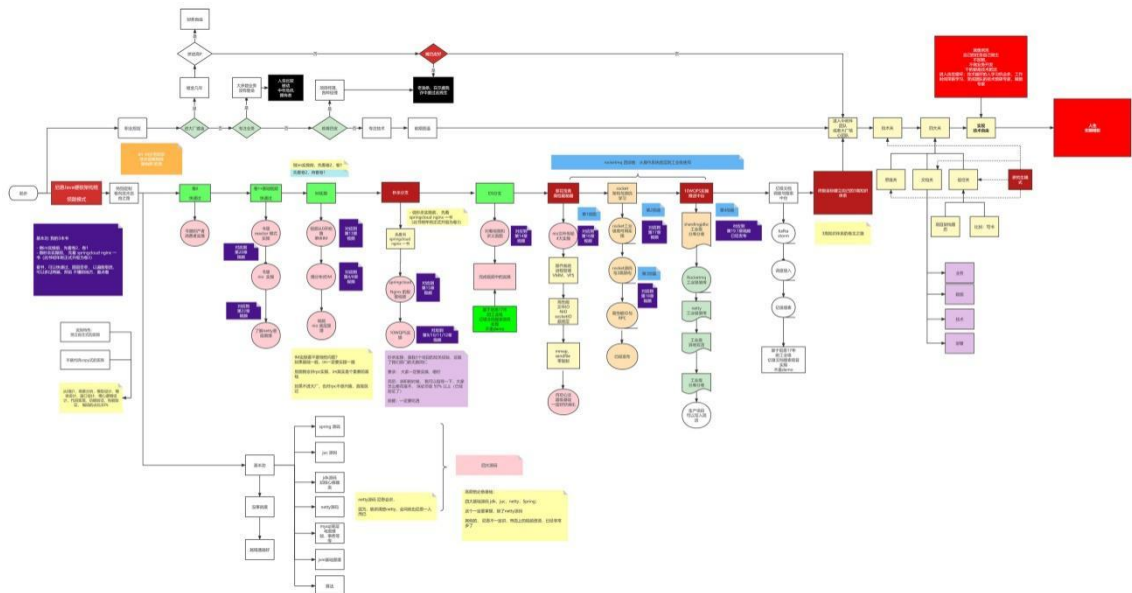


牛逼的职业发展之路

40 岁老架构尼恩用一张图揭秘：Java 工程师的高端职业发展路径，走向食物链顶端的之路

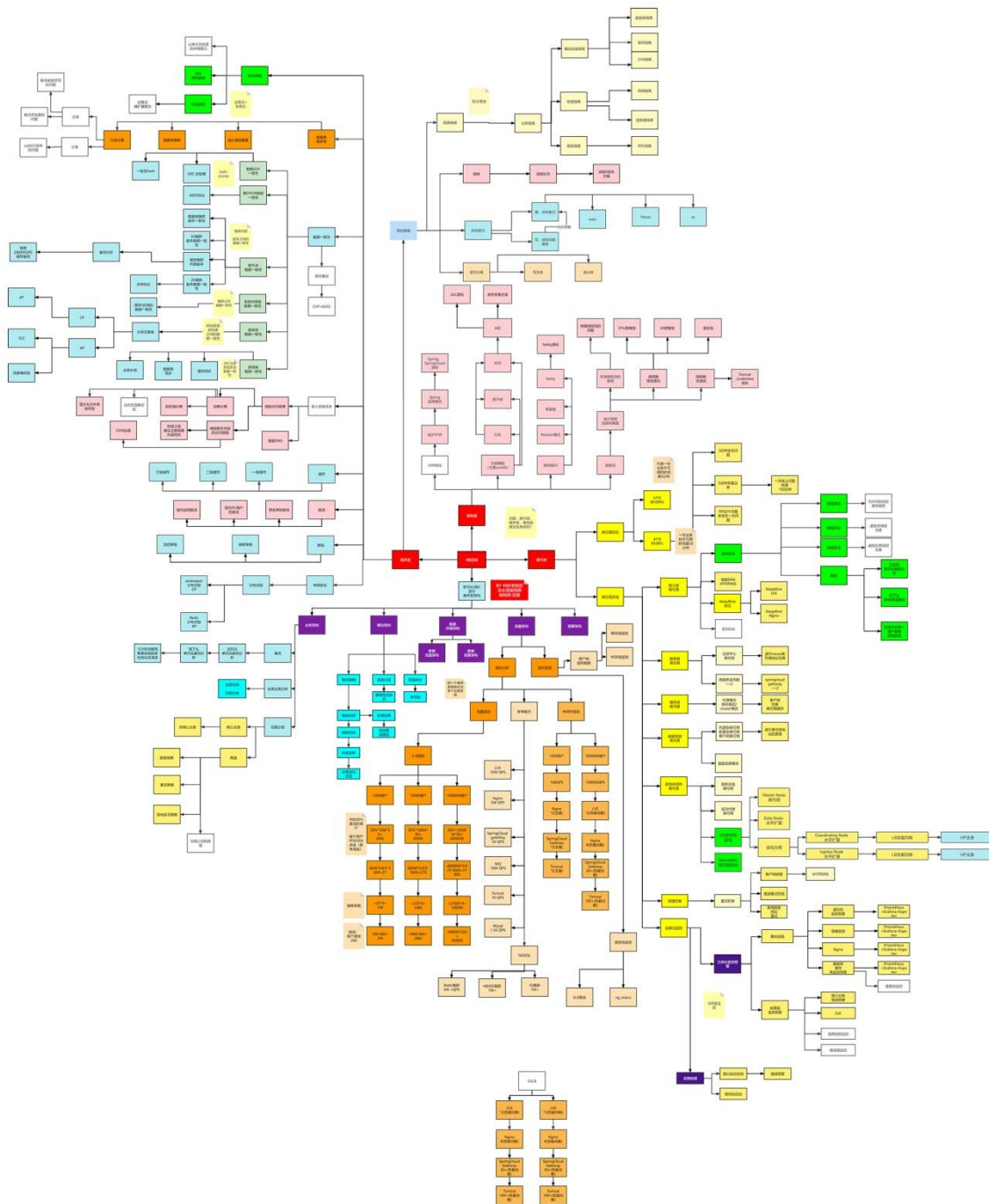
链接：<https://www.processon.com/view/link/618a2b62e0b34d73f7eb3cd7>



史上最全：价值10W的架构师知识图谱

此图梳理于尼恩的多个 3 高生产项目：多个亿级人民币的大型 SAAS 平台和智慧城市项目

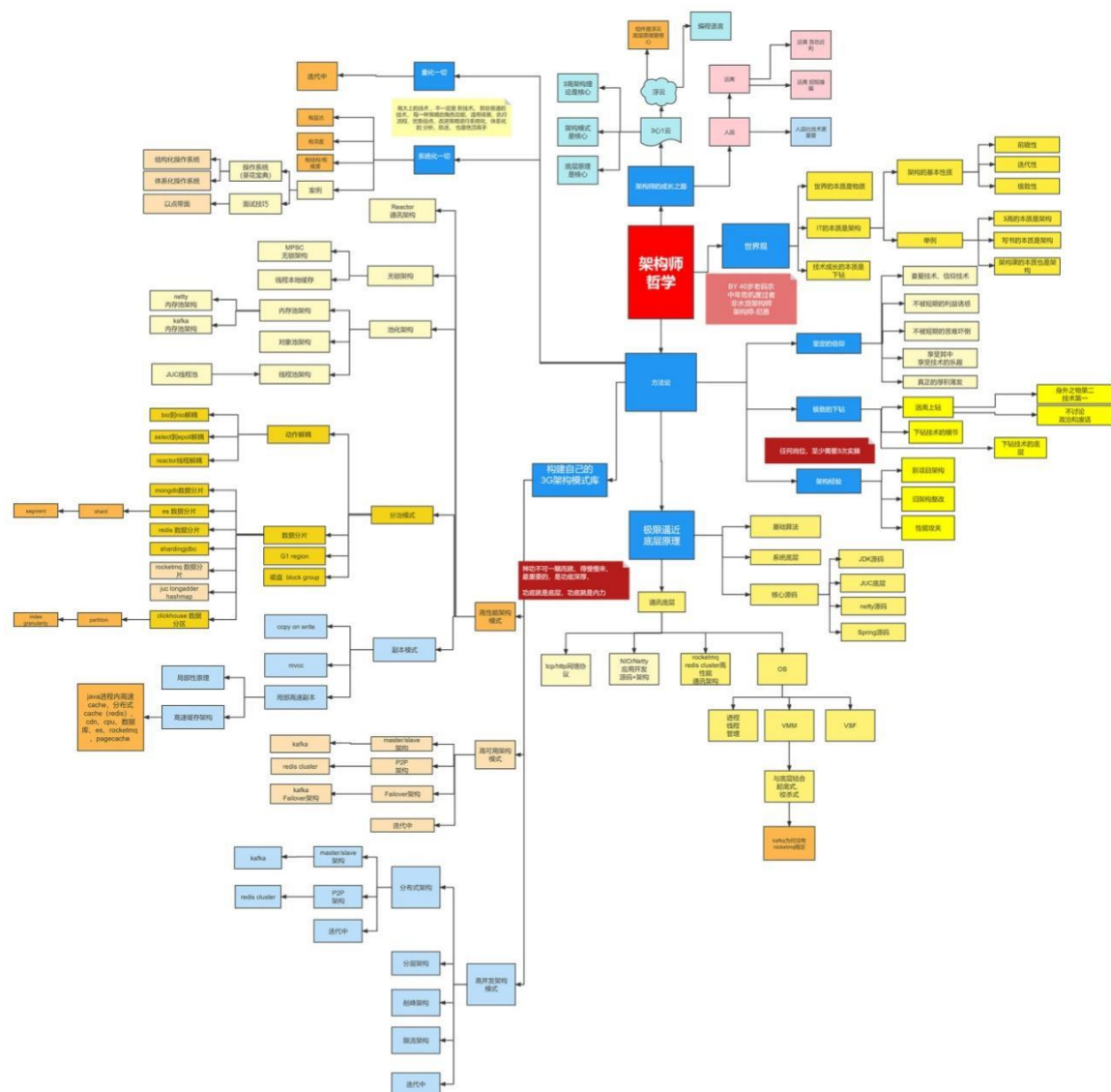
链接：<https://www.processon.com/view/link/60fb9421637689719d246739>



牛逼的架构师哲学

40 岁老架构师尼恩对自己的 20 年的开发、架构经验总结

链接: <https://www.processon.com/view/link/616f801963768961e9d9aec8>



牛逼的3高架构知识宇宙

尼恩 3 高架构知识宇宙，帮助大家穿透 3 高架构，走向技术自由，远离中年危机

链接: <https://www.processon.com/view/link/635097d2e0b34d40be778ab4>



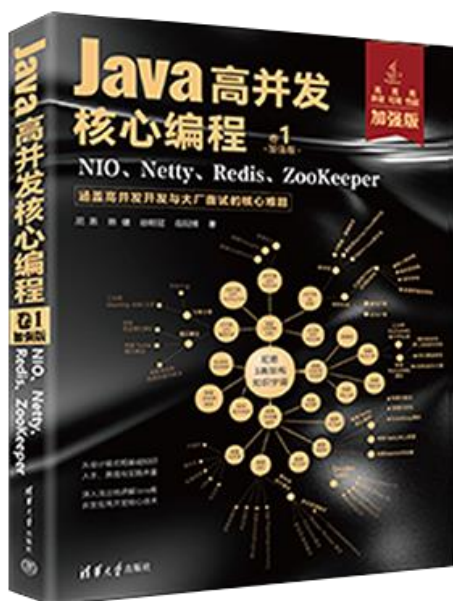
尼恩Java高并发三部曲（卷1加强版）

老版本：《Java 高并发核心编程 卷1：NIO、Netty、Redis、ZooKeeper》（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷1 **加强版**：NIO、Netty、Redis、ZooKeeper》

- 由浅入深地剖析了高并发 IO 的底层原理。
- 图文并茂地介绍了 TCP、HTTP、WebSocket 协议的核心原理。
- 细致深入地揭秘了 Reactor 高性能模式。
- 全面介绍了 Netty 框架，并完成单体 IM、分布式 IM 的实战设计。
- 详尽地介绍了 ZooKeeper、Redis 的使用，以帮助提升高并发、可扩展能力

详情：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



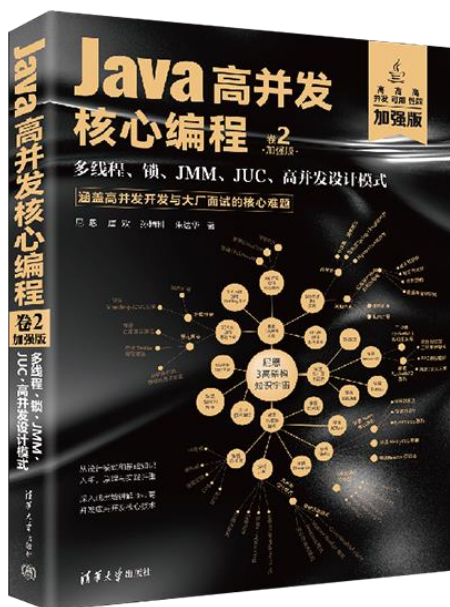
尼恩Java高并发三部曲（卷2加强版）

老版本：《Java 高并发核心编程 卷2：多线程、锁、JMM、JUC、高并发设计模式》
（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷2 **加强版**：多线程、锁、JMM、JUC、高并发设计模式》

- 由浅入深地剖析了 Java 多线程、线程池的底层原理。
- 总结了 IO 密集型、CPU 密集型线程池的线程数预估算法。
- 图文并茂的介绍了 Java 内置锁、JUC 显式锁的核心原理。
- 细致深入地揭秘了 JMM 内存模型。
- 全面介绍了 JUC 框架的设计模式与核心原理，并完成其高核心组件的实战介绍。
- 详尽地介绍了高并发设计模式的使用，以帮助提升高并发、可扩展能力

详情参阅：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



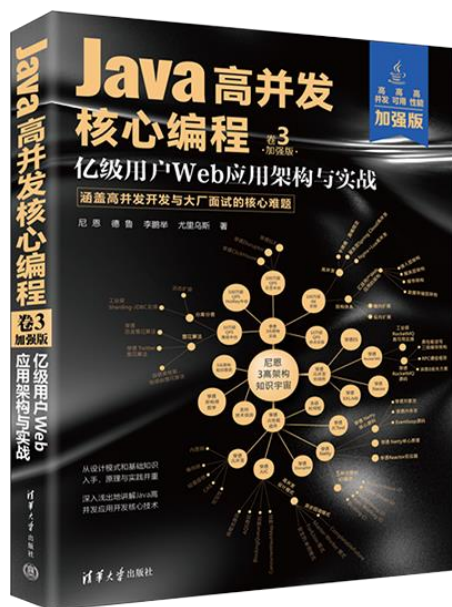
尼恩Java高并发三部曲（卷3加强版）

老版本：《SpringCloud Nginx 高并发核心编程》（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷3 **加强版**：亿级用户 Web 应用架构与实战》

- 在当今的面试场景中，3 高知识是大家面试必备的核心知识，本书基于亿级用户 3 高 Web 应用的架构分析理论，为大家对 3 高架构系统做一个系统化和清晰化的介绍。
- 从 Java 静态代理、动态代理模式入手，抽丝剥茧地解读了 SpringCloud 全家桶中 RPC 核心原理和执行过程，这是高级 Java 工程师面试必备的基础知识。
- 从Reactor 反应器模式入手，抽丝剥茧地解读了Nginx 核心思想和各配置项的底层知识和原理，这是高级 Java 工程师、架构师面试必备的基础知识。
- 从观察者模式入手，抽丝剥茧地解读了 RxJava、Hystrix 的核心思想和使用方法，这也是高级 Java 工程师、架构师面试必备的基础知识。

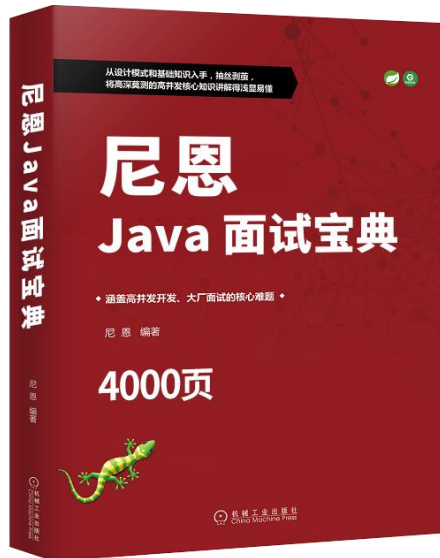
详情：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



尼恩Java面试宝典

40 个专题（卷王专供+ 史上最全 + 2023 面试必备）

详情：<https://www.cnblogs.com/crazymakercircle/p/13917138.html>



- 📖 专题01: JVM面试题 (卷王专供 + 史上最全 + 2023面试必备) -V20-from-尼恩Java面试宝典.pdf
- 📖 专题02: Java算法面试题 (卷王专供 + 史上最全 + 2023面试必备) -V2 -from-Java面试红宝书-release.pdf
- 📖 专题03: Java基础面试题 (卷王专供 + 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题04: 架构设计面试题 (卷王专供 + 史上最全 + 2023面试必备) -V24-from-Java面试红宝书-release.pdf
- 📖 专题05: Spring面试题_专题06: SpringMVC_专题07: Tomcat面试题 (卷王专供+ 史上最全 + 2023面试必备) -V3-from-尼恩面试宝典-release.pdf
- 📖 专题08: SpringBoot面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题09: 网络协议面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题10: TCP/IP协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题11: JUC并发包与容器类 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题12: 设计模式面试题 (卷王专供+ 史上最全 + 2023面试必备) -V14-from-Java面试红宝书.pdf
- 📖 专题13: 死锁面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题14: Redis 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V22-from-Java面试红宝书-release.pdf
- 📖 专题15: 分布式数据库面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题16: Zookeeper 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题17: 分布式事务面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题18: 一致性协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题19: Zab协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题20: Paxos 协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题21: raft 协议 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题22: Linux面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题23: Mysql 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V29-from-尼恩Java面试宝典-release.pdf
- 📖 专题24: SpringCloud 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V12-from-Java面试红宝书-release.pdf
- 📖 专题25: Netty 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题26: 消息队列面试题: RabbitMQ、Kafka、RocketMQ (卷王专供+ 史上最全 + 2023面试必备) -V10-from-Java面试红宝书-release.pdf
- 📖 专题27: 内存泄漏 内存溢出 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题28: JVM 内存溢出 实战 (卷王专供+ 史上最全 + 2023面试必备) -V17-from-Java面试红宝书-release.pdf
- 📖 专题29: 多线程面试题 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题30: HR面试题: 过五关斩六将后, 小心阴沟翻船! (史上最全、避坑宝典) -V2-from-Java面试红宝书-release.pdf
- 📖 专题31: Hash链表面试题 (卷王专供+ 史上最全 + 2023面试必备) -V16-from-Java面试红宝书-release.pdf
- 📖 专题32: 大厂面试的基本流程和面试准备 (阿里、腾讯、网易、京东、头条.....) -V2-from-Java面试红宝书-release.pdf
- 📖 专题33: BST、AVL、RBT红黑树、三大核心数据结构 (卷王专供+ 史上最全 + 2023面试必备) -V2-from-Java面试红宝书-release.pdf
- 📖 专题34: Elasticsearch面试题 (卷王专供+ 史上最全 + 2023面试必备) -V3-from-Java面试红宝书-release.pdf
- 📖 专题35: Mybatis面试题 (卷王专供+ 史上最全 + 2023面试必备) -V3-from-尼恩Java面试宝典-release.pdf
- 📖 专题36: Dubbo面试题 (卷王专供+ 史上最全 + 2023面试必备) -V21-from-尼恩Java面试宝典-release.pdf
- 📖 专题37: Docker面试题 (卷王专供+ 史上最全 + 2023面试必备) -V26-from-尼恩Java面试宝典-release.pdf
- 📖 专题38: K8S面试题 (卷王专供+ 史上最全 + 2023面试必备) -V26-from-尼恩Java面试宝典-release.pdf
- 📖 专题39: Nginx面试题 (卷王专供+ 史上最全 + 2023面试必备) -V27-from-尼恩Java面试宝典-release.pdf
- 📖 专题40: 操作系统面试题 (卷王专供+ 史上最全 + 2023面试必备) -V28-from-尼恩Java面试宝典-release.pdf

Seata 学习圣经

Seata 学习圣经

Seata AT模式实战（图解+秒懂+史上最全）

分布式事务使用场景

数据库的水平分割倒逼分布式事务

微服务化倒逼分布式事务

10WQPS秒杀实操的分库架构

Spring Cloud Alibaba Seata

AT模式(业务侵入小)

AT模式角色如下

AT模式(2PC)基本处理逻辑如下

AT模式第一阶段

这也是Seata和XA事务的不同之处：

AT模式第二阶段

场景一：提交，全局提交

场景2：回滚，全局回滚

AT模式相对于XA模式的优势

10WQPS秒杀实操的AT分布式事务架构

快速开始搭建Seata环境

全局事务和分支事务的存储模式

file 模式

db 模式

配置和启动Seata-server服务（TC服务）

seata-server包下载和解压

启动 TC Server（单体的实例）

使用 File 存储器全局会话

seata-server的TC端配置

TC端的参数清单，大致如下：

部署集群 TC Server

使用 Nacos 作为注册中心

TC端数据库初始化

创建TC端的专属独立库seata。

通过 source 命令，导致sql脚本就可以了。

db 数据库共享全局事务会话信息：

如果使用file作为配置文件

如何使用nacos作为配置中心：

补充：MySQL8 的支持

启动 第一个TC Server

启动 第2个TC Server

实验演示：注册到nacos之后，可以在nacos上看效果

控制台提交nacos配置脚本

修改 自己定义的服务组

修改数据库连接

将文件中的配置导入nacos 即可

直接使用nacos 的data-id配置文件

制作和上传配置文件

config文件中的修改

重启 TC，一切OK

Namespace、Group、DataId

Namespace：区分不同的环境

进行Group的切换

- 进行NameSpace的切换
- AT模式的RM&TM&TC直接的交互流程
- TM&RM 应用开发
 - 业务场景
 - 中间件版本选型
 - 数据库准备
 - 创建RM端的专属独立库seata。
 - 创建RM端的专属独立的表。
- TM端的注册中心配置
 - file.conf本地配置
 - file.conf 配置文件分部分说明
 - 1.transport
 - 2.service
 - 3.client
- 项目准备
 - 模块架构
 - 模块的角色架构
 - maven依赖
 - 事务分组是什么？
- TM&RM配置详解
 - registry.conf配置：配置注册中心和配置中心
 - 注册中心
 - 配置中心
 - file.conf
 - 事务日志存储配置：
 - TC信息配置
 - 客户端相关工作的机制
 - DataSourceConfig
 - 事务注解
- 代码实现
 - 10WQPS秒杀实操的AT分布式事务架构
 - 服务秒杀
 - seckillController
 - seckillServiceImpl
 - 库存服务的Feign类
 - 订单服务的Feign类
 - 订单服务
 - orderController
 - OrderServiceImpl
 - 库存服务
 - stockController
 - StockServiceImpl
 - 实验演示：启动后进行事务演示
 - 错误排除
- Seata AT分布式事务执行流程演示
 - 正常执行
 - 观察每个阶段的undo_log和TC的记录
 - 本实验演示：通过配套视频给出
 - 异常执行
 - 观察每个阶段的undo_log和TC的记录
 - 本实验演示：通过配套视频给出

Seata TCC 模式实战

- Seata TCC基本原理
 - Seata TCC模式的流程图
 - Seata TCC 事务的3个操作
- 10WQPS秒杀的TCC分布式事务架构
- 库存服务
 - controller

- service
- 订单服务
 - controller
 - service
- 秒杀服务
 - controller
 - service

以下两个实验，请参见配套视频

一个金额扣减的例子：

一个库存扣减操作：

Seata TCC 的3大优势

优势1： try 阶段的资源锁定不需要阻塞等待

优势1： 异步提交优化

优势2： RPC 次数减少了 50%

Seata TCC 事务的3大常见问题

TCC 问题1： 幂等问题

TCC 问题2： 空回滚

TCC 问题3： 防悬挂

Seata TCC核心源码分析

GlobalTransactionScanner事务切面的注解扫描器

基础知识： Spring AOP自动代理创建流程

TransactionalInterceptor 源代码拦截器的创建

全局事务处理拦截器（GlobalTransactionalInterceptor）

TransactionalTemplate 有四个重要的模版方法

全局事务的默认实现 DefaultGlobalTransaction

DefaultTransactionManager 的几个核心方法

全局事务提交

全局事务回滚

一阶段分支事务Try处理流程（TccActionInterceptor）

分支注册&&上下文参数处理

tcc_fence_log

tcc_fence_log表结构

TCC 屏障处理器 TCCFenceHandler

一阶段 准备屏障 TCCFenceHandler.prepareFence() :

二阶段 提交屏障 TCCFenceHandler.commitFence() :

二阶段 回滚 屏障 TCCFenceHandler.rollbackFence() :

二阶段处理流程

Commit处理流程

Rollback处理流程

TCC幂等&&空回滚&&防悬挂控制总结

源码解读： @GlobalLock 注解 + select for update语句执行过程

源码解读： 提交过程

终于明白： GlobalLock 注解的价值

XID 如何进行远程传递

seata 压力测试

压力测试 参考实验1： AT 20 tps

压力测试 参考实验 2： TCC 100tps， AT 20 tps

Seata相关的面试题以及参考答案

阿里面试： Seata 如何实现 RC ? 保证事务的隔离性？

尼恩说在前面

Seata 事务隔离级别解读

Seata 的事务隔离级别

Seata事务 + 独立事务 场景的 脏读、脏写

如何防止脏读/脏写

方案一： 独立事务执行时加 @GlobalTransactional注解：

方案二： 独立事务业务二执行时加 @GlobalLock注解：

方案三： 业务二执行时加 @GlobalLock 注解 + select for update语句：

说在最后： 有问题找老架构取经

面试真题：怎么使用Seata框架，来保证事务的隔离性？

Seata隔离级别

面试真题：说说，Seata AT模式的事务流程

回顾：Seata执行流程

为什么Seata在第一阶段就直接提交了分支事务？

面试真题：比较一下，经典XA和Seata AT模式的区别？

参考文档：

Seata AT模式实战（图解+秒懂+史上最全）

阅读此文之前，请先阅读：

[分布式事务（图解 + 史上最全 + 吐血推荐）](#)

参考链接

系统架构知识图谱（一张价值10w的系统架构知识图谱）

<https://www.processon.com/view/link/60fb9421637689719d246739>

秒杀系统的架构

<https://www.processon.com/view/link/61148c2b1e08536191d8f92f>

先来看下为什么会产生分布式事务问题

分布式事务使用场景

简单来说：

一次业务操作需要跨多个数据源或需要跨多个系统进行远程调用,就会产生分布式事务问题

作为典型案例，搬出经典的银行转账问题：

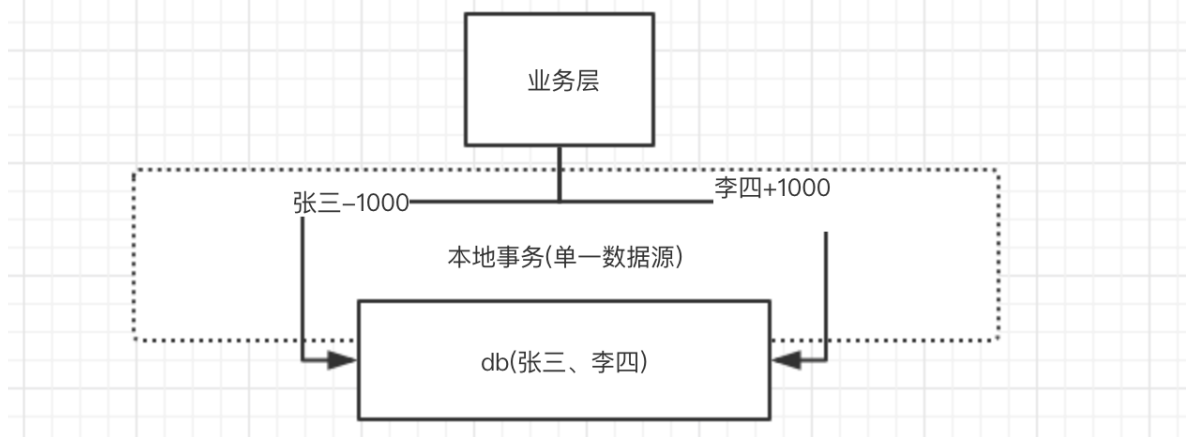
需求：

假设银行(bank)中有两个客户(name)张三和李四, 我们需要将张三的1000元存款(sal)转到李四的账户上

约束：

不能出现中间状态，张三减1000，李四没加，或者反之

数据库单表操作



如果两个用户对应的银行存款数据在一个数据源中，即一个数据库中，那么通过spring框架下的 `@Transactional` 注解来保证单一数据源增删改查的一致性。

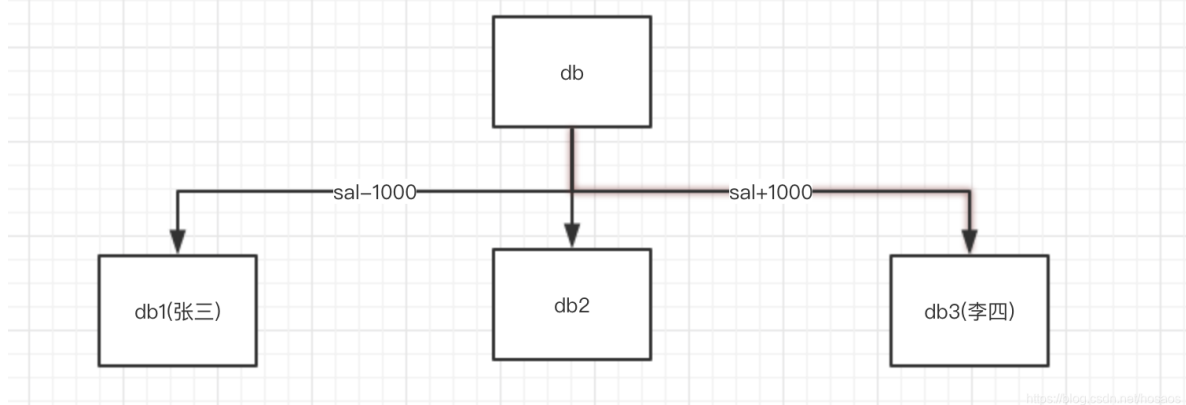
数据库的水平分割倒逼分布式事务

但是随着业务的不断扩大，用户数在不断变多，几百万几千万用户时数据可以存一个库甚至一个表里，假设有10个亿的用户？

一个表当然放不下，需要分表，当然需要分库来配合。

为了解决数据库上的瓶颈，分库是很常见的解决方案，不同用户就可能落在不同的数据库里，原来一个库里的事务操作，现在变成了跨数据库的事务操作。

数据库水平拆分

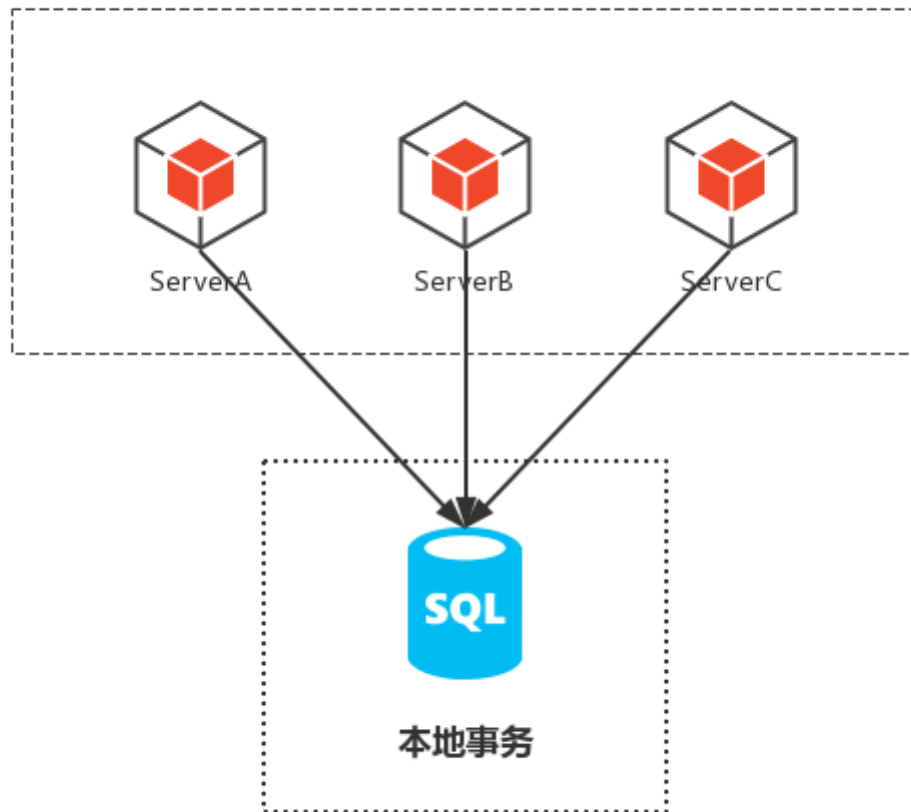


此时 `@Transactional` 注解就失效了，这就是跨数据库分布式事务问题

微服务化倒逼分布式事务

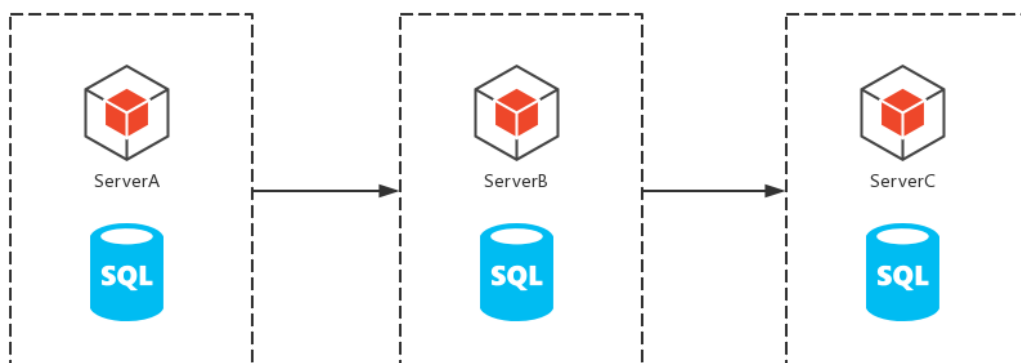
当然，更多的情形是随着业务不断增长，将业务中不同模块服务拆分成微服务后，同时调用多个微服务所产生的

设想一个传统的单体应用，无论多少内部调用，最后终归是在同一个数据库上进行操作来完成一向业务操作，如图：



随着业务量的发展，业务需求和架构发生了巨大的变化，整体架构由原来的单体应用逐渐拆分成为了微服务，

原来的3个服务被从一个单体架构上拆开了，成为了3个独立的服务，分别使用独立的数据源，也不在之前共享同一个数据源了，具体的业务将由三个服务的调用来完成，如图：



?

此时，每一个服务的内部数据一致性仍然有本地事务来保证。

但是面对整个业务流程上的事务应该如何保证呢？这就是在微服务架构下面临的挑战，如何保证在微服务中的数据一致性。

微服务化的银行转账情景往往是这样的

1. 调用交易系统服务创建交易订单；
2. 调用支付系统记录支付明细；
3. 调用账务系统执行 A 扣钱；
4. 调用账务系统执行 B 加钱；
5. 调用账务系统执行 B 加钱；

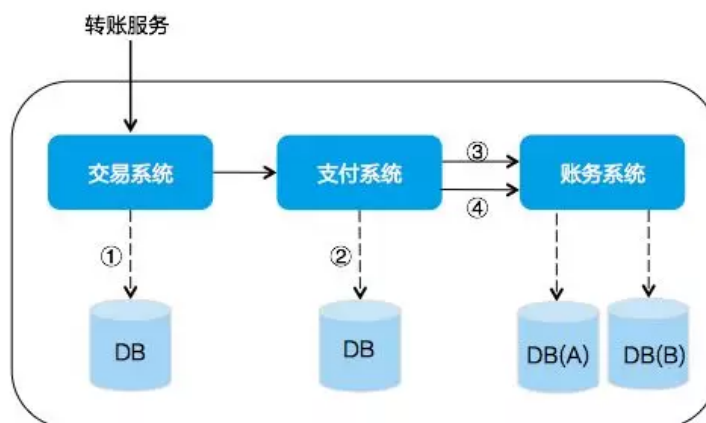
以转账场景为例：

A → ¥ 100 → B

一次转账，跨多个服务和数据库

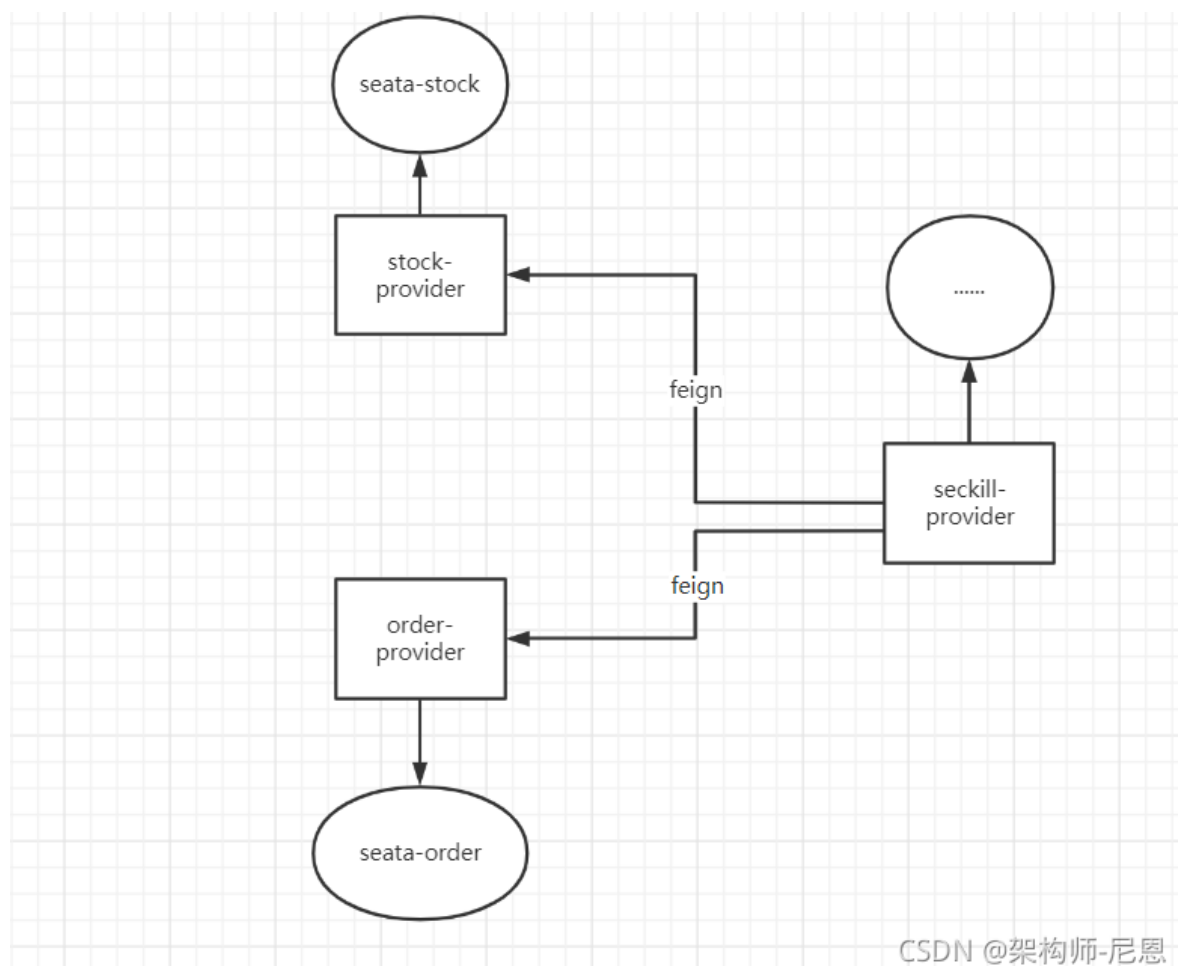
如何保证以下操作的原子性：

- 交易系统：创建交易订单
- 支付系统：记录支付明细
- 账务系统：A 扣钱
- 账务系统：B 加钱



如图所示，每个系统都对应一个独立的数据源，且可能位于不同机房，同时调用多个系统的服务很难保证同时成功，这就是**跨服务分布式事务问题**

10WQPS秒杀实操的分库架构



CSDN @架构师-尼恩

Spring Cloud Alibaba Seata

解决分布式事务问题，有两个设计初衷

- 对业务无侵入：即减少技术架构上的微服务化所带来的分布式事务问题对业务的侵入
- 高性能：减少分布式事务解决方案所带来的性能消耗

seata中，常用的有两种分布式事务实现方案，AT 及 TCC

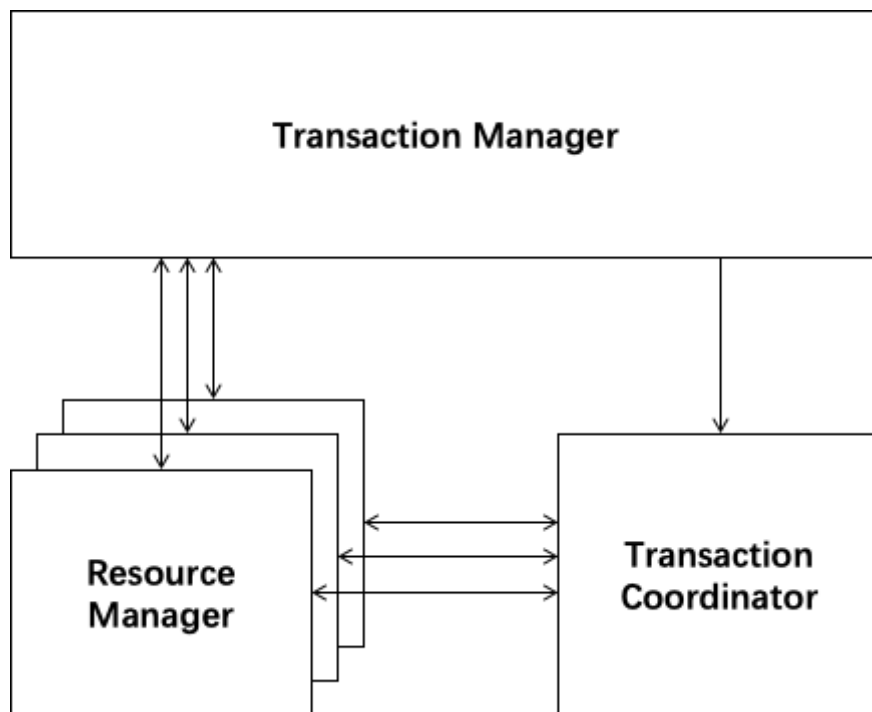
- AT模式主要关注多 DB 访问的数据一致性，当然也包括多服务下的多 DB 数据访问一致性问题
- TCC 模式主要关注业务拆分，在按照业务横向扩展资源时，解决微服务间调用的一致性问题

AT模式(业务侵入小)

Seata AT模式是基于XA事务演进而来的一个分布式事务中间件，

XA是一个基于数据库实现的分布式事务协议，本质上和两阶段提交一样，需要数据库支持，Mysql5.6以上版本支持XA协议，其他数据库如Oracle，DB2也实现了XA接口

AT模式角色如下



1. Transaction Coordinator (TC):

事务协调器，维护全局事务的运行状态，负责协调并驱动全局事务的提交或回滚

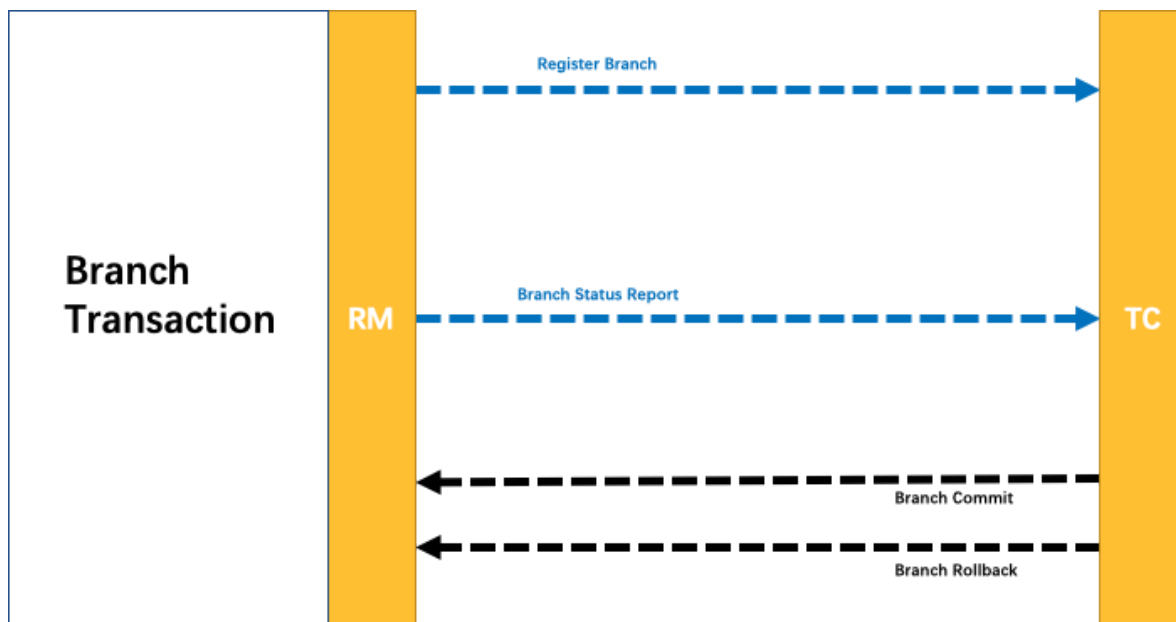
2. Transaction Manager™:

控制全局事务的边界，负责开启一个全局事务，并最终发起全局提交或全局回滚的决议

1. Resource Manager (RM):

控制分支事务，负责分支注册、状态汇报，并接收事务协调器的指令，驱动分支（本地）事务的提交和回滚

AT模式(2PC)基本处理逻辑如下

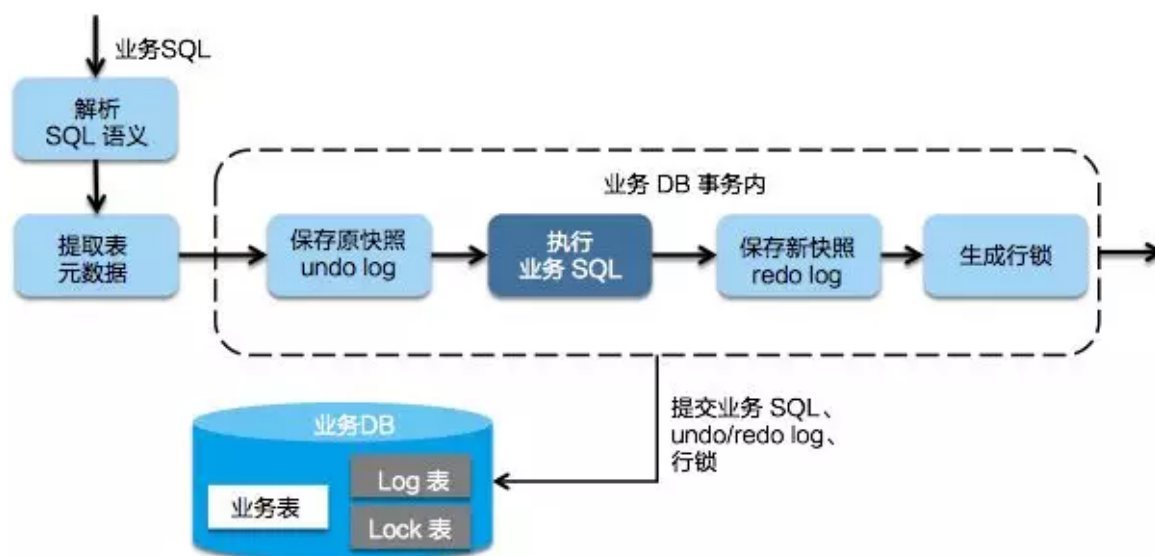


Branch就是指的分布式事务中每个独立的本地局部事务

AT模式第一阶段

Seata 的 JDBC 数据源代理通过对业务 SQL 的解析，把业务数据在更新前后的数据镜像组织成回滚日志，利用 本地事务 的 ACID 特性，将业务数据的更新和回滚日志的写入在同一个 本地事务 中提交。

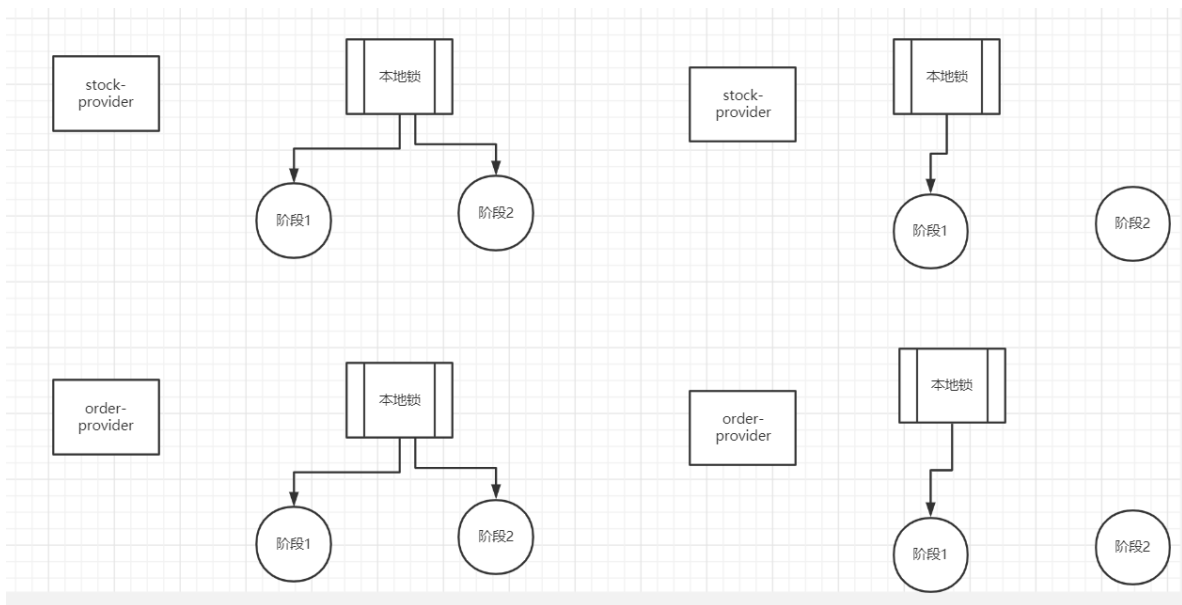
这样，可以保证：**任何提交的业务数据的更新一定有相应的回滚日志存在**



基于这样的机制，分支的本地事务便可以在全局事务的第一阶段提交，并马上释放本地事务锁定的资源

这也是Seata和XA事务的不同之处：

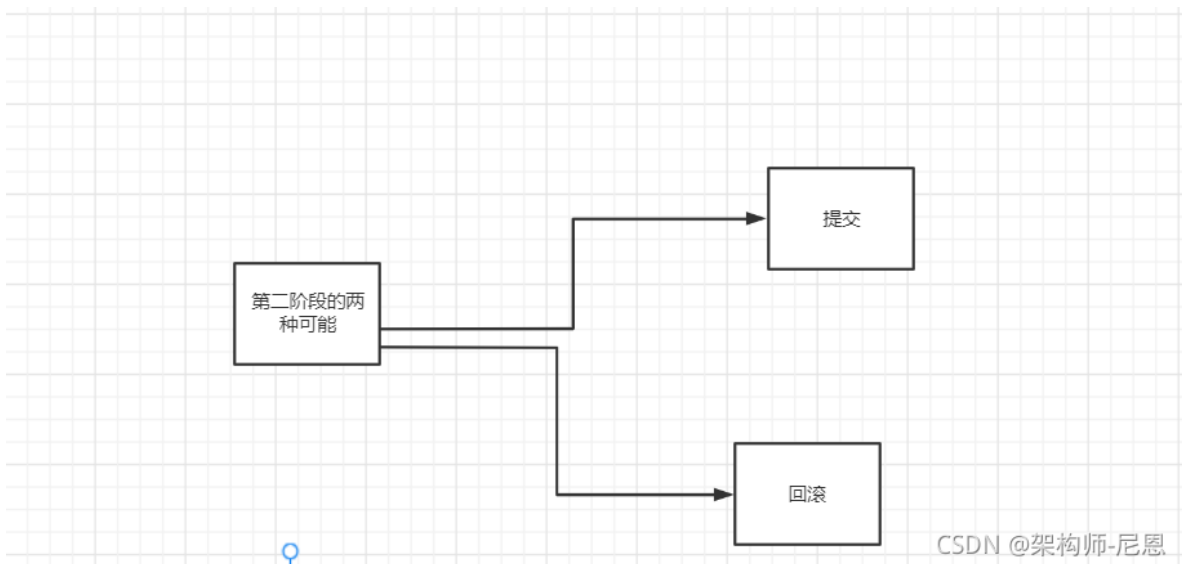
经典的2PC两阶段提交（XA）往往对**资源的锁定需要持续到第二阶段实际的提交或者回滚操作**，



AT模式，可以在第一阶段释放对资源的锁定，降低了锁范围

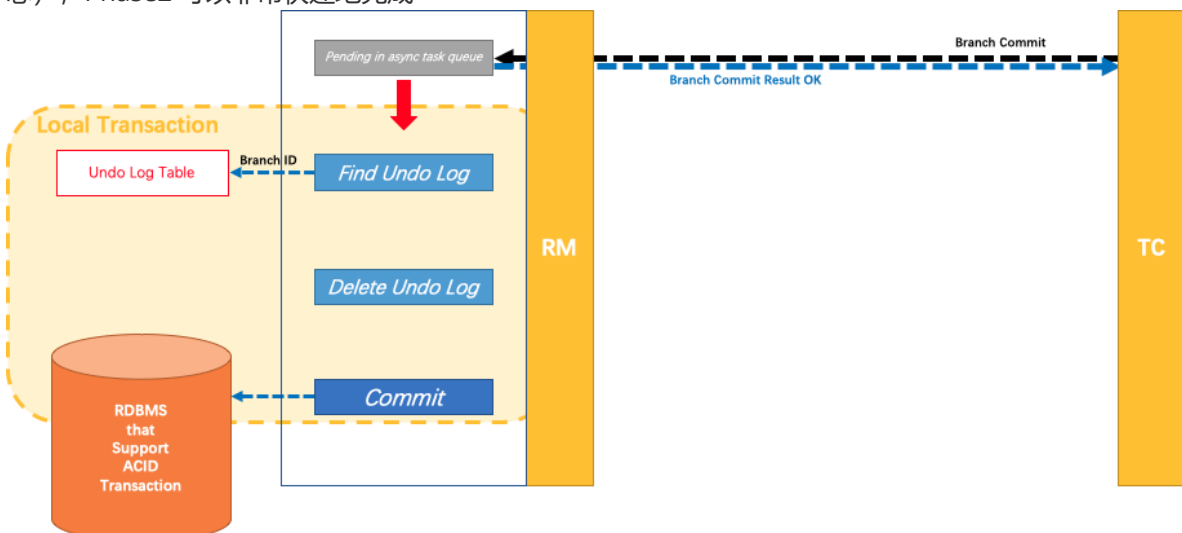
谁的功劳：回滚日志

AT模式第二阶段



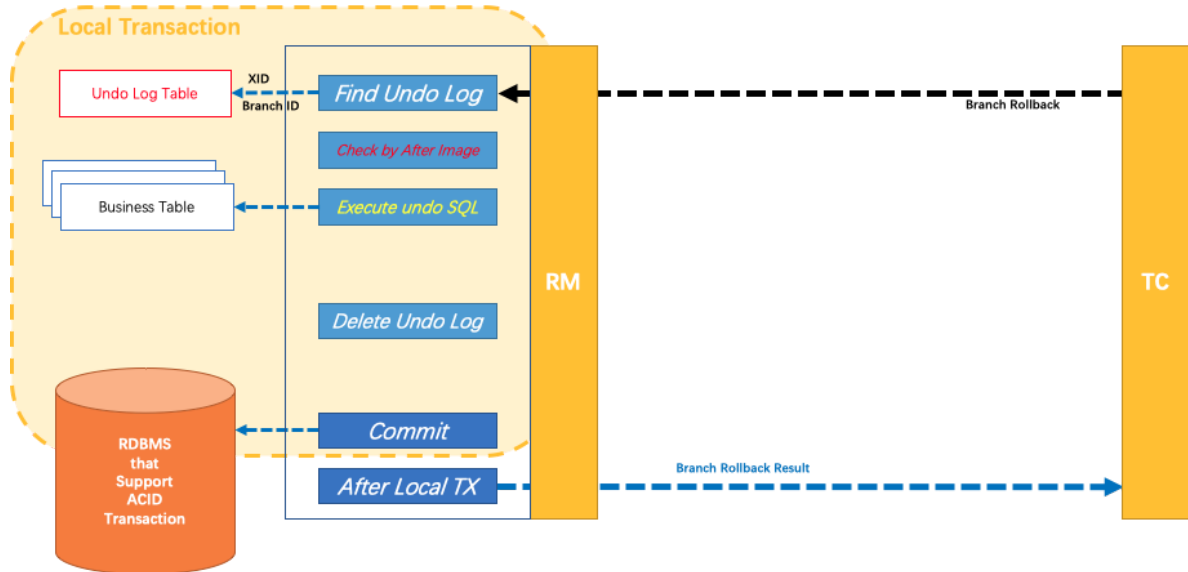
场景一：提交，全局提交

如果决议是**全局提交**，此时分支事务此时已经完成提交，不需要同步协调处理（只需要异步清理回滚日志），Phase2 可以非常快速地完成

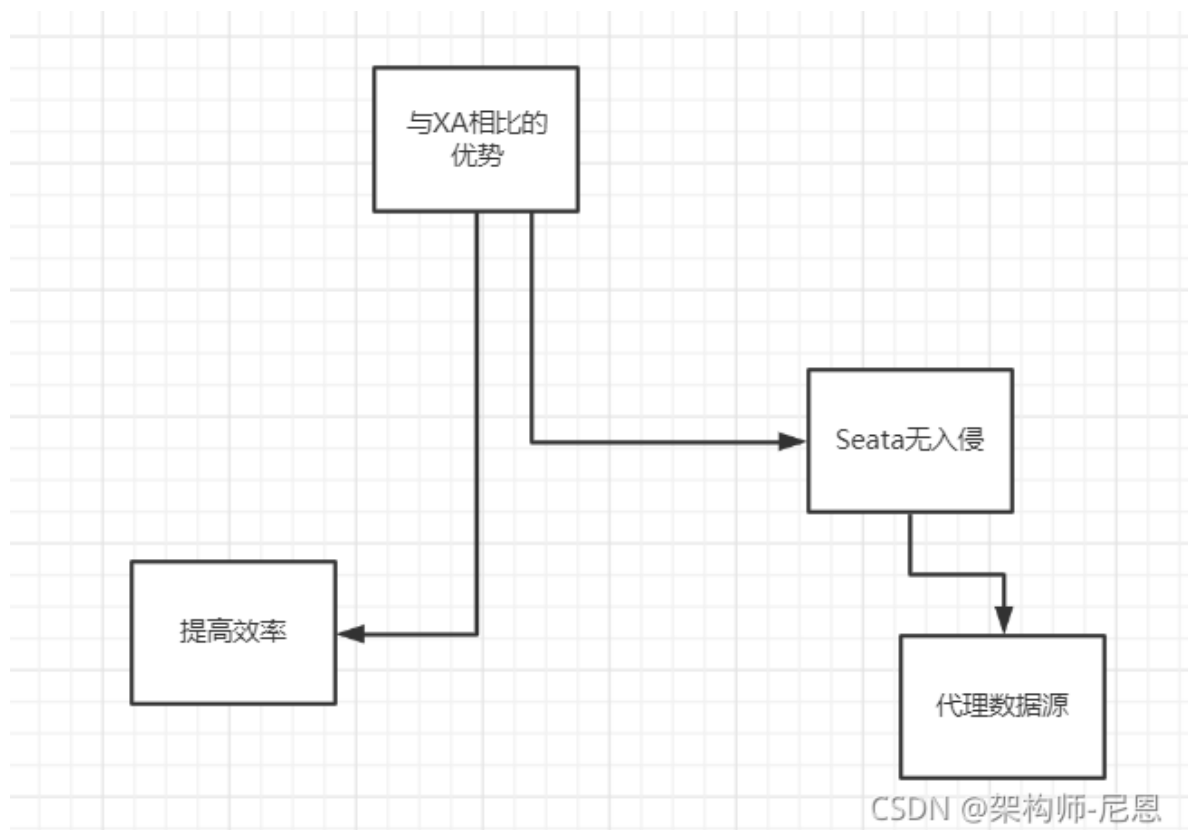


场景2：回滚，全局回滚

如果决议是全局回滚，RM 收到协调器发来的回滚请求，通过 XID 和 Branch ID 找到相应的回滚日志记录，**通过回滚记录生成反向的更新 SQL 并执行**，以完成分支的回滚



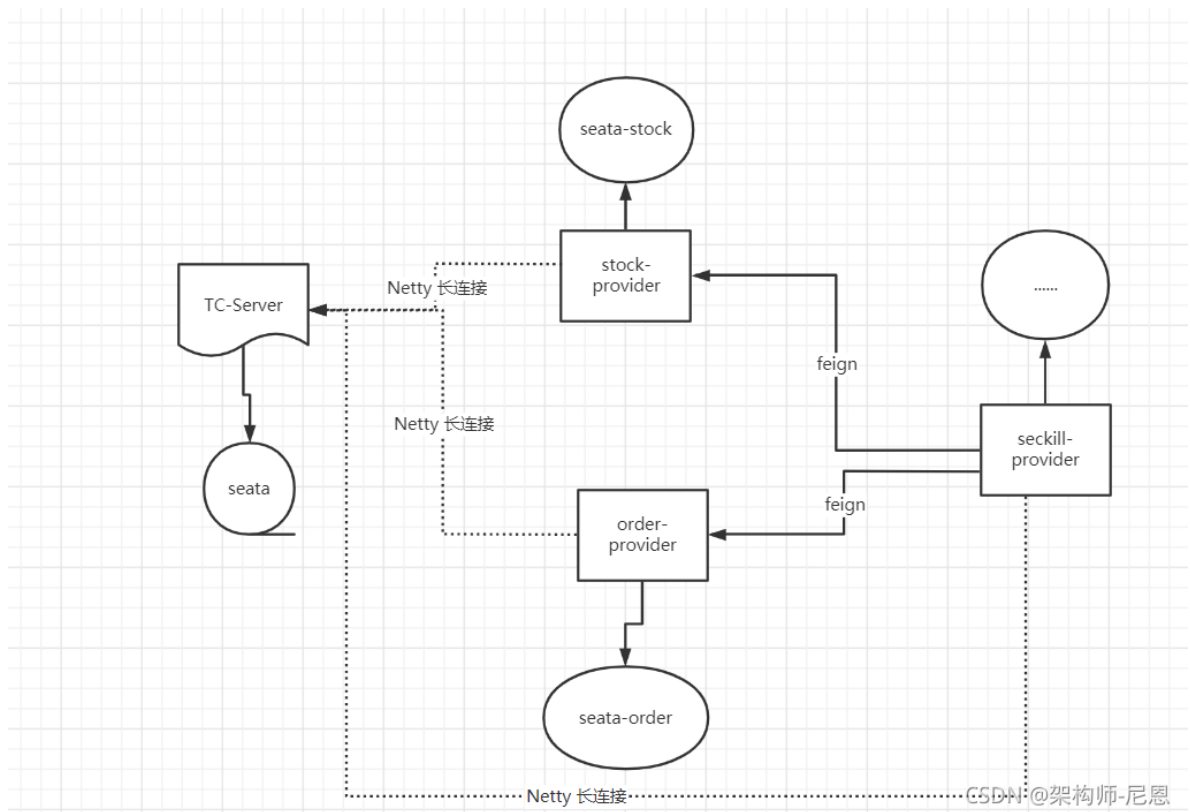
AT模式相对于XA模式的优势



提高效率，即使第二阶段发生异常需要回滚，只需找对undolog中对应数据并反解析成sql来达到回滚目的

同时Seata无入侵，通过代理数据源将业务sql的执行解析成undolog来与业务数据的更新同时入库，达到了对业务无侵入的效果

10WQPS秒杀实操的AT分布式事务架构



快速开始搭建Seata环境

全局事务和分支事务的存储模式

因为 TC 需要进行全局事务和分支事务的记录，所以需要对应的存储。

目前，TC 有两种存储模式(store.mode)：

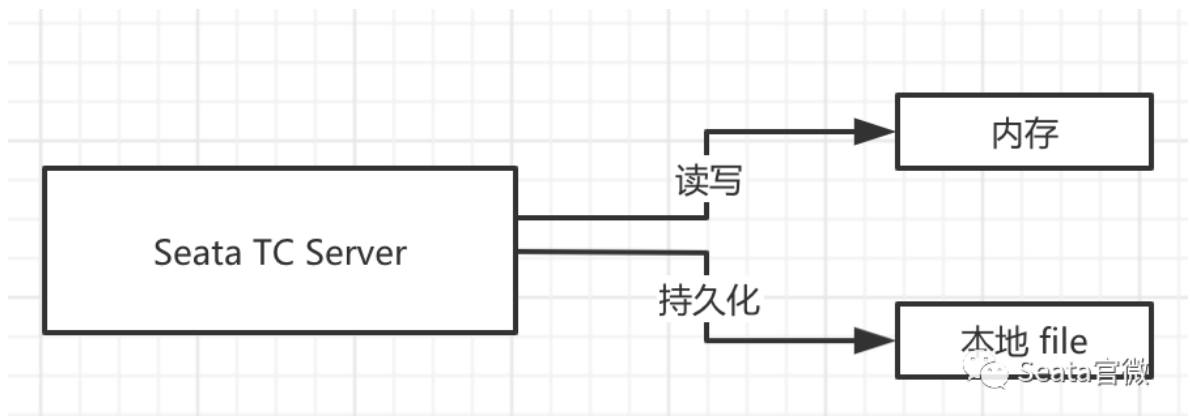
- file 模式：适合单机模式，全局事务会话信息在内存中读写，并持久化本地文件 root.data，性能较高。
- db 模式：适合集群模式，全局事务会话信息通过 db 共享，相对性能差点。

file 模式

全局事务会话信息在内存中读写

并持久化本地文件 root.data

file 模式，最终部署单机 TC Server 如下图所示：

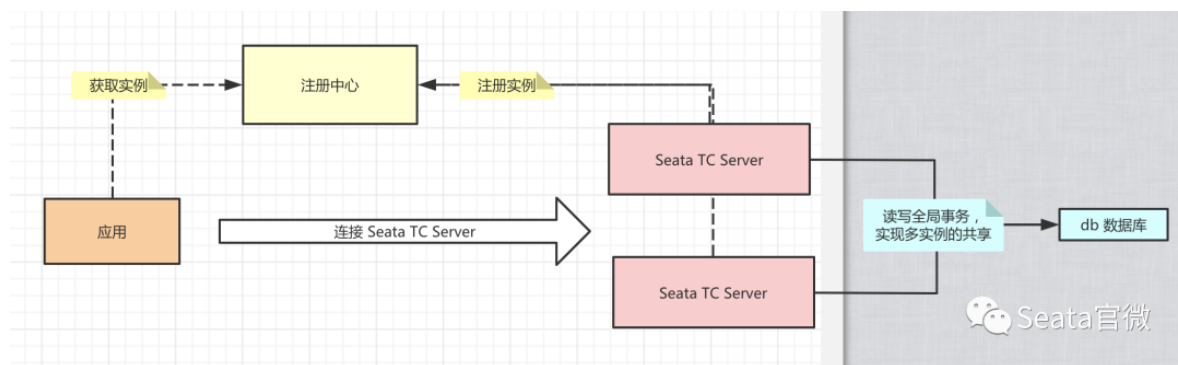


db 模式

集群 Seata TC Server，实现高可用，生产环境下必备。

在集群时，多个 Seata TC Server 通过 db 数据库，实现全局事务会话信息的共享。

每个 Seata TC Server 可以注册自己到注册中心上，方便应用从注册中心获得他们。集群 TC Server 如下图所示：



Seata TC Server 对主流的注册中心都提供了集成。国内使用 Nacos 作为注册中心越来越流行，推荐使用nacos。

#####

配置和启动Seata-server服务（TC服务）

seata 官方文档地址：

<http://seata.io/zh-cn/docs/overview/what-is-seata.html>

下载seata-server-1.3.0和seata-1.3.0源码

seate-server下载： <https://seata.io/zh-cn/blog/download.html>,

seata-1.3.0源码下载：

<https://github.com/seata/seata/releases>

<https://github.com/seata/seata/releases/download/v1.3.0/seata-server-1.3.0.tar.gz>

<https://gitee.com/seata-io/seata.git>

下载所有的源码后，切换到1.3的分支

seata-server包下载和解压

第一步:<https://github.com/seata/seata/releases> 下载seata-server包

推荐使用1.3.0版本，最新版本有些配套的依赖包，不一定来得及更新，可能会出现一些奇怪的问题

seata-server-1.3 上传后，大致如下：

```
[root@cdh1 ~]# cd /work/
[root@cdh1 work]# ll
total 333360

-rw-r--r--  1 root root 33959771 Sep 13 16:00 seata-server-1.3.0.tar.gz

drwxr-xr-x  5 root root      87 Dec 26 2020 zookeeper
```

然后解压缩，tar 命令如下：

```
[root@cdh1 ~]# rm -rf /work/seata
[root@cdh1 ~]# cd /work/
[root@cdh1 work]# tar -zxvf seata-server-1.3.0.tar.gz
seata/LICENSE
seata/conf/
seata/conf/META-INF/
seata/conf/META-INF/services/
seata/conf/logback.xml
seata/conf/file.conf
seata/conf/registry.conf
seata/conf/META-INF/services/io.seata.server.session.SessionManager
seata/conf/META-INF/services/io.seata.core.rpc.RegisterCheckAuthHandler
seata/conf/META-INF/services/io.seata.core.store.db.DataSourceProvider
seata/conf/META-INF/services/io.seata.server.coordinator.AbstractCore
seata/conf/META-INF/servi.....
```

启动 TC Server（单体的实例）

bin下执行 `sh ./seata-server.sh` 命令，启动 TC Server 在后台。

我们看到如下日志，说明启动成功：

```
[root@cdh1 bin]# cd /work/seata/bin/
[root@cdh1 bin]# sh bin/seata-server.sh
sh: bin/seata-server.sh: No such file or directory
[root@cdh1 bin]# sh ./seata-server.sh
Java HotSpot(TM) 64-Bit Server VM warning: Cannot open file
/work/seata/logs/seata_gc.log due to No such file or directory

.....
2021-09-15 04:47:36.143 WARN
[main]i.s.c.l.EnhancedServiceLoader$InnerEnhancedServiceLoader.loadFile:482 -The
same extension io.seata.server.storage.file.lock.FileLockManager has already
been loaded, skipped
2021-09-15 04:47:36.248 INFO
[main]io.seata.core.rpc.netty.RpcServerBootstrap.start:155 -Server started ...
```

- 默认配置下，Seata TC Server 启动在 8091 端点。

使用 File 存储器全局会话

因为我们没有修改任何配置文件，默认情况seata使用的是file模式进行数据持久化，所以可以看到用于持久化的本地文件 `root.data`；操作命令如下：

```
[root@cdh1 bin]# ll -ls /work/seata/bin/sessionStore/
total 0
0 -rw-r--r-- 1 root root 0 Sep 15 11:47 root.data
```

seata-server的TC端配置

file.conf 配置文件，是RM(各大微服务的)和TC之间的通信配置

TC端的参数清单，大致如下：

key	desc	remark
transaction.undo.log.save.days	undo保留天数	默认7天,log_status=1（附录3）和未正常清理的undo
transaction.undo.log.delete.period	undo清理线程间隔时间	默认86400000，单位毫秒
service.max.commit.retry.timeout	二阶段提交重试超时时长	单位ms,s,m,h,d,对应毫秒,秒,分,小时,天,默认毫秒。默认值-1表示无限重试。公式: timeout>=now-globalTransactionBeginTime,true表示超时则不再重试
service.max.rollback.retry.timeout	二阶段回滚重试超时时长	同commit
recovery.committing-retry-period	二阶段提交未完成状态全局事务重试提交线程间隔时间	默认1000，单位毫秒
recovery.asyn-committing-retry-period	二阶段异步提交状态重试提交线程间隔时间	默认1000，单位毫秒
recovery.rollbacking-retry-period	二阶段回滚状态重试回滚线程间隔时间	默认1000，单位毫秒
recovery.timeout-retry-period	超时状态检测重试线程间隔时间	默认1000，单位毫秒，检测出超时将全局事务置入回滚会话管理器
store.mode	事务会话信息存储方式	file本地文件(不支持HA)，db数据库(支持HA)
store.file.dir	file模式文件存储文件夹名	默认sessionStore
store.db.datasource	db模式数据源类型	默认dbcp
store.db.db-type	db模式数据库类型	默认mysql
store.db.driver-class-name	db模式数据库驱动	默认com.mysql.jdbc.Driver
store.db.url	db模式数据源库url	默认jdbc:mysql://127.0.0.1:3306/seata
store.db.user	db模式数据库账户	默认mysql
store.db.min-conn	db模式数据库初始连接数	默认1
store.db.max-conn	db模式数据库最大连接数	默认3

key	desc	remark
store.db.global.table	db模式全局事务表名	默认global_table
store.db.branch.table	db模式分支事务表名	默认branch_table
store.db.lock-table	db模式全局锁表名	默认lock_table
store.db.query-limit	db模式查询全局事务一次的最大条数	默认1000
metrics.enabled	是否启用Metrics	默认false关闭，在False状态下，所有与Metrics相关的组件将不会被初始化，使得性能损耗最低
metrics.registry-type	指标注册器类型	Metrics使用的指标注册器类型，默认为内置的compact（简易）实现，这个实现中的Meter仅使用有限内存计数，性能高足够满足大多数场景；目前只能设置一个指标注册器实现
metrics.exporter-list	指标结果Measurement数据输出器列表	默认prometheus，多个输出器使用英文逗号分割，例如"prometheus,jmx"，目前仅实现了对接prometheus的输出器
metrics.exporter-prometheus-port	prometheus输出器Client端口号	默认9898

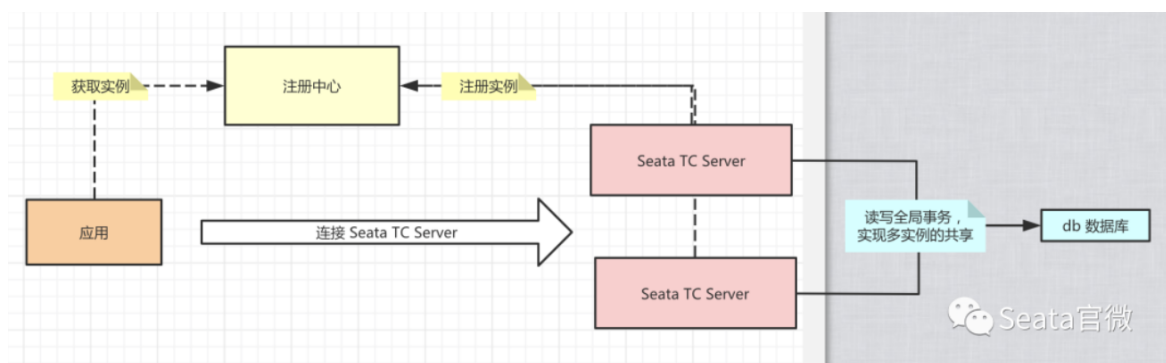
部署集群 TC Server

本小节，我们来学习部署集群 Seata TC Server，实现高可用，生产环境下必备。

在集群时，多个 Seata TC Server 通过 db 数据库，实现全局事务会话信息的共享。

每个 Seata TC Server 可以注册自己到注册中心上，方便应用从注册中心获得到他们。

最终我们部署 集群 TC Server 如下图所示：



使用 Nacos 作为注册中心

修改 `conf/registry.conf` 配置文件，设置使用 Nacos 注册中心。如下图所示：

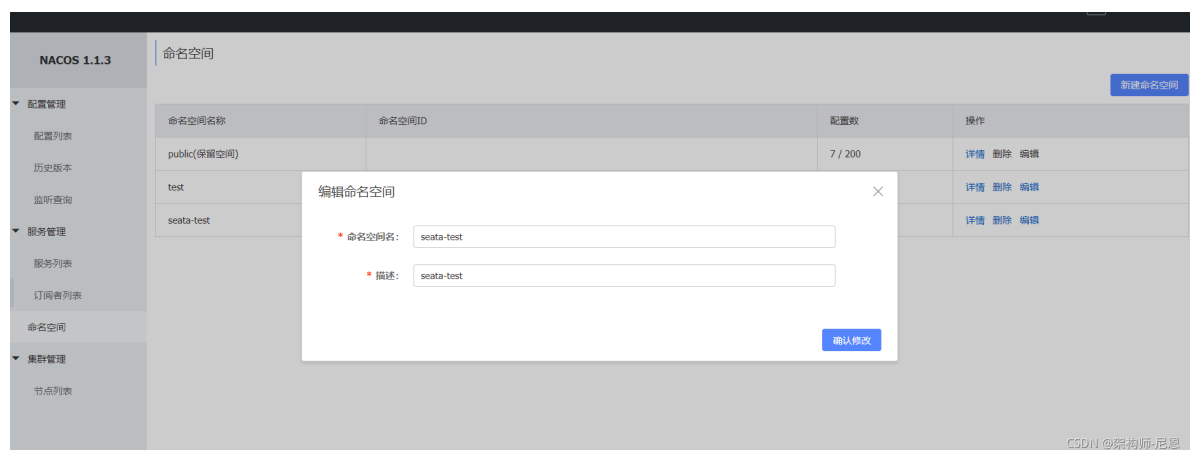

```

registry {
    # file 、nacos 、eureka、redis、zk、consul、etcd3、sofa
    # type = "file"
    type = "nacos"

    nacos {
        application = "seata-server"
        serverAddr = "localhost"
        namespace = ""
        cluster = "default"
        username = "nacos"
        password = "nacos"
    }
    eureka {
        serviceUrl = "http://localhost:8761/eureka"
        application = "default"
        weight = "1"
    }
    redis {
        serverAddr = "localhost:6379"
        db = 0
        password = ""
        cluster = "default"
        timeout = 0
    }
    zk {
        cluster = "default"
        serverAddr = "127.0.0.1:2181"
        sessionTimeout = 6000
        connectTimeout = 2000
        username = ""
        password = ""
    }
    consul {
        cluster = "default"
        serverAddr = "127.0.0.1:8500"
    }
    etcd3 {
        cluster = "default"
        serverAddr = "http://localhost:2379"
    }
    sofa {
        serverAddr = "127.0.0.1:9603"
        application = "default"
        region = "DEFAULT_ZONE"
        datacenter = "DefaultDataCenter"
        cluster = "default"
        group = "SEATA_GROUP"
        addressWaitTime = "3000"
    }
    file {
        name = "file.conf"
    }
}

```

如果设置的 -t 参数，表示设置了命名空间，则需要提前创建命名空间。然后拿到其 命名空间ID----uuid。



TC端数据库初始化

下载TC端的配置文件，可以从github：

<https://github.com/seata-io/seata/tree/v1.3.0/script/server/db>

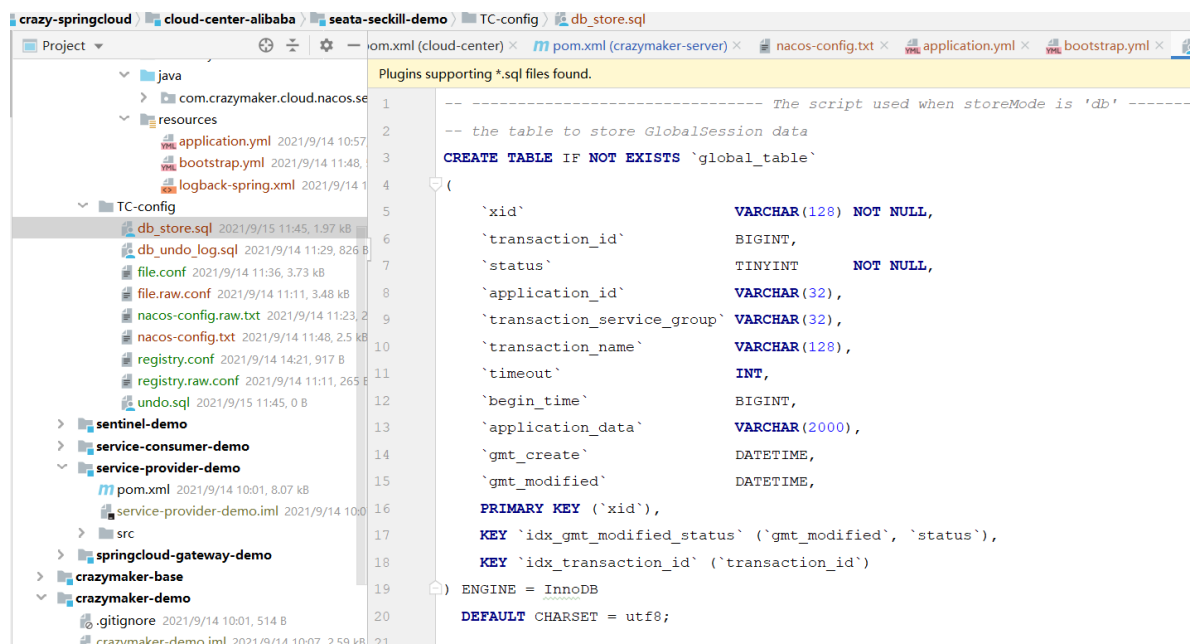
或者 gitee：

<https://gitee.com/seata-io/seata/tree/v1.3.0/script/server/db>

下载Seata1.3的版本对应的脚本。目前支持mysql、oracle、postgresql这三种数据库，

上述三种脚本是针对Seata的Sever端在协调处理分布式事务时所需要的3张表，提供了不同数据库的 global_table表、branch_table表、lock_table表创建脚本，根据自身数据库执行对应的sql脚本执行即可。

这里以mysql为例，在你的mysql数据库中创建名为seata的库，并执行以下sql，将会生成三张表：global_table表、branch_table表、lock_table表



创建TC端的专属独立库seata。

连接MYSQL：

格式：mysql -h主机地址 -u用户名 -p用户密码

```
mysql -h192.168.9.1 -uroot -p123456

mysql -uroot -p123456
123
/usr/bin/mysql -uroot -p"123456" --connect-expired-password <<EOF
CREATE DATABASE `seata` CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
show databases;
grant all privileges on seata.* to root@'%' identified by '123456' WITH
GRANT OPTION;
flush privileges;
EOF
```

授权的时候，如果出现以下提示：

ERROR 1819 (HY000): Your password does not satisfy the current policy requirements
那么导致问题的原因就是密码规则没有修改喽，以下提供两种方法更改简单密码：

1.在/etc/my.cnf关闭密码规则

validate_password = off

2.进入mysql以后修改密码规则

```
set global validate_password_policy=0; -----把密码策略设置成0
set global validate_password_length=0; -----把密码长度限制设置成0
flush privileges; -----刷新权限使其生效
```

需要在刚才配置的数据库中执行数据初始脚本 db_store.sql，这个是全局事务控制的表，需要提前初始化。

db_store.sql 为seata库必须的表，执行sql即可

通过 source 命令，导致sql脚本就可以了。

mysql> source /work/seata/conf/db_store.sql;

```
mysql> use seata;
Database changed
mysql> source /work/seata/conf/db_store.sql;
Query OK, 0 rows affected, 1 warning (0.00 sec)

Query OK, 0 rows affected (0.02 sec)

Query OK, 0 rows affected, 1 warning (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

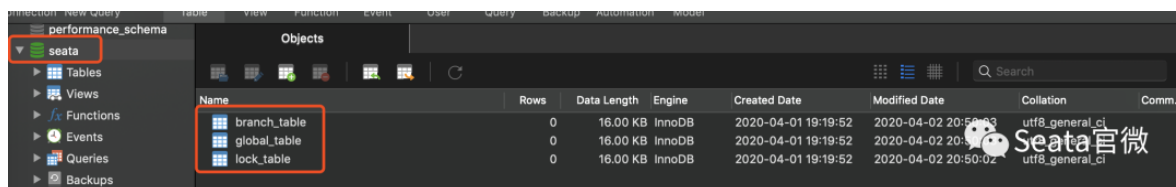
Query OK, 0 rows affected, 1 warning (0.00 sec)

Query OK, 0 rows affected (0.01 sec)
```

这里我们只是做演示，理论上上面三个业务服务应该分属不同的数据库，这里我们只是在同一台数据库下面创建三个 Schema，分别为 db_account、db_order 和 db_storage，具体如图：

```
mysql> show tables;
+-----+
| Tables_in_seata |
+-----+
| branch_table    |
| global_table    |
| lock_table      |
+-----+
3 rows in set (0.00 sec)
```

通过windows 工具操作，也是类似的，导入 db_store.sql之后，创建了3个表 db_account、db_order 和 db_storage，具体如图：



db 数据库共享全局事务会话信息：

如果使用file作为配置文件

修改 `conf/file.conf` 配置文件，修改使用 db 数据库，实现 Seata TC Server 的全局事务会话信息的共享。

如下图所示：

```
## transaction log store, only used in seata-server
store {
    ## store mode: file、db
    ## mode = "file"
    mode = "db"

    ## file store property
    file {
        ## store location dir
        dir = "sessionStore"
        # branch session size , if exceeded first try compress lockkey, still
        exceeded throws exceptions
        maxBranchSessionSize = 16384
        # globe session size , if exceeded throws exceptions
        maxGlobalSessionSize = 512
        # file buffer size , if exceeded allocate new buffer
        filewriteBufferCacheSize = 16384
        # when recover batch read size
        sessionReloadReadSize = 100
        # async, sync
        flushDiskMode = async
    }
}
```

```

## database store property
db {
    ## the implement of javax.sql.DataSource, such as
    DruidDataSource(druid)/BasicDataSource(dbcp) etc.
    datasource = "druid"
    ## mysql/oracle/postgresql/h2/oceanbase etc.
    dbType = "mysql"
    ## driverClassName = "com.mysql.jdbc.Driver"
    driverClassName = "com.mysql.jdbc.Driver"
    url = "jdbc:mysql://192.168.56.121:3306/seata?
useUnicode=true&characterEncoding=utf8&allowMultiQueries=true&useSSL=true&server
Timezone=UTC"
    user = "root"
    password = "123456"
    minConn = 5
    maxConn = 30
    globalTable = "global_table"
    branchTable = "branch_table"
    lockTable = "lock_table"
    queryLimit = 100
    maxWait = 5000
}
}

```

如何使用nacos作为配置中心:

修改使用 db 数据库, 实现 Seata TC Server 的全局事务会话信息的共享。

```

transport.type=TCP
transport.server=NIO
transport.heartbeat=true
transport.enableClientBatchSendRequest=false
transport.threadFactory.bossThreadPrefix=NettyBoss
transport.threadFactory.workerThreadPrefix=NettyServerNIOWorker
transport.threadFactory.serverExecutorThreadPrefix=NettyServerBizHandler
transport.threadFactory.shareBossWorker=false
transport.threadFactory.clientSelectorThreadPrefix=NettyClientSelector
transport.threadFactory.clientSelectorThreadSize=1
transport.threadFactory.clientWorkerThreadPrefix=NettyClientWorkerThread
transport.threadFactory.bossThreadSize=1
transport.threadFactory.workerThreadSize=default
transport.shutdown.wait=3
service.vgroupMapping.my_test_tx_group=default
service.vgroupMapping.seata-seckill-demo-seata-service-group=default
service.vgroupMapping.seata-order-demo-seata-service-group=default
service.vgroupMapping.seata-stock-demo-seata-service-group=default
service.default.grouplist=cdh1:8091
service.enableDegradе=false
service.disableGlobalTransaction=false
client.rm.asyncCommitBufferLimit=10000
client.rm.lock.retryInterval=10
client.rm.lock.retryTimes=30
client.rm.lock.retryPolicyBranchRollbackOnConflict=true
client.rm.reportRetryCount=5
client.rm.tableMetaCheckEnable=false

```

```

client.rm.sqlParserType=druid
client.rm.reportSuccessEnable=false
client.rm.sagaBranchRegisterEnable=false
client.tm.commitRetryCount=5
client.tm.rollbackRetryCount=5
#store.mode=file
store.mode=db
store.file.dir=file_store/data
store.file.maxBranchSessionSize=16384
store.file.maxGlobalSessionSize=512
store.file.fileWriteBufferCacheSize=16384
store.file.flushDiskMode=async
store.file.sessionReloadReadSize=100
store.db.datasource=druid
store.db.dbType=mysql
store.db.driverClassName=com.mysql.jdbc.Driver
store.db.url=jdbc:mysql://cdh1:3306/seata?useUnicode=true
store.db.user=root
store.db.password=123456
store.db.minConn=5
store.db.maxConn=30
store.db.globalTable=global_table
store.db.branchTable=branch_table
store.db.queryLimit=100
store.db.lockTable=lock_table
store.db.maxWait=5000
server.recovery.committingRetryPeriod=1000
server.recovery.asyncCommittingRetryPeriod=1000
server.recovery.rollbackingRetryPeriod=1000
server.recovery.timeoutRetryPeriod=1000
server.maxCommitRetryTimeout=-1
server.maxRollbackRetryTimeout=-1
server.rollbackRetryTimeoutUnlockEnable=false
client.undo.dataValidation=true
client.undo.logSerialization=jackson
server.undo.logSaveDays=7
server.undo.logDeletePeriod=86400000
client.undo.logTable=undo_log
client.log.exceptionRate=100
transport.serialization=seata
transport.compressor=none
metrics.enabled=false
metrics.registryType=compact
metrics.exporterList=prometheus
metrics.exporterPrometheusPort=9898

```

补充: MySQL8 的支持

如果使用的 MySQL 是 8.X 版本, 需要下载 MySQL 8.X JDBC 驱动, 命令行操作如下:

```

$ cd lib
$ wget https://repo1.maven.org/maven2/mysql/mysql-connector-java/8.0.19/mysql-connector-java-8.0.19.jar

```

然后, 修改 conf/file 配置文件, 使用该 MySQL 8.X JDBC 驱动。如下图所示:

```
## database store property
db {
    ## the implement of javax.sql.DataSource, such as DruidDataSource(druid)/BasicDataSource(dbcp) etc
    datasource = "dbcp"
    ## mysql/oracle/h2/oceanbase etc.
    dbType = "mysql"
    #driverClassName = "com.mysql.jdbc.Driver"
    driverClassName = "com.mysql.cj.jdbc.Driver"
    url = "jdbc:mysql://127.0.0.1:3306/seata"
    user = "root"
    password = ""
    minConn = 1
}
```

修改为 MySQL 8.X JDBC 驱动

启动 第一个TC Server

执行 `nohup sh bin/seata-server.sh -p 18091 -n 1 &` 命令，启动第一个 TC Server 在后台。

-p: Seata TC Server 监听的端口。

-n: Server node。在多个 TC Server 时，需区分各自节点，用于生成不同区间的 transactionId 事务编号，以免冲突。

```
nohup sh /work/seata/bin/seata-server.sh -p 18091 -n 1 >
/work/seata/bin/consol.log &

tail -f /work/seata/bin/consol.log
```

在 consol.log 文件中，我们看到如下日志，说明启动成功：

```
e.session.FileSessionManager has already been loaded, skipped
2021-09-15 07:46:33.472 WARN
[main]i.s.c.l.EnhancedServiceLoader$InnerEnhancedServiceLoader.loadFile:482 -The
same extension io.seata.server.storage.db.session.DataBaseSessionManager has
already been loaded, skipped
.....
2021-09-15 07:46:34.094 WARN
[main]i.s.c.l.EnhancedServiceLoader$InnerEnhancedServiceLoader.loadFile:482 -The
same extension io.seata.server.storage.file.lock.FileLockManager has already
been loaded, skipped
2021-09-15 07:46:34.220 INFO
[main]io.seata.core.rpc.netty.RpcServerBootstrap.start:155 -Server started ...
```

启动 第2个TC Server

执行 `nohup sh bin/seata-server.sh -p 18092 -n 2 &` 命令，启动第二个 TC Server 在后台。

```
nohup sh /work/seata/bin/seata-server.sh -p 18092 -n 1 >
/work/seata/bin/consol2.log &

tail -f /work/seata/bin/consol2.log
```

实验演示：注册到nacos之后，可以在nacos上看效果

控制台提交nacos配置脚本

下载nacos的配置文件，原始config.txt文件可以从github：

<https://github.com/seata/seata/tree/develop/script/config-center>

或者 gitee：

<https://gitee.com/seata-io/seata/tree/develop/script/config-center>

下载的并修改：

```
transport.type=TCP
transport.server=NIO
transport.heartbeat=true
transport.enableClientBatchSendRequest=true
transport.threadFactory.bossThreadPrefix=NettyBoss
transport.threadFactory.workerThreadPrefix=NettyServerNIOWorker
transport.threadFactory.serverExecutorThreadPrefix=NettyServerBizHandler
transport.threadFactory.shareBossWorker=false
transport.threadFactory.clientSelectorThreadPrefix=NettyClientSelector
transport.threadFactory.clientSelectorThreadSize=1
transport.threadFactory.clientWorkerThreadPrefix=NettyClientWorkerThread
transport.threadFactory.bossThreadSize=1
transport.threadFactory.workerThreadSize=default
transport.shutdown.wait=3
service.vgroup_mapping.my_test_tx_group=default
service.vgroup_mapping.seata-seckill-demo=default
service.vgroup_mapping.seata-order-demo=default
service.vgroup_mapping.seata-stock-demo=default
service.default.grouplist=127.0.0.1:8091
service.enabledDegrade=false
service.disableGlobalTransaction=false
client.rm.asyncCommitBufferLimit=10000
client.rm.lock.retryInterval=10
client.rm.lock.retryTimes=30
client.rm.lock.retryPolicyBranchRollbackOnConflict=true
client.rm.reportRetryCount=5
client.rm.tableMetaCheckEnable=false
client.rm.tableMetaCheckerInterval=60000
client.rm.sqlParserType=druid
client.rm.reportSuccessEnable=false
client.rm.sagaBranchRegisterEnable=false
client.rm.tccActionInterceptorOrder=-2147482648
client.tm.commitRetryCount=5
client.tm.rollbackRetryCount=5
client.tm.defaultGlobalTransactionTimeout=60000
client.tm.degradeCheck=false
client.tm.degradeCheckAllowTimes=10
client.tm.degradeCheckPeriod=2000
client.tm.interceptorOrder=-2147482648
store.mode=file
store.lock.mode=file
store.session.mode=file
store.publickey=
store.file.dir=file_store/data
store.file.maxBranchSessionSize=16384
```



```
store.file.maxGlobalSessionSize=512
store.file.fileWriteBufferCacheSize=16384
store.file.flushDiskMode=async
store.file.sessionReloadReadSize=100
store.db.datasource=druid
store.db.dbType=mysql
store.db.driverClassName=com.mysql.jdbc.Driver
store.db.url=jdbc:mysql://cdh1:3306/seata?
useUnicode=true&rewriteBatchedStatements=true
store.db.user=root
store.db.password=123456
store.db.minConn=5
store.db.maxConn=30
store.db.globalTable=global_table
store.db.branchTable=branch_table
store.db.queryLimit=100
store.db.lockTable=lock_table
store.db.maxWait=5000
store.redis.mode=single
store.redis.single.host=cdh1
store.redis.single.port=6379
store.redis.sentinel.masterName=
store.redis.sentinel.sentinelHosts=
store.redis.maxConn=10
store.redis.minConn=1
store.redis.maxTotal=100
store.redis.database=0
store.redis.password=123456
store.redis.queryLimit=100
server.recovery.committingRetryPeriod=1000
server.recovery.asynCommittingRetryPeriod=1000
server.recovery.rollbackingRetryPeriod=1000
server.recovery.timeoutRetryPeriod=1000
server.maxCommitRetryTimeout=-1
server.maxRollbackRetryTimeout=-1
server.rollbackRetryTimeoutUnlockEnable=false
server.distributedLockExpireTime=10000
client.undo.dataValidation=true
client.undo.logSerialization=jackson
client.undo.onlyCareUpdateColumns=true
server.undo.logSaveDays=7
server.undo.logDeletePeriod=86400000
client.undo.logTable=undo_log
client.undo.compress.enable=true
client.undo.compress.type=zip
client.undo.compress.threshold=64k
log.exceptionRate=100
transport.serialization=seata
transport.compressor=none
metrics.enabled=false
metrics.registryType=compact
metrics.exporterList=prometheus
metrics.exporterPrometheusPort=9898
```

修改 自己定义的服务组

在 nacos-config.txt 文件 修改 自己定义的服务组，参考如下：

```
service.vgroup_mapping.my_test_tx_group=default  
service.vgroup_mapping.seata-seckill-demo=default  
service.vgroup_mapping.seata-order-demo=default  
service.vgroup_mapping.seata-stock-demo=default
```

中间的\${your-service-group}为自己定义的服务组名称，这里需要我们在程序的配置文件中配置，笔者这里直接使用程序的 spring.application.name。

修改数据库连接

用到2个文件，nacos-config.txt和nacos-config.sh，都在config目录下。

config.txt需要：

- 修改数据库连接

```
store.db.datasource=druid  
store.db.dbType=mysql  
store.db.driverClassName=com.mysql.jdbc.Driver # 这里  
store.db.url=jdbc:mysql://127.0.0.1:3306/seata?useUnicode=true # 这里  
store.db.user=username # 这里  
store.db.password=password # 这里
```

将文件中的配置导入nacos 即可

下载nacos的配置文件，原始config.txt文件可以从github：

<https://github.com/seata/seata/tree/develop/script/config-center>

或者 gitee：

<https://gitee.com/seata-io/seata/tree/develop/script/config-center>

提供的nacos脚本nacos-config.sh，将上面的config.txt文件复制到seata的config目录,通过 nacos-config.sh，将文件中的配置导入nacos 即可。执行以下命令：

```
sh /work/seata/script/config-center/nacos/nacos-config.sh -h localhost -p 8848 -  
g SEATA_GROUP -t 5a3c7d6c-f497-4d68-a71a-2e5e3340b3ca -u nacos -w nacos
```

将以上信息提交到nacos控制台，当然，如果有需要修改参数，可直接通过登录nacos控制台修改。

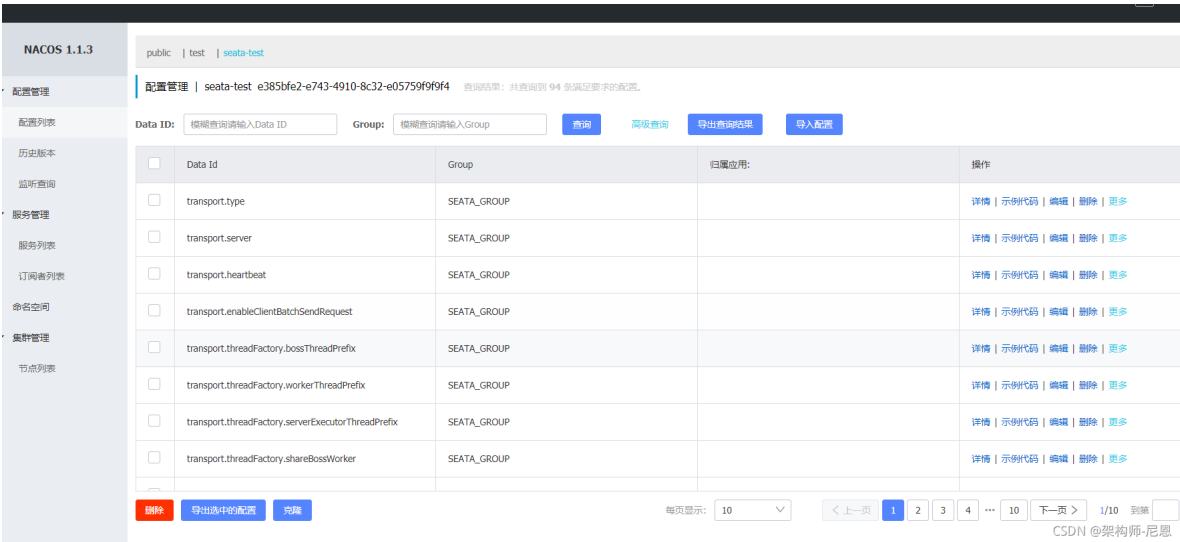
```
[root@cdh1 script]# sh /work/seata/script/nacos-config.sh -h localhost -p 8848 -  
g SEATA_GROUP -t 5a3c7d6c-f497-4d68-a71a-2e5e3340b3ca -u nacos -w nacos  
set nacosAddr=localhost:8848  
set group=SEATA_GROUP  
Set transport.type=TCP successfully  
Set transport.server=NIO successfully  
Set transport.heartbeat=true successfully  
Set transport.enableClientBatchSendRequest=true successfully  
Set transport.threadFactory.bossThreadPrefix=NettyBoss successfully  
Set transport.threadFactory.workerThreadPrefix=NettyServerNIOworker successfully
```

```
Set transport.threadFactory.serverExecutorThreadPrefix=NettyServerBizHandler
successfully
Set transport.threadFactory.shareBossWorker=false successfully
Set transport.threadFactory.clientSelectorThreadPrefix=NettyClientSelector
successfully
Set transport.threadFactory.clientSelectorThreadSize=1 successfully
Set transport.threadFactory.clientWorkerThreadPrefix=NettyClientWorkerThread
successfully
Set transport.threadFactory.bossThreadSize=1 successfully
Set transport.threadFactory.workerThreadSize=default successfully
Set transport.shutdown.wait=3 successfully
Set service.vgroup_mapping.my_test_tx_group=default successfully
Set service.vgroup_mapping.seata-seckill-demo=default successfully
Set service.vgroup_mapping.seata-order-demo=default successfully
Set service.vgroup_mapping.seata-stock-demo=default successfully
Set service.default.grouplist=127.0.0.1:8091 successfully
Set service.enableDegrade=false successfully
Set service.disableGlobalTransaction=false successfully
Set client.rm.asyncCommitBufferLimit=10000 successfully
Set client.rm.lock.retryInterval=10 successfully
Set client.rm.lock.retryTimes=30 successfully
Set client.rm.lock.retryPolicyBranchRollbackOnConflict=true successfully
Set client.rm.reportRetryCount=5 successfully
Set client.rm.tableMetaCheckEnable=false successfully
Set client.rm.tableMetaCheckerInterval=60000 successfully
Set client.rm.sqlParserType=druid successfully
Set client.rm.reportSuccessEnable=false successfully
Set client.rm.sagaBranchRegisterEnable=false successfully
Set client.rm.tccActionInterceptorOrder=-2147482648 successfully
Set client.tm.commitRetryCount=5 successfully
Set client.tm.rollbackRetryCount=5 successfully
Set client.tm.defaultGlobalTransactionTimeout=60000 successfully
Set client.tm.degradeCheck=false successfully
Set client.tm.degradeCheckAllowTimes=10 successfully
Set client.tm.degradeCheckPeriod=2000 successfully
Set client.tm.interceptorOrder=-2147482648 successfully
Set store.mode=file successfully
Set store.lock.mode=file successfully
Set store.session.mode=file successfully
Set store.publicKey= failure
Set store.file.dir=file_store/data successfully
Set store.file.maxBranchSessionSize=16384 successfully
Set store.file.maxGlobalSessionSize=512 successfully
Set store.file.fileWriteBufferCacheSize=16384 successfully
Set store.file.flushDiskMode=async successfully
Set store.file.sessionReloadReadSize=100 successfully
Set store.db.datasource=druid successfully
Set store.db.dbType=mysql successfully
Set store.db.driverClassName=com.mysql.jdbc.Driver successfully
Set store.db.url=jdbc:mysql://cdh1:3306/seata?
useUnicode=true&rewriteBatchedStatements=true successfully
Set store.db.user=root successfully
Set store.db.password=123456 successfully
Set store.db.minConn=5 successfully
Set store.db.maxConn=30 successfully
Set store.db.globalTable=global_table successfully
Set store.db.branchTable=branch_table successfully
Set store.db.queryLimit=100 successfully
```

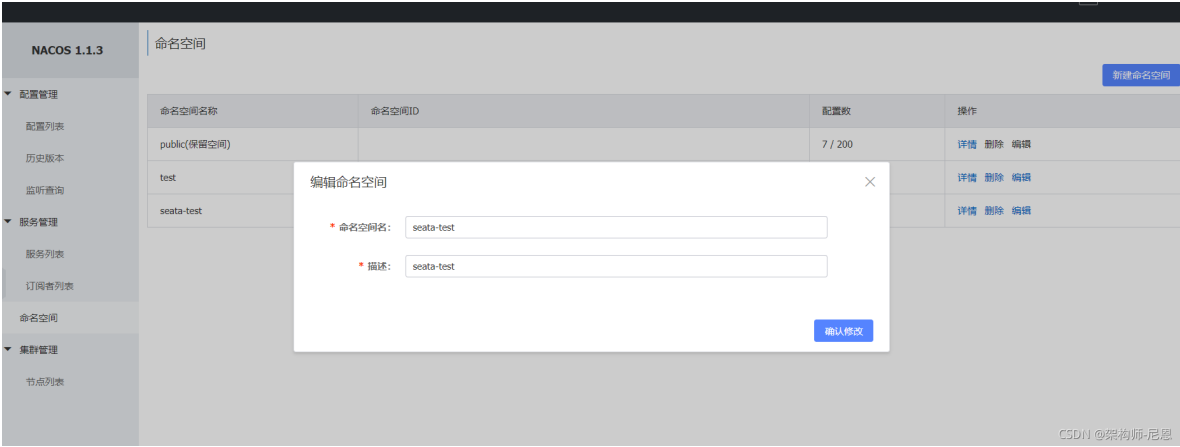
```
Set store.db.lockTable=lock_table successfully
Set store.db.maxwait=5000 successfully
Set store.redis.mode=single successfully
Set store.redis.single.host=cdh1 successfully
Set store.redis.single.port=6379 successfully
Set store.redis.sentinel.masterName= failure
Set store.redis.sentinel.sentinelHosts= failure
Set store.redis.maxConn=10 successfully
Set store.redis.minConn=1 successfully
Set store.redis.maxTotal=100 successfully
Set store.redis.database=0 successfully
Set store.redis.password=123456 successfully
Set store.redis.queryLimit=100 successfully
Set server.recovery.committingRetryPeriod=1000 successfully
Set server.recovery.asynCommittingRetryPeriod=1000 successfully
Set server.recovery.rollbackingRetryPeriod=1000 successfully
Set server.recovery.timeoutRetryPeriod=1000 successfully
Set server.maxCommitRetryTimeout=-1 successfully
Set server.maxRollbackRetryTimeout=-1 successfully
Set server.rollbackRetryTimeoutUnlockEnable=false successfully
Set server.distributedLockExpireTime=10000 successfully
Set client.undo.dataValidation=true successfully
Set client.undo.logSerialization=jackson successfully
Set client.undo.onlyCareUpdateColumns=true successfully
Set server.undo.logSaveDays=7 successfully
Set server.undo.logDeletePeriod=86400000 successfully
Set client.undo.logTable=undo_log successfully
Set client.undo.compress.enable=true successfully
Set client.undo.compress.type=zip successfully
Set client.undo.compress.threshold=64k successfully
Set log.exceptionRate=100 successfully
Set transport.serialization=seata successfully
Set transport.compressor=none successfully
Set metrics.enabled=false successfully
Set metrics.registryType=compact successfully
Set metrics.exporterList=prometheus successfully
Set metrics.exporterPrometheusPort=9898 successfully

=====
Complete initialization parameters, total-count:97 , failure-count:3
=====
init nacos config fail.
```

顺利完成后，nacos中如下



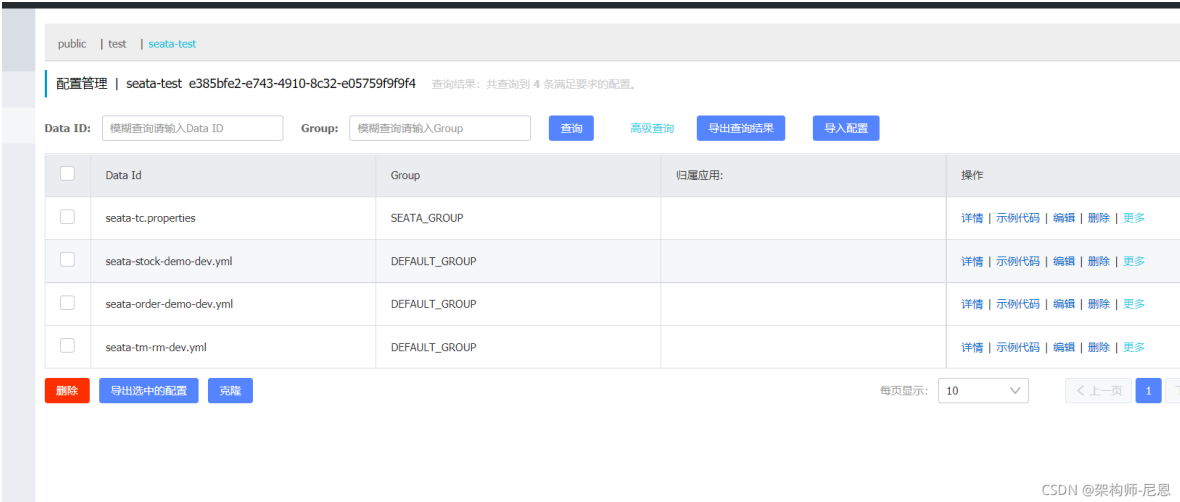
如果设置的 -t 参数，表示设置了命名空间，则需要提前创建命名空间。然后拿到其 命名空间ID----uuid。



直接使用nacos 的data-id配置文件

制作和上传配置文件

把上面的配置，做成一个文件，放在nacos中



config文件中的修改

```
##配置seata-server的注册中心，支持file 、nacos 、eureka、redis、zk、consul、etcd3、sofa

registry {
```

```

# file 、nacos 、eureka、redis、zk、consul、etcd3、sofa
# type = "file"
type = "nacos"

nacos {
    application = "seata-server"
    serverAddr = "cdh1:8848"
    namespace = "e385bfe2-e743-4910-8c32-e05759f9f9f4"
    cluster = "default"
    username = "nacos"
    password = "nacos"
}
eureka {
    serviceUrl = "http://localhost:8761/eureka"
    application = "default"
    weight = "1"
}
redis {
    serverAddr = "localhost:6379"
    db = 0
    password = ""
    cluster = "default"
    timeout = 0
}
zk {
    cluster = "default"
    serverAddr = "127.0.0.1:2181"
    sessionTimeout = 6000
    connectTimeout = 2000
    username = ""
    password = ""
}
consul {
    cluster = "default"
    serverAddr = "127.0.0.1:8500"
}
etcd3 {
    cluster = "default"
    serverAddr = "http://localhost:2379"
}
sofa {
    serverAddr = "127.0.0.1:9603"
    application = "default"
    region = "DEFAULT_ZONE"
    datacenter = "DefaultDataCenter"
    cluster = "default"
    group = "SEATA_GROUP"
    addressWaitTime = "3000"
}
file {
    name = "file.conf"
}
}
##配置seata-server的配置中心，支持file、nacos 、apollo、zk、consul、etcd3

config {
    # file、nacos 、apollo、zk、consul、etcd3
    # type = "file"

```

```

type = "nacos"
#nacos {
#   serverAddr = "ch"
#   namespace = "e385bfe2-e743-4910-8c32-e05759f9f9f4"
#   group = "SEATA_GROUP"
#   username = ""
#   password = ""
# }
nacos {
  application = "seata-server"
  # serverAddr = "192.168.56.121:8848"
  serverAddr = "cdh1:8848"
  namespace = "e385bfe2-e743-4910-8c32-e05759f9f9f4"
  group = "SEATA_GROUP"
  dataId = "seata-tc.properties"
  cluster = "default"
  username = "nacos"
  password = "nacos"
}

consul {
  serverAddr = "127.0.0.1:8500"
}
apollo {
  appId = "seata-server"
  apolloMeta = "http://192.168.1.204:8801"
  namespace = "application"
}
zk {
  serverAddr = "127.0.0.1:2181"
  sessionTimeout = 6000
  connectTimeout = 2000
  username = ""
  password = ""
}
etcd3 {
  serverAddr = "http://localhost:2379"
}
file {
  name = "file.conf"
}
}

```

重启 TC, 一切OK

NameSpace、Group、DataId

nacos中提供了NameSpace、Group、DataId，他的作用是能让我们对配置文件进行分类管理，三个能够确定唯一的配置文件。我说一点，可能不同的公司，会对这3个的定义是不同的。

比如 定义一：

1. NameSpace：区分不同的环境
2. Group：区分不同的项目或系统
3. DataId：项目中的配置文件

定义二：

1. NameSpace：区分不同的项目
2. Group：区分不同的模块
3. DataId：区分不同的环境

定义三：

1. NameSpace：区分不同的租户
2. Group：区分不同的应用
3. DataId：区分不同的环境

还有其他的定义，看公司。

Namespace：区分不同的环境

一般开发都会有多套环境，如果多套环境公用一个nacos，那么配置中心和注册中心都会发生冲突，所以需要用namespace隔离开

```
address: 0.0.0.0
port: 8083
servlet:
  # 这里设置了context-path
  context-path: /settlement/v1

spring:
  application:
    name: settlement
  cloud:
    nacos:
      config:
        server-addr: nacos-headless.default.svc.cluster.local:8848
        # 控制台创建命名空间得到的uuid
        namespace: c9ad103a-5420-4628-aba7-c147e3048d9d
      discovery:
        server-addr: nacos-headless.default.svc.cluster.local:8848
        # 控制台创建命名空间得到的uuid
        namespace: c9ad103a-5420-4628-aba7-c147e3048d9d
      metadata:
        management:
          # 这里要适配下健康检查的endpoint
          context-path: '${server.servlet.context-path}/actuator'

management:
  endpoints:
    web:
      exposure:
        # actuator暴露所有endpoint
        include: "*"

```

进行Group的切换

假设要进行Group的切换，只需要改下面的配置的值：

```
server:
  port: 3377
spring:
  application:
    name: nacos-config-client
  cloud:
    nacos:
      discovery:
        server-addr: localhost:8848 #注册中心的地址
      config:
        server-addr: localhost:8848 #配置中心的地址
        file-extension: yaml # 要读取nacos上的配置文件的后缀，这里只能是yaml，不能是ym1
        group: TEST_GROUP
```

进行NameSpace的切换

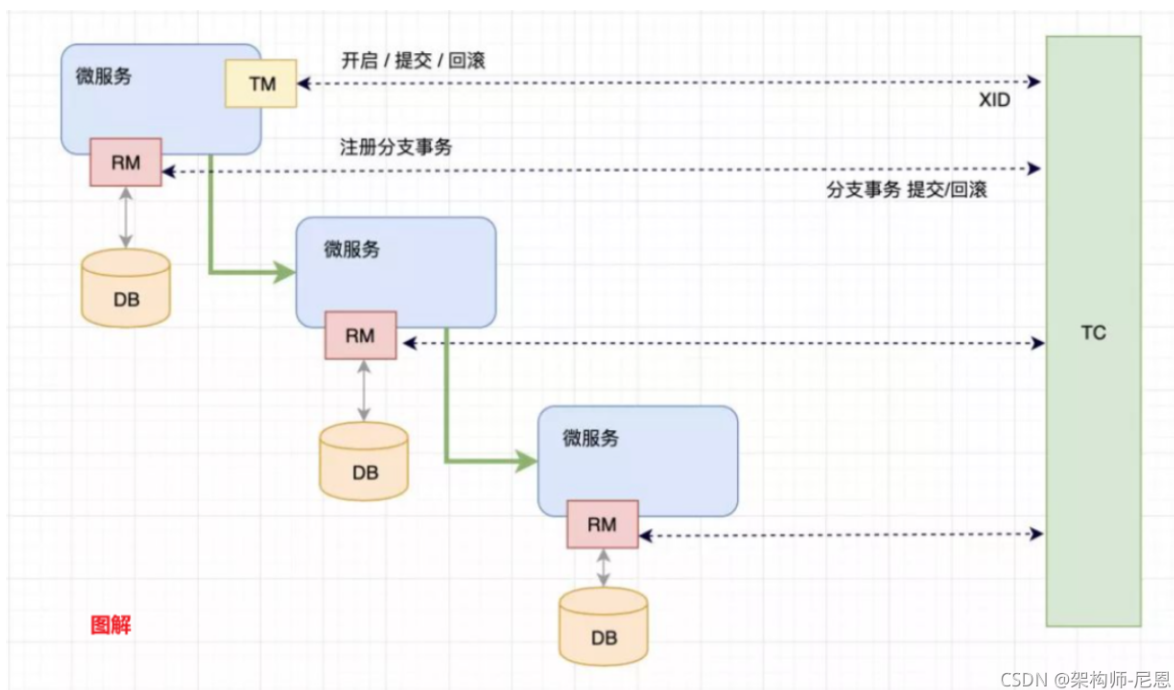
假设要进行NameSpace的切换，只需要改下面的配置的值：

```
server:
  port: 3377
spring:
  application:
    name: nacos-config-client
  cloud:
    nacos:
      discovery:
        server-addr: localhost:8848 #注册中心的地址
        namespace: 命名空间的ID
      config:
        server-addr: localhost:8848 #配置中心的地址
        file-extension: yaml # 要读取nacos上的配置文件的后缀，这里只能是yaml，不能是ym1
        namespace: 命名空间的ID
```

假设同一个NameSpace、Group，要进行DataId的 profile 后缀切换，只需要改下面的配置的值：

```
spring:
  profiles:
    active: dev
123
```

AT模式的RM&TM&TC直接的交互流程



- 事务协调器 TC：维护全局和分支事务的状态，指示全局提交或者回滚。
- 事务管理者 TM：开启、提交或者回滚一个全局事务。
- 资源管理者 RM(数据库)：管理执行分支事务的那些资源，向TC注册分支事务、上报分支事务状态、控制分支事务的提交或者回滚。

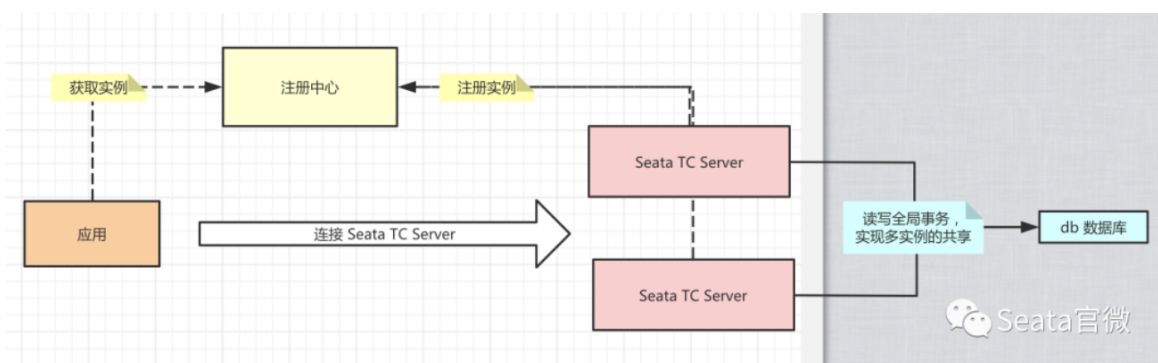
大致的流程

- ① TM 请求 TC，开始一个新的全局事务，TC 会为此全局事务生成一个唯一XID
- ② XID 通过微服务的调用链传递到其他微服务。
- ③ RM 向TC 注册分支事务，将其纳入XID 对应全局事务的管辖；
- ④ TM 请求 TC 对这个 XID 进行提交或回滚
- ⑤ TC 指挥这个 XID 下面的所有分支事务进行提交、回滚。

上面的流程有点复杂，如果搞不清楚，可以看下 配套视频

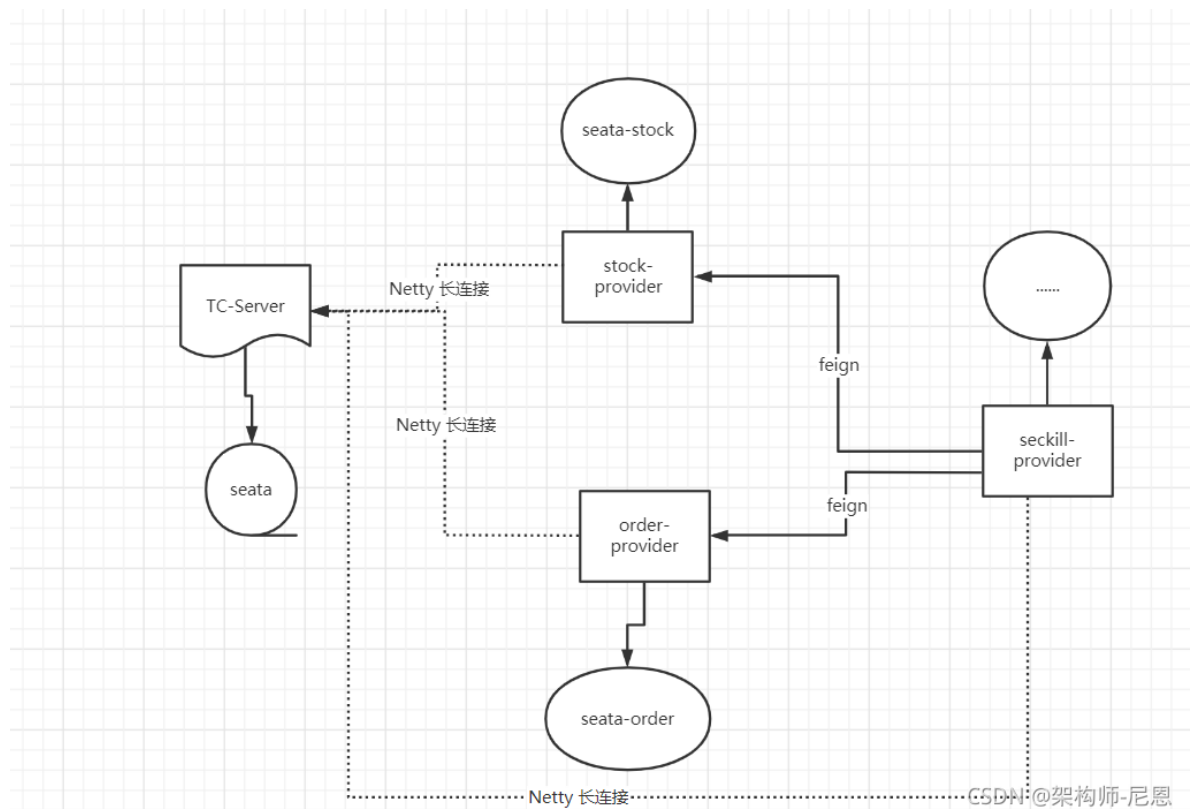
TM&RM 应用开发

如果你经过前面的步骤搭建Seata环境完成了，那么你可以尝试一下分布式事务的应用开发，也就是TM&RM这块。



业务场景

那么下面准备以Seata官方文档上的一个经典例子为题，模拟用户下单，创建订单同时扣减库存数量这一过程中产生的分布式事务问题，然后使用Seata解决，正好使用以下Seata的特性。



中间件版本选型

在当下微服务架构比较火热时，新一代微服务解决方案Spring Cloud Alibaba提供的开源分布式事务解决框架Seata无疑成为了我们在解决分布式事务时的首要之选，

版本选择：Spring Cloud Alibaba与Spring Boot、Spring Cloud版本对应关系

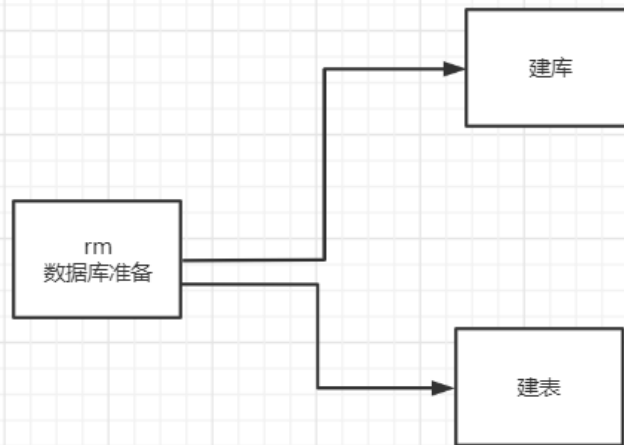
组件	版本号
jdk.version	1.8
nacos	1.2.1
spring cloud	Greenwich.SR2
spring boot	2.1.6.RELEASE
spring cloud alibaba	2.1.3.RELEASE

坑点：

如果项目中使用了druid数据库连接池，引入的是SpringBoot的Starter依赖druid-spring-boot-starter，那么需要把druid-spring-boot-starter依赖换成druid1.1.23，

因为seata源码中引入的druid依赖跟druid-spring-boot-starter的自动装配类冲突了，冲突的情况下项目启动出现异常

数据库准备



CSDN @架构师-尼恩

连接MYSQL:

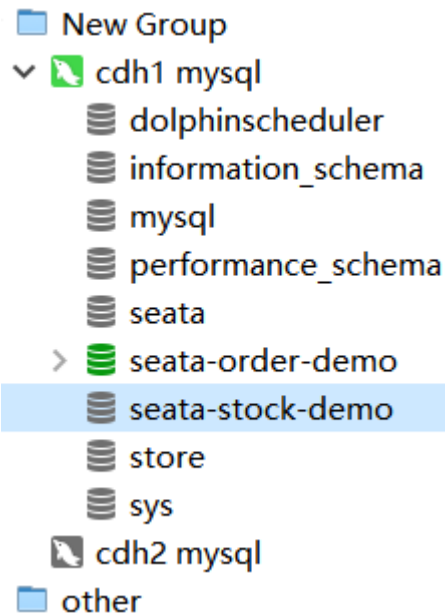
格式: mysql -h主机地址 -u用户名 -p用户密码

```
mysql -h192.168.9.1 -uroot -p123456
```

```
mysql -uroot -p123456
```

创建RM端的专属独立库seata。

```
/usr/bin/mysql -uroot -p"123456" --connect-expired-password <<EOF
CREATE DATABASE `seata-stock-demo` default character set utf8mb4 collate
utf8mb4_unicode_ci;
CREATE DATABASE `seata-order-demo` default character set utf8mb4 collate
utf8mb4_unicode_ci;
show databases;
grant all privileges on `seata-order-demo`.* to root@'%' identified by '123456'
WITH GRANT OPTION;
grant all privileges on `seata-stock-demo`.* to root@'%' identified by '123456'
WITH GRANT OPTION;
flush privileges;
EOF
```



CSDN @架构师-尼恩

创建RM端的专属独立的表。

在自己微服务数据库里面创建undo_log表，sql在源码里面有

下载TM端的配置文件，原始config.txt文件可以从github：

<https://github.com/seata-io/seata/blob/v1.3.0/script/client/at/db/mysql.sql>

或者 gitee：

<https://gitee.com/seata-io/seata/blob/v1.3.0/script/client/at/db/mysql.sql>

mysql> source /work/seata/conf/un_do.sql;

```
mysql -uroot -p123456

mysql> use seata-stock-demo;
Database changed
mysql>source    /work/seata/conf/un_do.sql;

mysql> use seata-order-demo;
Database changed
mysql>source    /work/seata/conf/un_do.sql;
```

这里我们只是做演示，理论上上面三个业务服务应该分属不同的数据库，这里我们只是在同一台数据库下面创建三个 Schema，分别为 db_account、db_order 和 db_storage，具体如图：

```
mysql> show tables;
+-----+
| Tables_in_seata |
+-----+
| branch_table    |
| global_table    |
| lock_table      |
+-----+
3 rows in set (0.00 sec)
```

TM端的注册中心配置

下载TM端的配置文件，原始config.txt文件可以从github：

<https://github.com/seata/seata/tree/develop/script/client>

或者 gitee：

<https://gitee.com/seata-io/seata/tree/develop/script/client>

下载的并修改 register.config：

```
registry {
  # file 、nacos 、eureka、redis、zk、consul、etcd3、sofa、custom
  # type = "file"
  type = "nacos"

  nacos {
    application = "seata-server"
    serverAddr = "cdh1"
    namespace = "e385bfe2-e743-4910-8c32-e05759f9f9f4"
    cluster = "SEATA_GROUP"
    username = "nacos"
    password = "nacos"
  }
  # nacos {
  #   application = "seata-server"
  #   serverAddr = "127.0.0.1:8848"
  #   group = "SEATA_GROUP"
  #   namespace = ""
  #   username = ""
  #   password = ""
  # }
  eureka {
    serviceUrl = "http://localhost:8761/eureka"
    weight = "1"
  }
  redis {
    serverAddr = "localhost:6379"
    db = "0"
    password = ""
    timeout = "0"
  }
  zk {
```

```

serverAddr = "127.0.0.1:2181"
sessionTimeout = 6000
connectTimeout = 2000
username = ""
password = ""
}
consul {
serverAddr = "127.0.0.1:8500"
aclToken = ""
}
etcd3 {
serverAddr = "http://localhost:2379"
}
sofa {
serverAddr = "127.0.0.1:9603"
region = "DEFAULT_ZONE"
datacenter = "DefaultDataCenter"
group = "SEATA_GROUP"
addressWaitTime = "3000"
}
file {
name = "file.conf"
}
custom {
name = ""
}
}

config {
# file、nacos 、apollo、zk、consul、etcd3、springCloudConfig、custom
type = "file"

nacos {
serverAddr = "127.0.0.1:8848"
namespace = ""
group = "SEATA_GROUP"
username = ""
password = ""
dataId = "seata.properties"
}
consul {
serverAddr = "127.0.0.1:8500"
aclToken = ""
}
apollo {
appId = "seata-server"
apolloMeta = "http://192.168.1.204:8801"
namespace = "application"
apolloAccesskeySecret = ""
}
zk {
serverAddr = "127.0.0.1:2181"
sessionTimeout = 6000
connectTimeout = 2000
username = ""
password = ""
nodePath = "/seata/seata.properties"
}
}

```

```

etcd3 {
    serverAddr = "http://localhost:2379"
}
file {
    name = "file.conf"
}
custom {
    name = ""
}
}

```

file.conf本地配置

如果配置中心的类型都不是file时，就不需要file.conf文件了

如果配置中心的类型是file时，就需要file.conf

```

transport {
    # tcp udt unix-domain-socket
    type = "TCP"
    #NIO NATIVE
    server = "NIO"
    #enable heartbeat
    heartbeat = true
    # the client batch send request enable
    enableClientBatchSendRequest = true
    #thread factory for netty
    threadFactory {
        bossThreadPrefix = "NettyBoss"
        workerThreadPrefix = "NettyServerNIOWorker"
        serverExecutorThread-prefix = "NettyServerBizHandler"
        shareBossworker = false
        clientSelectorThreadPrefix = "NettyClientSelector"
        clientSelectorThreadSize = 1
        clientWorkerThreadPrefix = "NettyClientWorkerThread"
        # netty boss thread size,will not be used for UDT
        bossThreadSize = 1
        #auto default pin or 8
        workerThreadSize = "default"
    }
    shutdown {
        # when destroy server, wait seconds
        wait = 3
    }
    serialization = "seata"
    compressor = "none"
}
service {
    #transaction service group mapping
    # my_test_tx_group为分组名称，需要与应用中的配置分组名称一致
    # "default"为seata-server的集群名称，即seata-server注册到注册中心的名称
    vgroupMapping.my_test_tx_group = "default"
    #only support when registry.type=file, please don't set multiple addresses
    # 当registry.type=file时，客户端通过ip:port连接到seata-server
    default.grouplist = "127.0.0.1:8091"
}

```



```

#degrade, current not support
enableDegrade = false
#disable seata
disableGlobalTransaction = false
}

client {
  rm {
    asyncCommitBufferLimit = 10000
    lock {
      retryInterval = 10
      retryTimes = 30
      retryPolicyBranchRollbackOnConflict = true
    }
    reportRetryCount = 5
    tableMetaCheckEnable = false
    reportSuccessEnable = false
  }
  tm {
    commitRetryCount = 5
    rollbackRetryCount = 5
  }
  undo {
    dataValidation = true
    logSerialization = "jackson"
    # undo表名称,SEATA AT模式需要UNDO_LOG表
    logTable = "undo_log"
  }
  log {
    exceptionRate = 100
  }
}

```

file.conf 配置文件分部分说明

这里说一下配置的内容，已默认的文件为例：

在file.conf中有3部分配置内容：

1.transport

transport部分的配置是关于Netty的配置，主要体现在io.seata.core.rpc.netty包下的NettyBaseConfig、NettyServerConfig、NettyClientConfig，client与server的通信使用的是Netty

2.service

仅针对client有效

```

# service configuration, only used in client side
service {
  #transaction service group mapping
  #my_test_tx_group-->自定义分布式事务组名称
  vgroupMapping.my_test_tx_group = "default"
  #only support when registry.type=file, please don't set multiple addresses
  #db模式改配置无效
  default.grouplist = "127.0.0.1:8091"
  #degrade, current not support

```

```

enableDegrade = false
#disable seata
#是否启用seata的分布式事务
disableGlobalTransaction = false
}

```

io.seata.spring.annotation.GlobalTransactionScanner

3.client

仅针对client有效

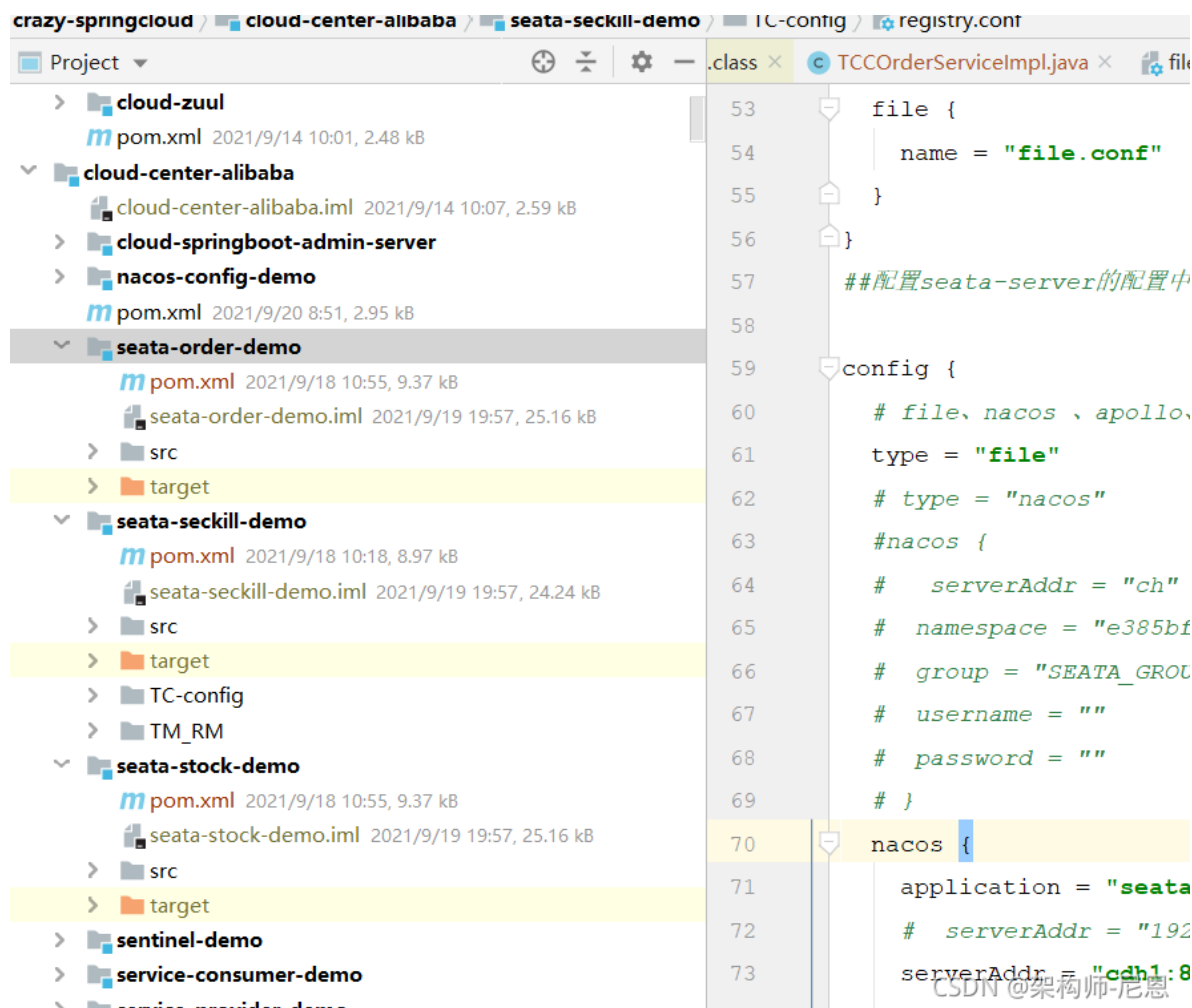
```

#client transaction configuration, only used in client side
client {
  rm {
    #RM接收TC的commit通知后缓冲上限
    asyncCommitBufferLimit = 10000
    lock {
      #校验或占用全局锁重试间隔 默认10, 单位毫秒
      retryInterval = 10
      #校验或占用全局锁重试次数 默认30
      retryTimes = 30
      #分支事务与其它全局回滚事务冲突时锁策略 默认true, 优先释放本地锁让回滚成功
      retryPolicyBranchRollbackOnConflict = true
    }
    #一阶段结果上报TC重试次数 默认5次
    reportRetryCount = 5
    #自动刷新缓存中的表结构 默认false
    tableMetaCheckEnable = false
    #是否上报一阶段成功 true、false, 从1.1.0版本开始,默认false.true用于保持分支事务生命周期记录完整, false可提高不少性能
    reportSuccessEnable = false
    sqlParserType = druid
  }
  tm {
    #一阶段全局提交结果上报TC重试次数 默认1次, 建议大于1
    commitRetryCount = 5
    #一阶段全局回滚结果上报TC重试次数 默认1次, 建议大于1
    rollbackRetryCount = 5
  }
  undo {
    #二阶段回滚镜像校验 默认true开启, false关闭
    dataValidation = true
    #undo序列化方式 默认jackson
    logSerialization = "jackson"
    #自定义undo表名 默认undo_log
    logTable = "undo_log"
  }
  log {
    #日志异常输出概率 默认100, 目前用于undo回滚失败时异常堆栈输出, 百分之一的概率输出, 回滚失败基本是脏数据, 无需输出堆栈占用硬盘空间
    exceptionRate = 100
  }
}

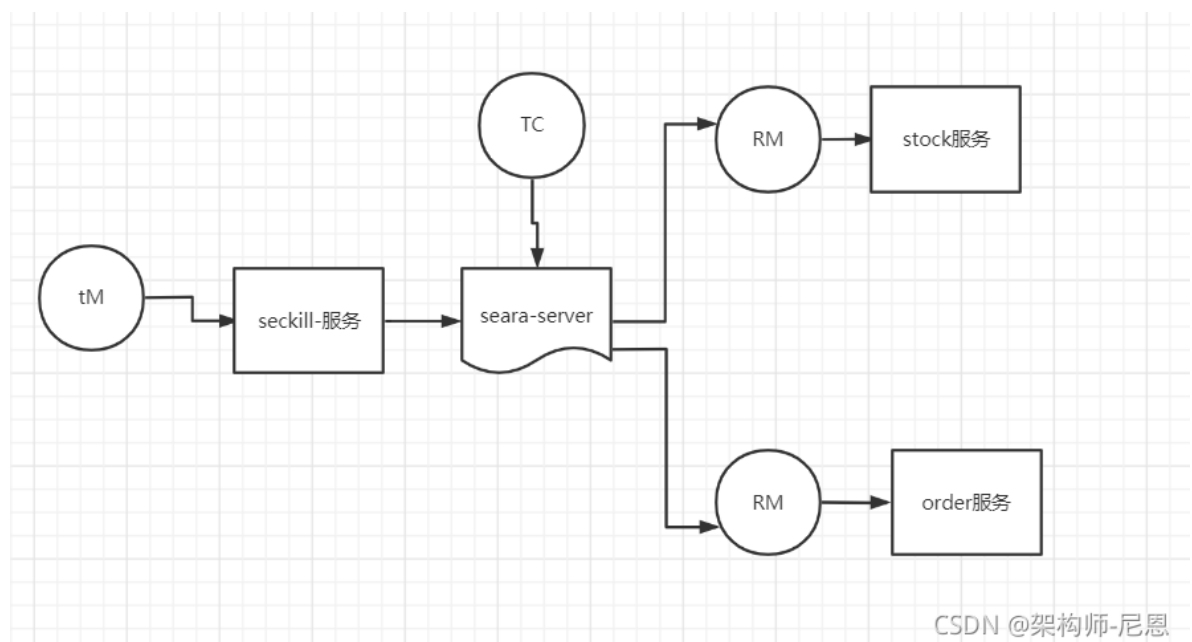
```

项目准备

模块架构



模块的角色架构



maven依赖

```
<!-- 分布式事务seata包 -->  
<!--seata begin-->
```

```

<dependency>
  <groupId>com.alibaba.cloud</groupId>
  <artifactId>spring-cloud-starter-alibaba-seata</artifactId>
  <version>2.1.3.RELEASE</version>
  <exclusions>
    <exclusion>
      <groupId>io.seata</groupId>
      <artifactId>seata-spring-boot-starter</artifactId>
    </exclusion>
  </exclusions>
</dependency>
<dependency>
  <groupId>io.seata</groupId>
  <artifactId>seata-spring-boot-starter</artifactId>
  <version>1.3.0</version>
</dependency>
<!--seata end-->

```

事务分组是什么？

于是，上官网查看相关的参数配置，搜索serviceGroup

<https://seata.io/zh-cn/docs/user/transaction-group.html>

事务分组是什么？

事务分组是 Seata 的资源逻辑，类似于服务实例。在file.conf中的 my_test_tx_group就是一个事务分组。

通过事务分组如何找到后端集群？

首先程序中配置了事务分组（GlobalTransactionScanner 构造方法的txServiceGroup参数），程序会通过用户配置的配置中心去寻找 service.vgroupMapping.事务分组配置项，取得配置项的值就是TC集群的名称。拿到集群名称程序通过一定的前后缀+集群名称去构造服务名，各配置中心的服务名实现不同。拿到服务名去相应的注册中心去拉取相应服务名的服务列表，获得后端真实的TC服务列表。

为什么这么设计，不直接取服务名？

这里多了一层获取事务分组到映射集群的配置。这样设计后，事务分组可以作为资源的逻辑隔离单位，当发生故障时可以快速failover。

根据第 2 点的说明，就可以知道问题所在了：

TM&RM配置详解

1. registry.conf：配置注册中心和配置中心，默认是file。
2. file.conf：seata工作规则信息
3. DataSourceConfig：配置代理数据源实现分支事务，如果没有注入，事务无法成功回滚

registry.conf配置：配置注册中心和配置中心

该文件包含两部分配置：

1. 注册中心
2. 配置中心

注册中心

```

registry { # 注册中心配置
    # 可选项: file 、nacos 、eureka、redis、zk
    type = "nacos" # 指定nacos注册中心，默认是file。由于项目整体使用nacos，所以后续选择nacos

    nacos {
        serverAddr = "127.0.0.1:8848"
        namespace = "public"
        cluster = "default"
    }
    eureka {
        serviceUrl = "http://localhost:1001/eureka"
        application = "default"
        weight = "1"
    }
    redis {
        serverAddr = "localhost:6381"
        db = "0"
    }
    zk {
        cluster = "default"
        serverAddr = "127.0.0.1:2181"
        session.timeout = 6000
        connect.timeout = 2000
    }
    file {
        name = "file.conf"
    }
}

```

配置中心

```

config { # 配置中心
    # 可选项: file、nacos 、apollo、zk
    type = "file" # 指向file配置中心，也可以指向nacos等其他注册中心

    nacos {
        serverAddr = "localhost"
        namespace = "public"
        cluster = "default"
    }
    apollo {
        app.id = "fescar-server"
        apollo.meta = "http://192.168.1.204:8801"
    }
    zk {
        serverAddr = "127.0.0.1:2181"
        session.timeout = 6000
        connect.timeout = 2000
    }
    file {
        name = "file.conf" # 通过file.conf配置seata参数，指向第二个配置文件
    }
}

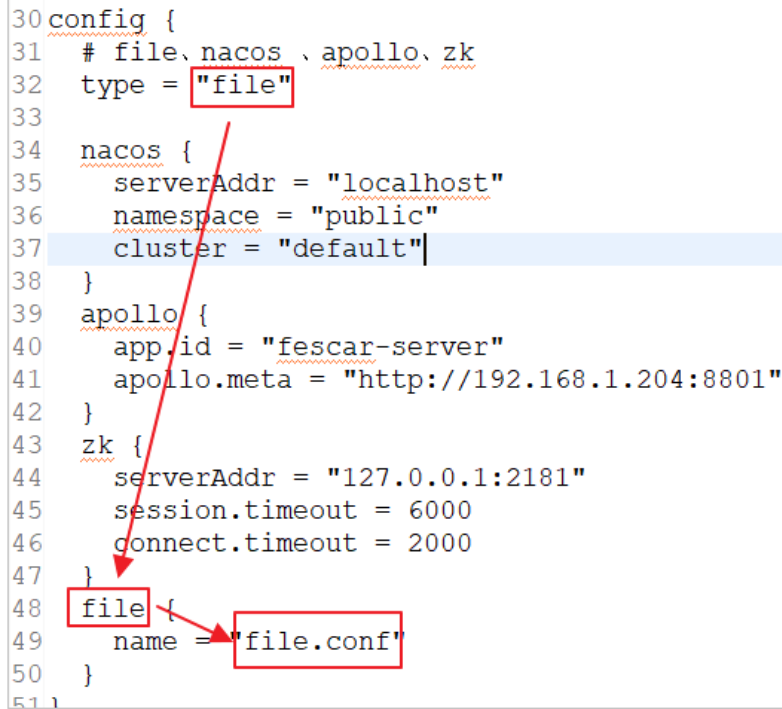
```

file.conf

该文件的命名取决于registry.conf配置中心的配置

由于registry.conf中配置的是

```
30 config {
31   # file、nacos、apollo、zk
32   type = "file"
33
34   nacos {
35     serverAddr = "localhost"
36     namespace = "public"
37     cluster = "default"
38   }
39   apollo {
40     app.id = "fescar-server"
41     apollo.meta = "http://192.168.1.204:8801"
42   }
43   zk {
44     serverAddr = "127.0.0.1:2181"
45     session.timeout = 6000
46     connect.timeout = 2000
47   }
48   file {
49     name = "file.conf"
50   }
51 }
```



也就是说: file.conf

文件名取决于registry的配置中心(config{...})配置, 如果registry配置的配置中心不是file, 可以没有改文件。

如果配置中心是nacos, 这是file.conf文件就不需要了, 把file.conf文件内容交给nacos就可

事务日志存储配置:

```
store {
  ## store mode: file、db
  mode = "file" # 存储方式file、db

  ## file store
  file {
    dir = "sessionStore"

    # branch session size , if exceeded first try compress lockkey, still
    exceeded throws exceptions
    max-branch-session-size = 16384
    # globe session size , if exceeded throws exceptions
    max-global-session-size = 512
    # file buffer size , if exceeded allocate new buffer
    file-write-buffer-cache-size = 16384
    # when recover batch read size
    session.reload.read_size = 100
    # async, sync
    flush-disk-mode = async
  }

  ## database store
}
```

```

db {
    driver_class = ""
    url = ""
    user = ""
    password = ""
}
}

```

TC信息配置

当前微服务在seata服务器中注册的信息配置：

```

service {
    #vgroup->rgroup
    #必须和服务名一致: ${spring.applicaiton.name}
    #vgroup_mapping.${spring.application.name}-fescar-service-group = "default"
    vgroup_mapping.${spring.application.name}-fescar-service-group = "default"
    #only support single node
    default.grouplist = "cdh1:18091" #seata-server服务器地址，默认是8091
    #degrade current not support
    enableDegrade = false
    #disable
    disable = false
}

```

客户端相关工作的机制

```

client {
    async.commit.buffer.limit = 10000
    lock {
        retry.internal = 10
        retry.times = 30
    }
}

```

DataSourceConfig

每一个微服务原来自己的数据源都必须使用DataSourceProxy代理，这样seata才能掌控所有事务。

```

@Configuration
public class DataSourceConfig {

    @Bean
    @ConfigurationProperties(prefix = "spring.datasource")
    public DruidDataSource druidDataSource() {
        return new DruidDataSource();
    }

    /**
     * 需要将 DataSourceProxy 设置为主数据源，否则事务无法回滚
     *
     * @param druidDataSource The DruidDataSource
     * @return The default datasource
     */
}

```

```

    */
    @Primary
    @Bean("dataSource")
    public DataSource dataSource(DruidDataSource druidDataSource) {
        return new DataSourceProxy(druidDataSource);
    }
}

```

事务注解

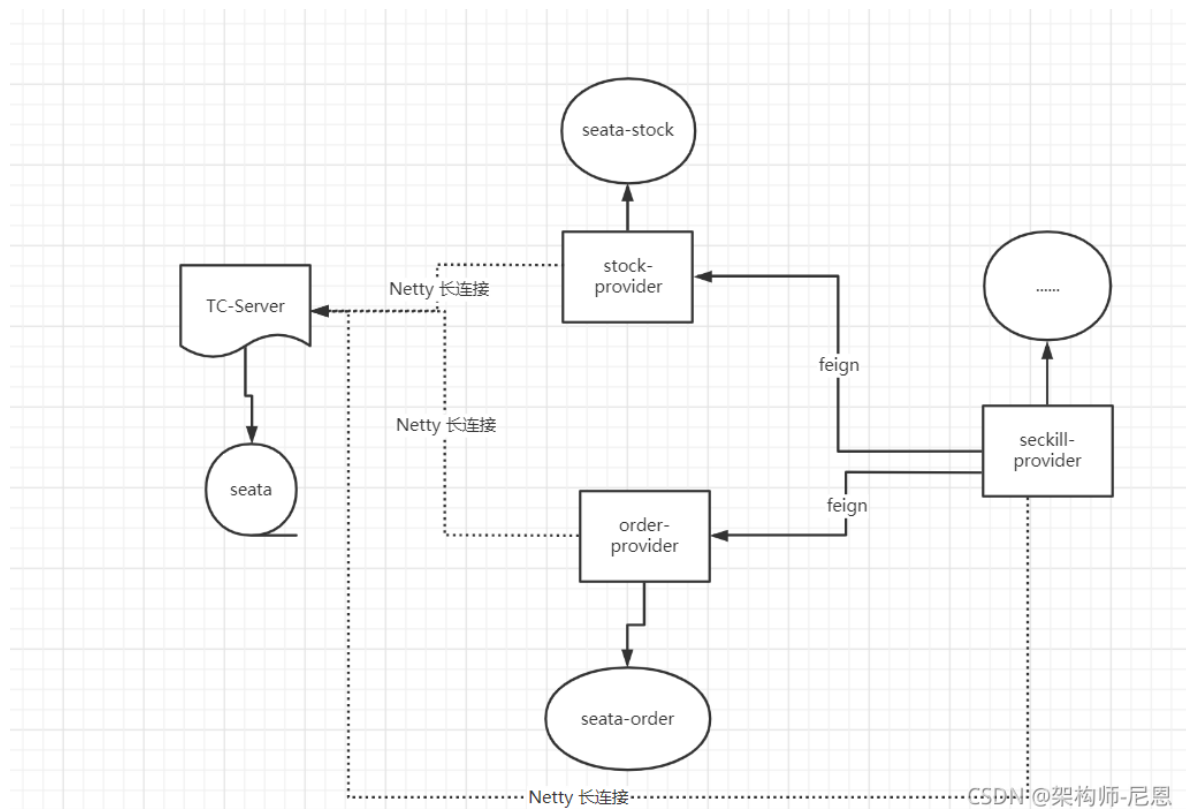
RM主业务方法添加全局事务：@GlobalTransactional

TM分支业务方法添加本地事务注解：@Transactional

说明：以上的配置，如果不能理解，请参考配套视频

代码实现

10WQPS秒杀实操的AT分布式事务架构



如果你经过前面的步骤搭建Seata环境完成了，那么你可以尝试一下启动项目，控制台无异常则搭建成功。

那么下面准备以Seata官方文档上的一个经典例子为题，模拟用户下单，创建订单同时扣减库存数量这一过程中产生的分布式事务问题，然后使用Seata解决，正好使用以下Seata的特性。

服务秒杀

seckillController

```

@RestController
@RequestMapping("/api/seckill/seglock/")
@Api(tags = "秒杀练习分布式事务 版本")
public class SeckillBySegmentLockController {

```



```

@Resource
seataSeckillServiceImpl seataSeckillServiceImpl;

/**
 * 执行秒杀的操作
 * 减库存，下订单
 * <p>
 * {
 * "exposedKey": "4b70903f6e1aa87788d3ea962f8b2f0e",
 * "newStockNum": 10000,
 * "seckillSkuId": 1247695238068177920,
 * "seckillToken": "0f8459cbae1748c7b14e4cea3d991000",
 * "userId": 37
 * }
 *
 * @return
 */
@ApiOperation(value = "秒杀")
@PostMapping("/doSeckill/v1")
RestOut<SeckillDTO> doSeckill(@RequestBody SeckillDTO dto) {

    seataSeckillServiceImpl.doSeckill(dto);

    return RestOut.success(dto).setRespMsg("秒杀成功");
}

}

```

seckillServiceImpl

```

public class SeataSeckillServiceImpl {

    @Autowired
    private SeataDemoOrderFeignClient stockFeignClient;
    @Autowired
    private SeataDemoStockFeignClient orderFeignClient;

    /**
     * 减库存，下订单
     */
    @GlobalTransactional //开启全局事务（重点） 使用 seata 的全局事务
    public void doSeckill(@RequestBody SeckillDTO dto) {

        orderFeignClient.minusStock(dto);
        stockFeignClient.addOrder(dto);
    }

}

```

库存服务的Feign类

```

@FeignClient(name = "seata-stock-demo", path = "/seata-stock-
demo/api/seckill/sku/")
public interface SeataDemoStockFeignClient {
    /**
     * minusStock 秒杀库存
     *
     * @param dto 商品与库存
     * @return 商品 skuDTO
     */
    @RequestMapping(value = "/minusStock/v1", method = RequestMethod.POST)
    RestOut<SeckillSkuDTO> minusStock(@RequestBody SeckillDTO dto);
}

```

订单服务的Feign类

```

@FeignClient(name = "seata-order-demo", path = "/seata-order-
demo/api/seckill/order/")
public interface SeataDemoOrderFeignClient {

    @RequestMapping(value = "/addOrder/v1", method = RequestMethod.POST)
    RestOut<SeckillOrderDTO> addOrder(@RequestBody SeckillDTO dto);
}

```

订单服务

orderController

```

@RestController
@RequestMapping("/api/seckill/order/")
@Api(tags = "秒杀练习 订单管理")
public class SeataATOrderController {
    @Resource
    seckillOrdersServiceImpl seckillOrdersService;

    /**
     * 查询用户订单信息
     *
     * @param userId 用户id
     * @return 商品 dto
     */
    @PostMapping("/user/{id}/list/v1")
    @ApiOperation(value = "查询用户订单信息")
    RestOut<PageOut<SeckillOrderDTO>> userOrders(
        @PathVariable(value = "id") Long userId, @RequestBody PageReq
pageReq) {
        PageOut<SeckillOrderDTO> dto =
seckillOrdersService.findOrderByUserID(userId, pageReq);
        if (null != dto) {
            return RestOut.success(dto).setRespMsg("查询成功");
        }
    }
}

```

```

    }
    return RestOut.error("查询失败");
}

/**
 * 查询用户的订单信息
 *
 * @param userId 用户id
 * @param skuId 商品id
 * @return 商品 dto
 */
@GetMapping("/{userId}/{skuId}/v1")
@ApiOperation(value = "查询用户订单信息")
RestOut<SeckillOrderDTO> userOrders(
    @PathVariable(value = "userId") Long userId,
    @PathVariable(value = "skuId") Long skuId,
    @RequestBody PageReq pageReq) {
    List<SeckillOrderPO> pos =
seckillOrderService.findOrderByUserIDAndSkuId(userId, skuId);
    if (null != pos && pos.size() > 0) {
        SeckillOrderDTO orderDTO = new SeckillOrderDTO();
        BeanUtils.copyProperties(pos.get(0), orderDTO);
        return RestOut.success(orderDTO).setRespMsg("查询成功");
    }
    return RestOut.error("查询失败");
}

/**
 * 清除用户订单信息
 *
 * @param dto 含有 用户id的dto
 * @return 操作结果
 */
@PostMapping("/user/clear/v1")
@ApiOperation(value = "清除用户订单信息")
RestOut<String> userOrdersClear(@RequestBody SeckillDTO dto) {
    Long userId = dto.getUserId();
    String result = seckillOrderService.clearOrderByUserID(userId);

    return RestOut.success(result).setRespMsg("处理完成");
}

/**
 * 执行秒杀的操作
 * <p>
 * <p>
 * {
 * "exposedKey": "4b70903f6e1aa87788d3ea962f8b2f0e",
 * "newStockNum": 10000,
 * "seckillSkuId": 1157197244718385152,
 * "seckillToken": "0f8459cbae1748c7b14e4cea3d991000",
 * "userId": 37
 * }
 *
 * @return
 */

```

```

@ApiOperation(value = "下订单")
@PostMapping("/addOrder/v1")
RestOut<SeckillOrderDTO> addOrder(@RequestBody SeckillDTO dto) {
    SeckillOrderDTO orderDTO = seckillOrdersService.addOrder(dto);
    return RestOut.success(orderDTO).setRespMsg("下订单成功");
}
}

```

OrderServiceImpl

```

/**
 * 执行秒杀下单
 *
 * @param inDto
 * @return
 */
@Transactional //开启本地事务
// @GlobalTransactional//不，开启全局事务（重点） 使用 seata 的全局事务
public SeckillOrderDTO addOrder(SeckillDTO inDto) {

    long skuId = inDto.getSeckillSkuId();
    Long userId = inDto.getUserId();

    /**
     * 创建订单对象
     */
    SeckillOrderPO order =
        seckillOrderPO.builder()
            .skuId(skuId).userId(userId).build();

    Date nowTime = new Date();
    order.setCreateTime(nowTime);
    order.setStatus(SeckillConstants.ORDER_VALID);

    SeckillOrderDTO dto = null;

    /**
     * 创建重复性检查的订单对象
     */
    SeckillOrderPO checkOrder =
        seckillOrderPO.builder().skuId(
            order.getSkuId()).userId(order.getUserId()).build();

    //记录秒杀订单信息
    long insertCount = seckillOrderDao.count(Example.of(checkOrder));

    //唯一性判断: skuId,id 保证一个用户只能秒杀一件商品
    if (insertCount >= 1) {
        //重复秒杀
        log.error("重复秒杀");
        throw BusinessException.builder().errMsg("重复秒杀").build();
    }
}

```

```

/**
 * 插入秒杀订单
 */
seckillOrderDao.save(order);

dto = new SeckillOrderDTO();
BeanUtils.copyProperties(order, dto);

return dto;
}

```

库存服务

stockController

```

@Slf4j
@RestController
@RequestMapping("/api/seckill/sku/")
@Api(tags = "商品库存")
public class SeataSeckillStockController {
    @Resource
    SeataStockServiceImpl seckillSkuStockService;

    /**
     * minusStock 秒杀库存
     *
     * @param dto 商品与库存
     * @return 商品 skuDTO
     */
    @PostMapping("/minusStock/v1")
    @ApiOperation(value = "减少秒杀库存")
    RestOut<SeckillSkuDTO> minusStock(@RequestBody SeckillDTO dto,
    HttpServletRequest request) {

        // 绑定 xID, 自动创建分支事物
        // 异常后, 整个调用链路回滚
        String keyId = request.getHeader(RootContext.KEY_XID);

        if (null != keyId) {

            log.info("RootContext.KEY_XID is {}", keyId);
            // 绑定 xID, 自动创建分支事物
            RootContext.bind(keyId);
        }

        Long skuId = dto.getSeckillSkuId();
    }
}

```

```

        SeckillSkuDTO skuDTO = seckillSkuStockService.minusStock(dto);

        if (null != skuDTO) {
            return RestOut.success(skuDTO).setRespMsg("减少秒杀库存成功");
        }
        return RestOut.error("未找到指定秒杀商品");
    }
}

```

StockServiceImpl

```

/**
 * 执行秒杀下单
 *
 * @param inDto
 * @return
 */
@Transactional
public SeckillSkuDTO minusStock(SeckillDTO inDto) {

    long skuId = inDto.getSeckillSkuId();

    Optional<SeckillSkuPO> optional = seckillSkuDao.findById(skuId);

    if (!optional.isPresent()) {
        throw BusinessException.builder().errMsg("商品不存在").build();
    }

    SeckillSkuPO po = optional.get();
    if (po.getStockCount() <= 0) {
        throw BusinessException.builder().errMsg("库存不够").build();
    }
    seckillSkuDao.decreaseStockCountById(skuId);
    // po.setStockCount(po.getStockCount() - 1);
    SeckillSkuDTO dto = new SeckillSkuDTO();
    dto.setStockCount(po.getStockCount() - 1);
    BeanUtils.copyProperties(po, dto);
    return dto;

}

```

实验演示：启动后进行事务演示

启动之后，可以进行秒杀下单演示，注意，第二阶段的成功或者回滚

具体演示，参见此博客的配套视频

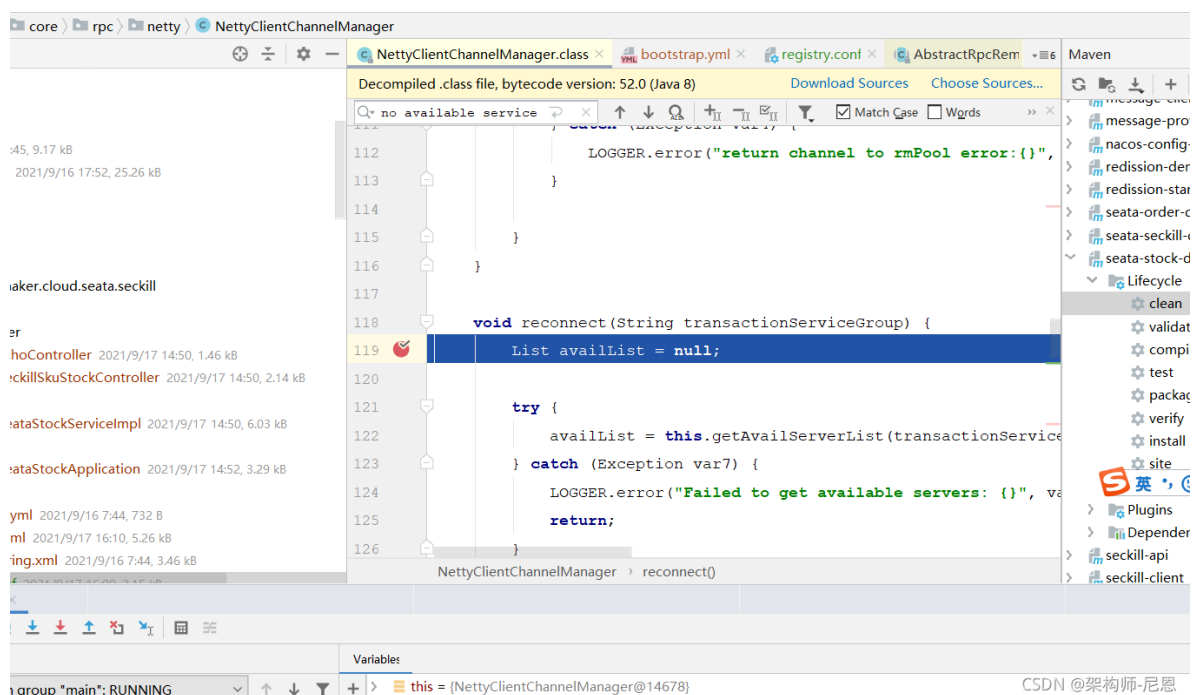
错误排除

NettyClientChannelManager : no available service 'null' found, please make sure registry config...

项目中在使用 Seata 作为分布式事务的时候，好像特别容易遇到这个错误~
总结原因，还是对Seata配置参数不熟悉导致的。
首先是看到错误描述，找到抛出错误的地方：

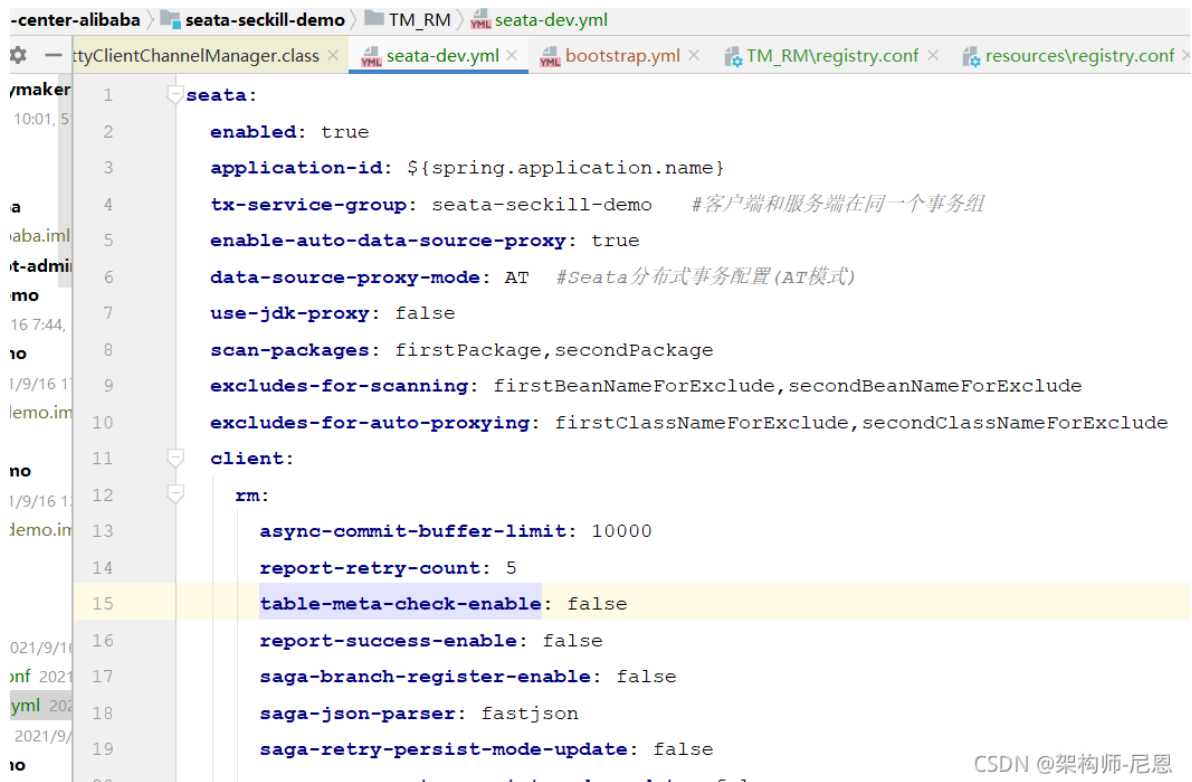


```
2021-09-17 16:29:19.055 INFO 21220 --- [.naming.updater] com.alibaba.nacos.client.naming LN:191 new ips (1) service: DEFAULT_GROUP@sea
2021-09-17 16:29:19.066 INFO 21220 --- [.naming.updater] com.alibaba.nacos.client.naming LN:228 current ips: (1) service: DEFAULT_GROUP@sea
2021-09-17 16:30:02.341 ERROR 21220 --- [timeoutChecker_2] i.s.c.r.netty.NettyClientChannelManager LN:170 no available service 'null' found, please make sure registry config is correct
2021-09-17 16:30:16.736 ERROR 21220 --- [timeoutChecker_1] i.s.c.r.netty.NettyClientChannelManager LN:170 no available service 'null' found, please make sure registry config is correct
2021-09-17 16:30:16.736 ERROR 21220 --- [timeoutChecker_2] i.s.c.r.netty.NettyClientChannelManager LN:170 no available service 'null' found, please make sure registry config is correct
2021-09-17 16:30:17.179 ERROR 21220 --- [timeoutChecker_1] i.s.c.r.netty.NettyClientChannelManager LN:170 no available service 'null' found, please make sure registry config is correct
2021-09-17 16:30:17.179 ERROR 21220 --- [timeoutChecker_2] i.s.c.r.netty.NettyClientChannelManager LN:170 no available service 'null' found, please make sure registry config is correct
2021-09-17 16:30:17.570 ERROR 21220 --- [timeoutChecker_1] i.s.c.r.netty.NettyClientChannelManager LN:170 no available service 'null' found, please make sure registry config is correct
```



```
112     LOG.error("return channel to rmPool error: {}", e);
113 }
114
115 }
116
117
118 void reconnect(String transactionServiceGroup) {
119     List availList = null;
120
121     try {
122         availList = this.getAvailServerList(transactionServiceGroup);
123     } catch (Exception var7) {
124         LOG.error("Failed to get available servers: {}", var7);
125         return;
126     }
127 }
```

看到 getAvailServerList(transactionServiceGroup); 的入参是：transactionServiceGroup。刚好在配置文件中也有一项配置与之对应：



CSDN @架构师-尼恩

Seata AT分布式事务执行流程演示

同步调用的3个阶段：

- 1、主事务调用分支事务之前；
- 2、分支事务结束并返回之前；
- 3、分支事务提交后，且主事务提交前

同步调用出问题的情况：

- 1、主事务成功，分支事务失败
- 2、主事务失败，分支事务成功

正常执行

观察每个阶段的undo_log和TC的记录

阶段一全局事务：

通过seckill 服务，观察 TC的记录 几个表的数据。

阶段一分支事务观察：

通过库存服务，观察undolog记录；

阶段一观察：

观察seata全局锁。

阶段二全局事务观察：

通过seckill 服务，观察 TC的记录 几个表的数据。

阶段二分支事务观察：

通过库存服务，观察undolog记录；

本实验演示：通过配套视频给出

异常执行

观察每个阶段的undo_log和TC的记录

阶段一全局事务：

通过seckill 服务，观察 TC的记录 几个表的数据。

阶段一分支事务观察：

通过库存服务，观察undolog记录；

阶段二全局事务观察：

通过seckill 服务，观察 TC的记录 几个表的数据。

阶段二分支事务观察：

通过库存服务，观察undolog记录；

本实验演示：通过配套视频给出

Seat TCC 模式实战

Seata TCC基本原理

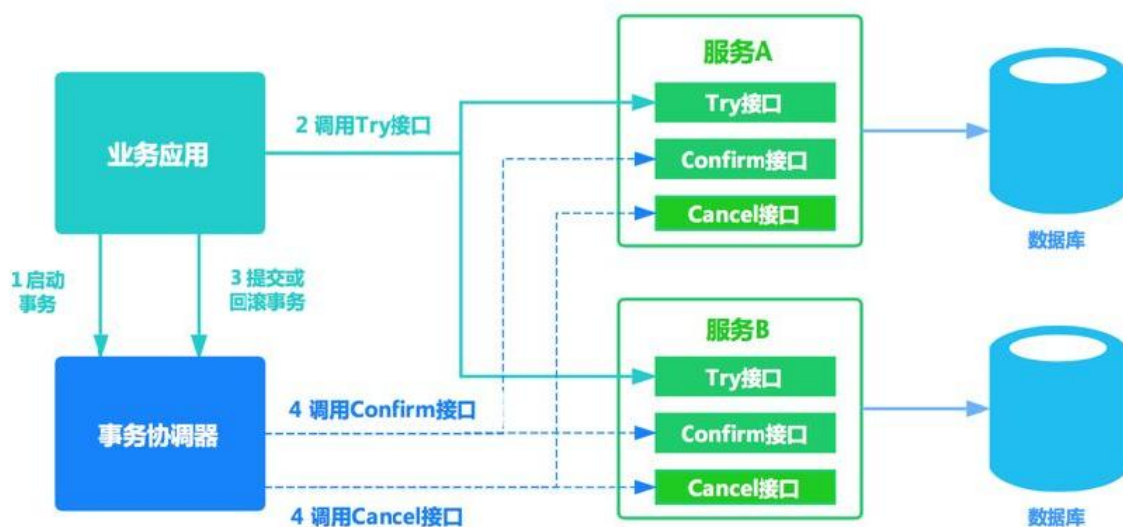
AT模式的依赖的还是依赖单个服务或单个数据源自己的事务控制(分支事务)，采用的是wal的思想，提交事务的时候同时记录undolog，如果全局事务成功，则删除undolog，如果失败，则使用undolog的数据回滚分支事务，最后删除undolog。

Seata TCC模式的流程图

TCC模式的特点是不再依赖于undolog，但是还是采用2阶段提交的方式：

第一阶段使用prepare尝试事务提交，第二阶段使用commit或者rollback让事务提交或者回滚。

引用网上一张TCC原理的参考图片



Seata TCC 事务的3个操作

TCC 将事务提交分为 Try - Confirm - Cancel 3个操作。

其和两阶段提交有点类似，Try为第一阶段，Confirm - Cancel为第二阶段，是一种应用层面侵入业务的两阶段提交。

操作方法	含义
Try	预留业务资源/数据效验
Confirm	确认执行业务操作，实际提交数据，不做任何业务检查，try成功，confirm必定成功，需保证幂等
Cancel	取消执行业务操作，实际回滚数据，需保证幂等

其核心在于将业务分为两个操作步骤完成。不依赖 RM 对分布式事务的支持，而是通过对业务逻辑的分解来实现分布式事务。

下面还以银行转账例子来说明

假设用户user表中有两个字段：可用余额(available_money)、冻结余额(frozen_money)

- A扣钱对应服务A(ServiceA)
- B加钱对应服务B(ServiceB)
- 转账订单服务(OrderService)
- 业务转账方法服务(BusinessService)

ServiceA，ServiceB，OrderService都需分别实现try()，confirm()，candle()方法，方法对应业务逻辑如下

	ServiceA	ServiceB	OrderService
try()	校验余额(并发控制) 冻结余额+1000 余额-1000	冻结余额+1000	创建转账订单，状态待转账
confirm()	冻结余额-1000	余额+1000 冻结余额-1000	状态变为转账成功
candle()	冻结余额-1000 余额+1000	冻结余额-1000	状态变为转账失败

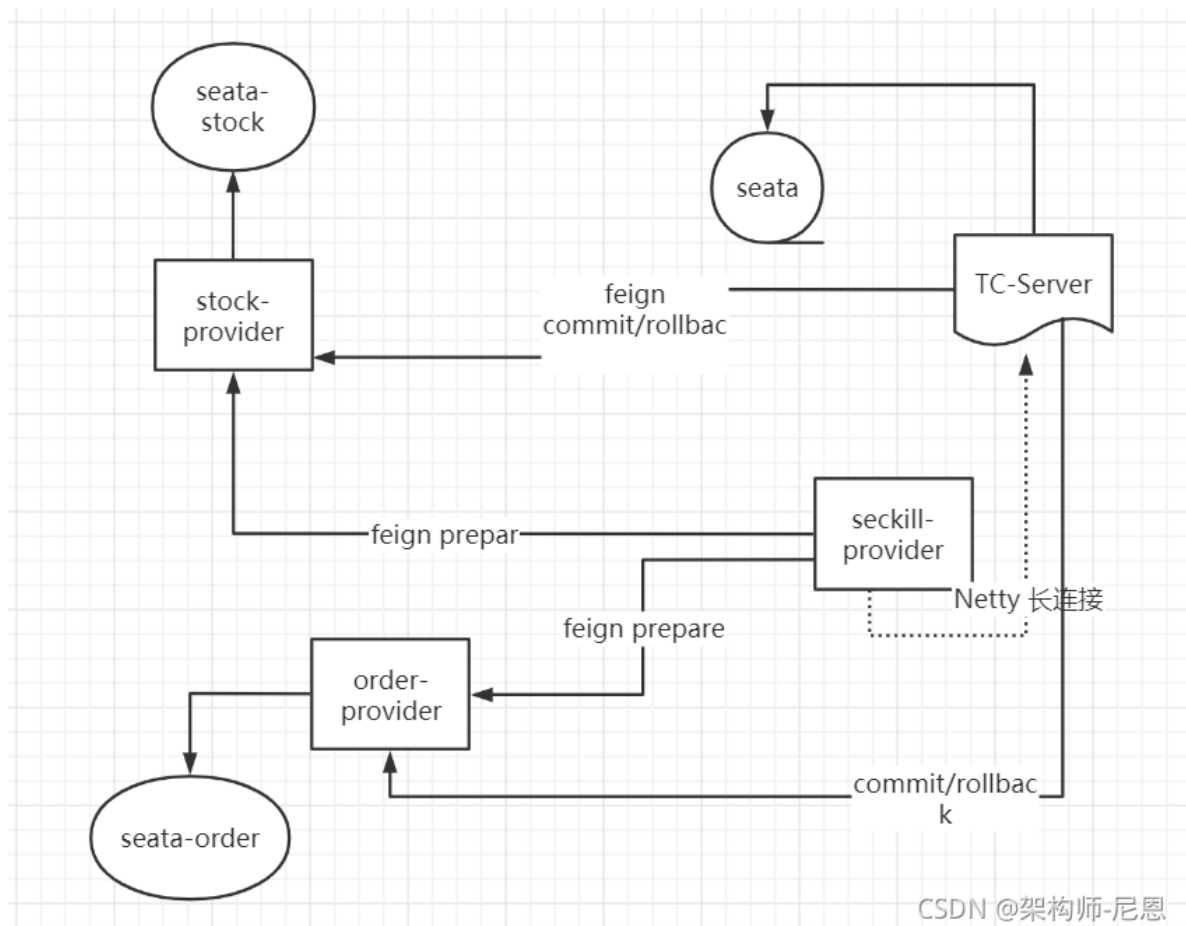
其中业务调用方BusinessService中就需要调用

- ServiceA.try()
- ServiceB.try()
- OrderService.try()

1、当所有try()方法均执行成功时，对全局事物进行提交，即由事物管理器调用每个微服务的confirm()方法

2、当任意一个方法try()失败(预留资源不足，抑或网络异常，代码异常等任何异常)，由事物管理器调用每个微服务的candle()方法对全局事务进行回滚

10WQPS秒杀的TCC分布式事务架构



库存服务

controller

```
package com.crazymaker.cloud.seata.seckill.controller;

@Slf4j
@RestController
@RequestMapping("/api/tcc/sku/")
@Api(tags = "商品库存")
public class SeataTCCStockController {
    @Resource
    SeataStockServiceImpl seckillSkuStockService;

    /**
     * minusStock 秒杀库存
     *
     * @return 商品 skuDTO
     */
    @PostMapping("/minusStock/v1")
    @ApiOperation(value = "减少秒杀库存")
    boolean minusStock(@RequestBody BusinessActionContext
        actionContext, @RequestParam("sku_id") Long skuId, @RequestParam("uid") Long uid)
    {
```

```

        boolean result = seckillSkuStockService.minusStock(actionContext,
skuId,uid);
        return result;
    }

    @ApiOperation(value = "提交")
    @PostMapping("/commit/v1")
    boolean commit(@RequestBody BusinessActionContext actionContext) {
        boolean result = seckillSkuStockService.commit(actionContext);
        return result;
    }

    @ApiOperation(value = "回滚")
    @PostMapping("/rollback/v1")
    boolean rollback(@RequestBody BusinessActionContext actionContext) {
        boolean result = seckillSkuStockService.rollback(actionContext);
        return result;
    }
}

```

service

```

package com.crazymaker.cloud.seata.seckill.impl;

@Configuration
@Slf4j
@Service
public class SeataStockServiceImpl {

    private Map<String, Statement> statementMap = new ConcurrentHashMap<>(100);
    private Map<String, Connection> connectionMap = new ConcurrentHashMap<>
(100);
    @Resource
    private DataSource dataSource;

    /**
     * 执行秒杀下单
     *
     * @param inDto
     * @param skuId
     * @return
     */
    // @Transactional
    public boolean minusStock(BusinessActionContext inDto, Long skuId, Long
userId) {
        Map<String, Object> params = inDto.getActionContext();

        try {
            log.info("减库存, prepare, xid:{", inDto.getXid());

```

```

        Connection connection = dataSource.getConnection();
        connection.setAutoCommit(false);

        int stock = 0;

        PreparedStatement pstmt = null;
        try {
            pstmt = connection.prepareStatement("SELECT `sku_id` , `stock_count` FROM `seckill_sku` WHERE `sku_id`=?" );
            pstmt.setLong(1, skuId);
            ResultSet resultSet = pstmt.executeQuery();
            if (resultSet.next()) {
                stock = resultSet.getInt("stock_count");
            }
            resultSet.close();
        } finally {
            if (pstmt != null) {
                pstmt.close();
            }
        }

        if (stock <= 0) {
            log.info("减库存, prepare 失败, xid:{}, inDto.getXid()", inDto.getXid());

            if (null != connection) {
                connection.close();
                connection.commit();
            }
            throw BusinessException.builder().errMsg("库存不够").build();
        }

        String sql = "UPDATE `seckill_sku` SET `stock_count` = `stock_count` -1 WHERE `sku_id` = ?";
        PreparedStatement stmt = connection.prepareStatement(sql);
        stmt.setLong(1, skuId);
        stmt.executeUpdate();
        statementMap.put(inDto.getXid(), stmt);
        connectionMap.put(inDto.getXid(), connection);

    } catch (SQLException e) {
        log.error("库存失败:", e);
        return false;
    }
    return true;
}

public boolean commit(BusinessActionContext dto) {

    String xid = dto.getXid();
    log.info("减库存, commit, xid:{}, xid");
    PreparedStatement statement = (PreparedStatement) statementMap.get(xid);
    Connection connection = connectionMap.get(xid);
    try {
        //判断一下, 防止空悬挂, 具备幂等性

```

```

        if (null != connection) {
            connection.rollback();
        }
    } catch (SQLException e) {
        log.error("提交失败:", e);
        return false;
    } finally {
        try {
            statementMap.remove(xid);
            connectionMap.remove(xid);
            if (null != statement) {
                statement.close();
            }
            if (null != connection) {
                connection.close();
            }
        } catch (SQLException e) {
            log.error("减库存回滚事务后归还连接失败:", e);
        }
    }
    return true;
}

public boolean rollback(BusinessActionContext dto) {
    String xid = dto.getXid();
    log.info("减库存, rollback, xid:{}, xid)", xid);
    PreparedStatement statement = (PreparedStatement) statementMap.get(xid);
    Connection connection = connectionMap.get(xid);
    try {
        if (null != connection) {
            connection.commit();
        }
    } catch (SQLException e) {
        log.error("回滚失败:", e);
        return false;
    } finally {
        try {
            statementMap.remove(xid);
            connectionMap.remove(xid);
            if (null != statement) {
                statement.close();
            }
            if (null != connection) {
                connection.close();
            }
        } catch (SQLException e) {
            log.error("减库存提交事务后归还连接池失败:", e);
        }
    }
    return true;
}
}
}

```

订单服务

controller

```
@RestController
@RequestMapping("/api/tcc/order/")
@Api(tags = "秒杀练习 订单管理")
public class SeataTCCOrderController {
    @Resource
    TCCOrderServiceImpl seckillOrdersService;

    /**
     * 执行秒杀的操作
     * <p>
     * <p>
     * {
     * "exposedKey": "4b70903f6e1aa87788d3ea962f8b2f0e",
     * "newStockNum": 10000,
     * "seckillSkuId": 1157197244718385152,
     * "seckillToken": "0f8459cbae1748c7b14e4cea3d991000",
     * "userId": 37
     * }
     *
     * @return
     */
    @ApiOperation(value = "下订单")
    @PostMapping("/addOrder/v1")
    boolean addOrder(@RequestBody BusinessActionContext actionContext,
        @RequestParam("sku_id") Long skuId, @RequestParam("uid") Long uid) {

        boolean orderDTO = seckillOrdersService.addOrder(actionContext,
            skuId, uid);
        return orderDTO;
    }

    @ApiOperation(value = "下订单提交")
    @PostMapping("/commit/v1")
    boolean commit(@RequestBody BusinessActionContext actionContext) {
        boolean orderDTO = seckillOrdersService.commitAddOrder(actionContext);
        return orderDTO;
    }

    @ApiOperation(value = "下订单回滚")
    @PostMapping("/rollback/v1")
    boolean rollback(@RequestBody BusinessActionContext actionContext) {
        boolean orderDTO = seckillOrdersService.rollbackAddOrder(actionContext);
        return orderDTO;
    }
}
```

service

```

@Slf4j
@Service
public class TCCOrderServiceImpl {

    private Map<String, Statement> statementMap = new ConcurrentHashMap<>(100);
    private Map<String, Connection> connectionMap = new ConcurrentHashMap<>(100);
    @Resource
    private DataSource dataSource;

    private IdGenerator idGenerator;

    public IdGenerator getIdGenerator() {
        if (null == idGenerator) {
            idGenerator = CommonSnowflakeIdGenerator.getFromMap("tcc_order");
        }
        return idGenerator;
    }

    /**
     * 执行秒杀下单
     *
     * @param inDto
     * @return
     */
    // @Transactional //开启本地事务
    // @GlobalTransactional//不，开启全局事务（重点） 使用 seata 的全局事务
    public boolean addOrder(BusinessActionContext inDto, Long skuId, Long
        userId) {

        Map<String, Object> params = inDto.getActionContext();

        //      long skuId = (Long) params.get("sku");
        //      Long userId = (Long) params.get("user");
        Long id = getIdGenerator().nextId();
        try {
            Connection connection = dataSource.getConnection();
            connection.setAutoCommit(false);

            boolean isExist = false;
            log.info("检查是否已经下单过");
            PreparedStatement pstmt = null;
            try {
                pstmt = connection.prepareStatement("SELECT * FROM `seckill_order` WHERE `user_id` =?");
                pstmt.setLong(1, userId);
                ResultSet resultSet = pstmt.executeQuery();
                if (resultSet.next()) {
                    isExist = true;
                }
                resultSet.close();
            } finally {
                if (pstmt != null) {
                    pstmt.close();
                }
            }
        }
    }
}

```



```

        if (isExist) {
            log.info("已经下单过");

            if (null != connection) {
                try {
                    connection.close();
                    connection.commit();
                } catch (Throwable t) {
                }
            }

            throw BusinessException.builder().errMsg("已经秒杀过了").build();
        }
        log.info("pass: 检查是否已经下单过");

        String sql = "INSERT INTO `seckill_order`(`order_id`, `sku_id`,
`status`, `user_id`) VALUES( ?, ?, 1, ?)";
        PreparedStatement stmt = connection.prepareStatement(sql);
        stmt.setLong(1, id);
        stmt.setLong(2, skuId);
        stmt.setLong(3, userId);
        stmt.executeUpdate();
        statementMap.put(inDto.getXid(), stmt);
        connectionMap.put(inDto.getXid(), connection);

        log.info("prepare 下单 完成");

        return true;
    } catch (SQLException e) {
        log.error("保存订单失败:", e);
        return false;
    }
}

public boolean commitAddOrder(BusinessActionContext dto) {
    String xid = dto.getXid();
    log.info("提交 下订单, commit, xid:{}, xid)", xid);
    PreparedStatement statement = (PreparedStatement) statementMap.get(xid);
    Connection connection = connectionMap.get(xid);
    try {
        if (null != connection) {
            connection.commit();
        }
    } catch (SQLException e) {
        log.error("提交失败:", e);
        return false;
    } finally {
        try {
            statementMap.remove(xid);
            connectionMap.remove(xid);
            if (null != statement) {
                statement.close();
            }
            if (null != connection) {
                connection.close();
            }
        }
    }
}

```

```

        }
    } catch (SQLException e) {
        log.error("下订单提交事务后归还连接池失败:", e);
    }
}
return true;
}

public boolean rollbackAddOrder(BusinessActionContext dto) {
    String xid = dto.getXid();
    log.info("回滚 下订单, rollback, xid:{}, xid)", xid);
    PreparedStatement statement = (PreparedStatement) statementMap.get(xid);
    Connection connection = connectionMap.get(xid);
    try {
        //判断一下, 防止空悬挂, 具备幂等性
        if (null != connection) {
            connection.rollback();
        }
    } catch (SQLException e) {
        log.error("回滚失败:", e);
        return false;
    } finally {
        try {
            statementMap.remove(xid);
            connectionMap.remove(xid);
            if (null != statement) {
                statement.close();
            }
            if (null != connection) {
                connection.close();
            }
        } catch (SQLException e) {
            log.error("下订单回滚事务后归还连接失败:", e);
        }
    }
    return true;
}
}

```

秒杀服务

controller

```

@RestController
@RequestMapping("/api/seckill/seglock/")
@Api(tags = "秒杀练习分布式事务 版本")
public class SeckillTCCController {

    @Resource
    TCCSeckillServiceImpl seataSeckillServiceImpl;

    /**
     * 执行秒杀的操作
     */
}

```

```

    * 减库存，下订单
    * <p>
    * {
    * "exposedKey": "4b70903f6e1aa87788d3ea962f8b2f0e",
    * "newStockNum": 10000,
    * "seckillSkuId": 1247695238068177920,
    * "seckillToken": "0f8459cbae1748c7b14e4cea3d991000",
    * "userId": 37
    * }
    *
    * @return
    */
@ApiOperation(value = "秒杀")
@PostMapping("/doSeckill/v1")
RestOut<SeckillDTO> doSeckill(@RequestBody SeckillDTO dto) {

    seataSeckillServiceImpl.doSeckill(dto);

    return RestOut.success(dto).setRespMsg("秒杀成功");
}

}

```

service

```

@Slf4j
@Service
public class TCCSeckillServiceImpl {

    @Autowired
    private OrderApi orderApi;
    @Autowired
    private StockApi stockApi ;

    /**
     * 减库存，下订单
     */
    //开启全局事务（重点） 使用 seata 的全局事务
    @GlobalTransactional
    public boolean doSeckill(@RequestBody SeckillDTO dto) {

        String xid = RootContext.getXID();
        log.info("----->分布式操作开始");
        BusinessActionContext actionContext = new BusinessActionContext();
        actionContext.setXid(xid);
        Long skuId=dto.getSeckillSkuId();
        Long uId=dto.getUserId();

        //远程方法 扣减库存
        log.info("----->扣减库存开始storage中");
        boolean result = stockApi.prepare(actionContext,skuId,uId);
        if (!result) {
            throw new RuntimeException("扣减库存失败");
        }
    }
}

```

```

        result = orderApi.prepare(actionContext,skuId,uId);

        if (!result) {
            throw new RuntimeException("保存订单失败");
        }

        log.info("----->分布式下订单操作完成");
        //        throw new RuntimeException("调用2阶段提交的rollback方法");
        return true;
    }
}

```

以下两个实验，请参见配套视频

基于TCC的分布式事务的提交实验

基于TCC的分布式事务的回滚实验

一个金额扣减的例子：

开始事务之前，比如有个商品价格 money=30，账户表中有 money=100, 此时我们发起下单请求。

	冻结金额 frozen	全部金额 total	可用金额 available
开始事务之前	0	100	100
try	0+30	100	100-30
commit	30-30	100-30	70-0
rollback	30-30	100	100-0

第一阶段try阶段：通过 prepare 方法中冻结了一笔金额为30的订单，那么就账户表的可用金额 available money 这时候就应该是100-30=70，但这里还需要一个冻结余额的字段：frozen=0+30=30。

第二阶段commit：如果后续业务全部执行成功，那么第二阶段就会去调用 commit 方法，我们在 commit 方法中的减掉相应的冻结金额 frozen=30-30=0，并且正式减掉总金额 total=100-30=70。

第二阶段rollback：如果后续有其他业务的try方法执行异常，需要回滚，那么第二阶段就调用我们实现的 rollback 方法，我们在 rollback 方法total不变、frozen=30-30=0。

三个方法都是我们自己实现的，但在调用的时候，我们自己只会调用 prepare 方法，commit 和 rollback 都是框架帮我们去调的。

一个库存扣减操作：

开始事务之前，比如有个商品库存 =100, 此时我们发起下单请求。

	冻结库存 freezed	全部库存 total	可用库存 available
开始事务之前	0	100	100
try	0+30	100	100-30
commit	30-30	100-30	70-0
rollback	30-30	100	100-0

第一阶段try阶段：通过 prepare 方法中，扣减了30个库存，那么可用库存 available 这时候就应该是 $100-30=70$ ，但这里还需要一个冻结库存的字段： $freezed=0+30=30$ 。

第二阶段commit：如果后续业务全部执行成功，那么第二阶段就会去调用 commit 方法，我们在 commit 方法中的减掉相应的冻结库存 $freezed=30-30=0$ ，并且正式减掉总库存 $total=100-30=70$ 。

第二阶段rollback：如果后续有其他业务的try方法执行异常，需要回滚，那么第二阶段就调用我们实现的 rollback 方法，我们在 rollback 方法total不变、freezed 冻结库存 $=30-30=0$ 。

三个方法都是我们自己实现的，但在调用的时候，我们自己只会调用 prepare 方法，commit 和 rollback 都是框架帮我们去调的。

Seata TCC 的3大优势

TCC 模式最大的优势是效率高。

优势1： try 阶段的资源锁定不需要阻塞等待

TCC 模式在 try 阶段的锁定资源并不是真正意义上的锁定，而是真实提交了本地事务，将资源预留到中间状态，并不需要阻塞等待，因此效率比其他模式要高。

优势1：异步提交优化

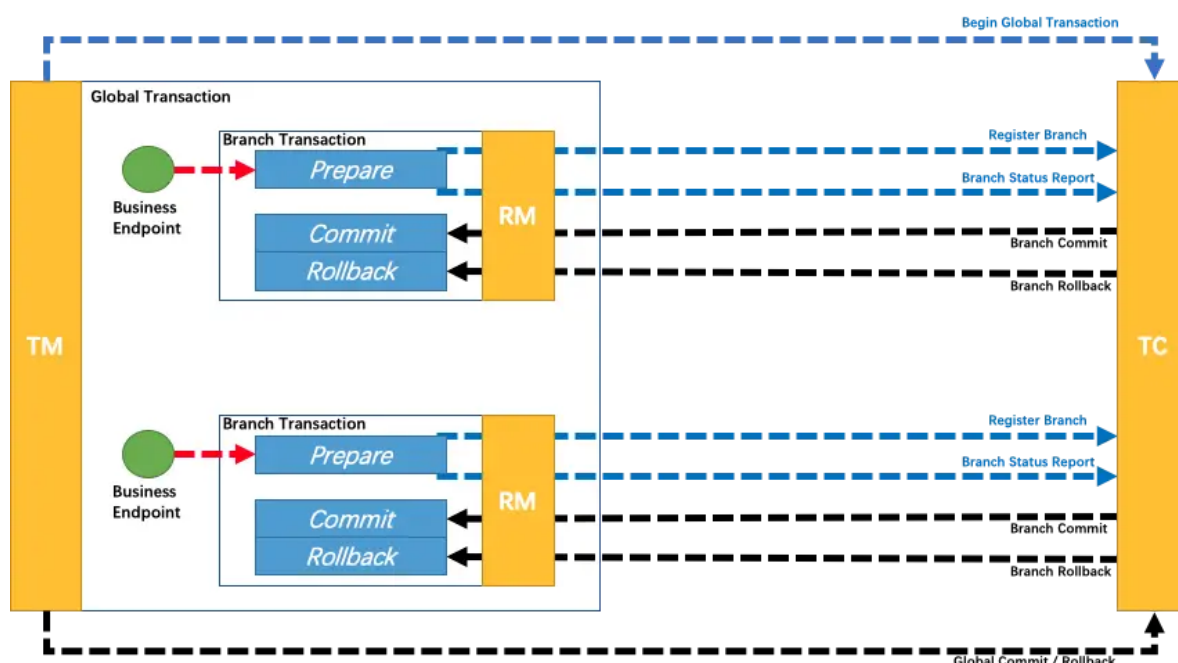
try 阶段成功后，立即进入 confirm/cancel 阶段，而是认为全局事务已经结束了，启动定时任务来异步执行 confirm/cancel，扣减或释放资源，这样会有很大的性能提升。

优势2: RPC 次数减少了 50%

TCC 模式中有三个角色：

- TM：管理全局事务，包括开启全局事务，提交/回滚全局事务；
- RM：管理分支事务；
- TC：管理全局事务和分支事务的状态。

下图来自 Seata 官网：



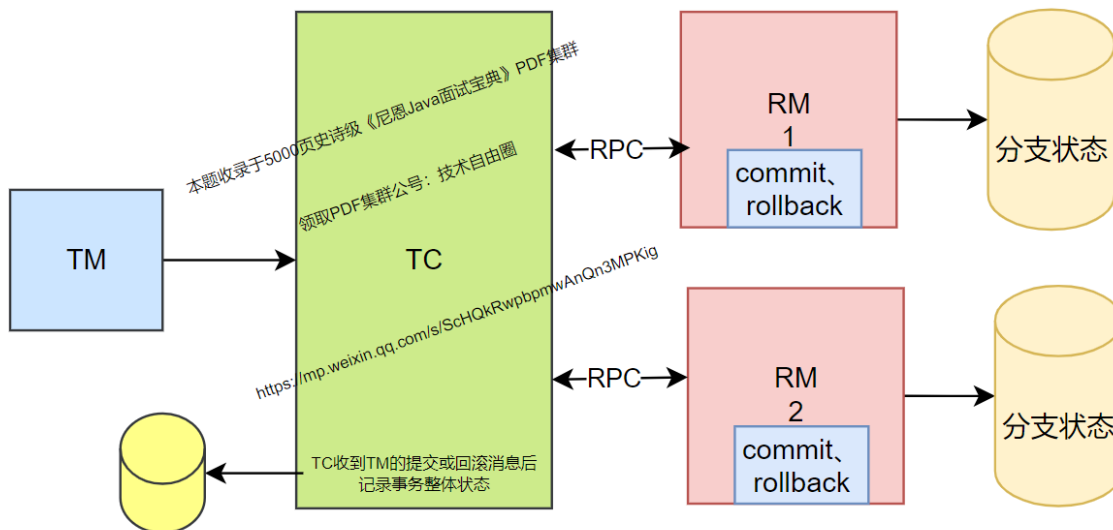
TM 开启全局事务时，RM 需要向 TC 发送注册消息，TC 保存分支事务的状态。TM 请求提交或回滚时，TC 需要向 RM 发送提交或回复消息。这样包含两个分支事务的分布式事务中，TC 和 RM 之间有四次 RPC。

TM 开启全局事务时，RM 需要向 TC 发送注册消息，TC 保存分支事务的状态。

TM 请求提交或回滚时，TC 需要向 RM 发送提交或回复消息。

这样，在包含两个分支事务的分布式事务中，TC 和 RM 之间有四次 RPC。

优化后的流程如下图：



TC 保存全局事务的状态。

TM 开启全局事务时，RM 不再需要向 TC 发送注册消息，而是把分支事务状态保存在本地。

TM 向 TC 发送提交或回滚消息后，RM 异步线程首先查出本地保存的未提交分支事务，然后向 TC 发送消息获取（本地分支事务所在地）全局事务状态，以决定是提交还是回滚本地事务。

这样优化后，RPC 次数减少了 50%，性能大幅提升。

Seata TCC 事务的3大常见问题

Seata TCC 事务的常见问题，主要有三个：

- 幂等问题
- 空回滚
- 悬挂问题

TCC 问题1：幂等问题

幂等就是对于同一个分布式事务的同一个分支事务，重复去调用该分支事务的第二阶段接口 Commit/Cancel，因此，要求 TCC 的二阶段 Confirm 和 Cancel 接口保证幂等，不会重复使用或者释放资源。

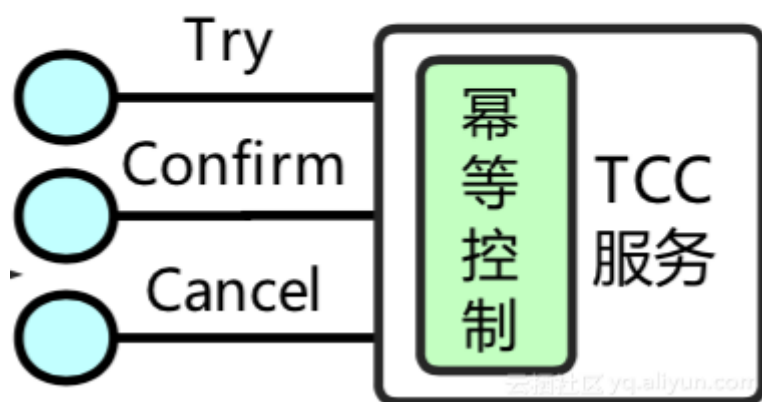
如果幂等控制没有做好，很有可能导致资损等严重问题。

幂等控制出现原因：

- 网络异常

使用TCC时要注意Try - Confirm - Cancel 3个操作的**幂等控制**，网络原因，或者重试操作都有可能造成这几个操作的重复执行

资源管理器



业务实现过程中需重点关注幂等实现，讲到幂等，以上述TCC转账例子中confirm()方法来说明在confirm()方法中 余额-1000，冻结余额-1000，这一步是实现幂等性的关键，你会怎么做？

大家在自己系统里操作资金账户时，为了防止并发情况下数据不一致的出现，肯定会避免出现这种代码。

因为这本质上是一个 读-改-写的过程，不是原子的，在并发情况下会出现数据不一致问题

所以最简单的做法是

这利用了数据库行锁特性解决了并发情况下的数据不一致问题，但是TCC中，单纯使用这个方法适用么？

答案是不行的，该方法能解决并发单次操作下的扣减余额问题，但是不能解决多次操作带来的多次扣减问题，假设我执行了两次，按这种方案，用户账户就少了2000块

那么具体怎么做？

上诉转账例子中，可以引入转账订单状态来做判断，若订单状态为已支付，则直接return

当然，新建一张去重表，用订单id做唯一建，若插入报错返回也是可以的，不管怎么样，核心就是保证操作幂等性

同样的也是在 TCC 事务控制表中增加一个记录状态的字段 status，该字段有 3 个值，分别为：

1. tried: 1
2. committed: 2
3. rollbacked: 3

二阶段 Confirm/Cancel 方法执行后，将状态改为 committed 或 rollbacked 状态。

当重复调用二阶段 Confirm/Cancel 方法时，判断事务状态即可解决幂等问题。

TCC 问题2：空回滚

空回滚就是对于一个分布式事务，在没有调用 RM资源的 Try 方法的情况下，调用了二阶段的 Cancel 方法。

空回滚，顾名思义，就是 没有 Try 只有cancel。

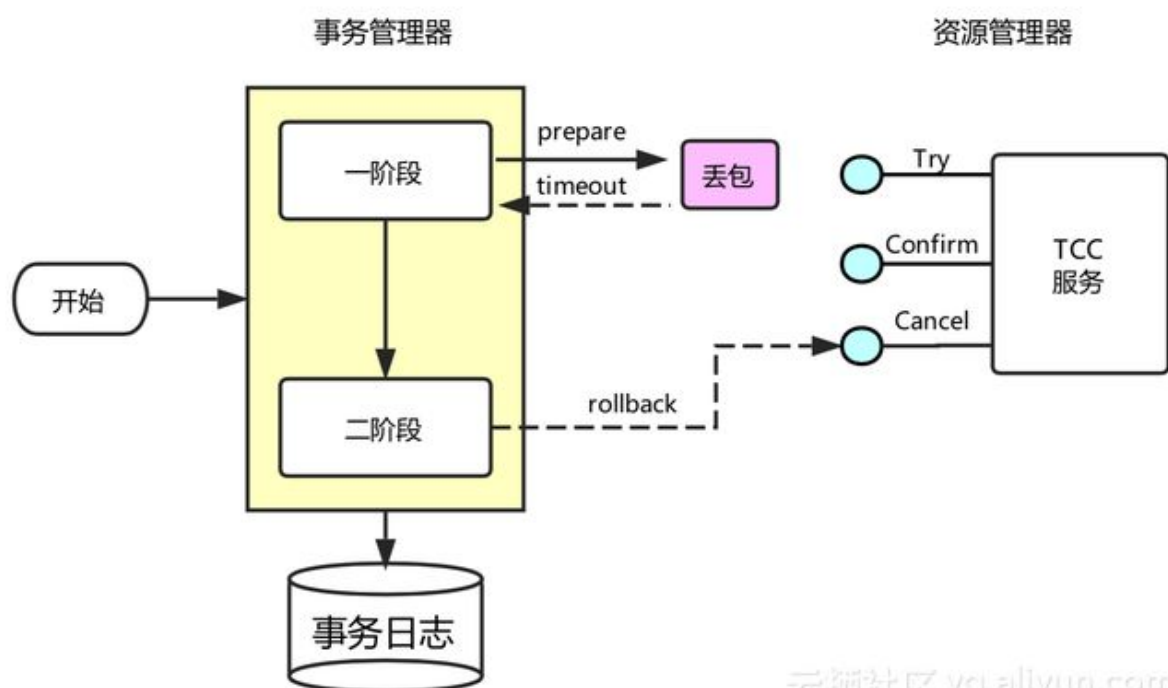
Cancel 方法需要识别出这是一个空回滚，然后直接返回成功。

出现原因：

- Try超时（丢包）>> 分布式事务回滚触发cancel >> RM 未收到try、收到cancel

如下图所示，事务协调器在调用TCC服务的一阶段Try操作时，可能会出现因为丢包而导致的网络超时，此时事务协调器会触发二阶段回滚，调用TCC服务的Cancel操作；

TCC服务在未收到Try请求的情况下收到Cancel请求，这种场景被称为空回滚；TCC服务在实现时应当允许空回滚的执行；



那么具体代码里怎么做呢？

分析下，如果try()方法没执行，那么订单一定没创建，所以cancle方法里可以加一个判断，如果上下文中订单编号orderNo不存在或者订单不存在，直接return

核心思想就是 **回滚请求处理时，如果对应的具体业务数据为空，则返回成功**

当然这种问题也可以通过中间件层面来实现，如，在第一阶段try()执行完后，向一张事务表中插入一条数据(包含事务id，分支id)，cancle()执行时，判断如果没有事务记录则直接返回，但是现在还不支持

解决方案：

要想防止空回滚，那么必须在 Cancel 方法中识别这是一个空回滚，在二阶段执行回滚 rollback 的时候，需要先检查一阶段是否有执行过 try 方法，如果执行过才能执行回滚 rollback 方法，如果没有执行过就不任何操作，

Seata 的做法是新增一个 TCC 事务控制表 tcc_fence_log，

在 try 阶段执行成功后在 tcc_fence_log 表中插入一条记录，在 rollback 时去查询 tcc_fence_log 表是否有 try 阶段执行成功的记录，如果有，才会执行 rollback，如果不存在记录说明 Try 方法没有执行，则不再执行 rollback 方法。

TCC 问题3：防悬挂

悬挂 就是倒挂， 第二阶段的动作，跑到第一阶段全面去了。

悬挂 具体来说，就是Commit/Cancel 比 Try 接口先执行。

出现的原因是 Try 由于网络拥堵而超时，事务管理器生成回滚，触发 Cancel 接口，而最终又收到了 Try 接口调用，但是 Cancel 比 Try 先到。

按照前面允许空回滚的逻辑，回滚会返回成功，事务管理器认为事务已回滚成功，则此时的 Try 接口不应该执行，否则会产生数据不一致

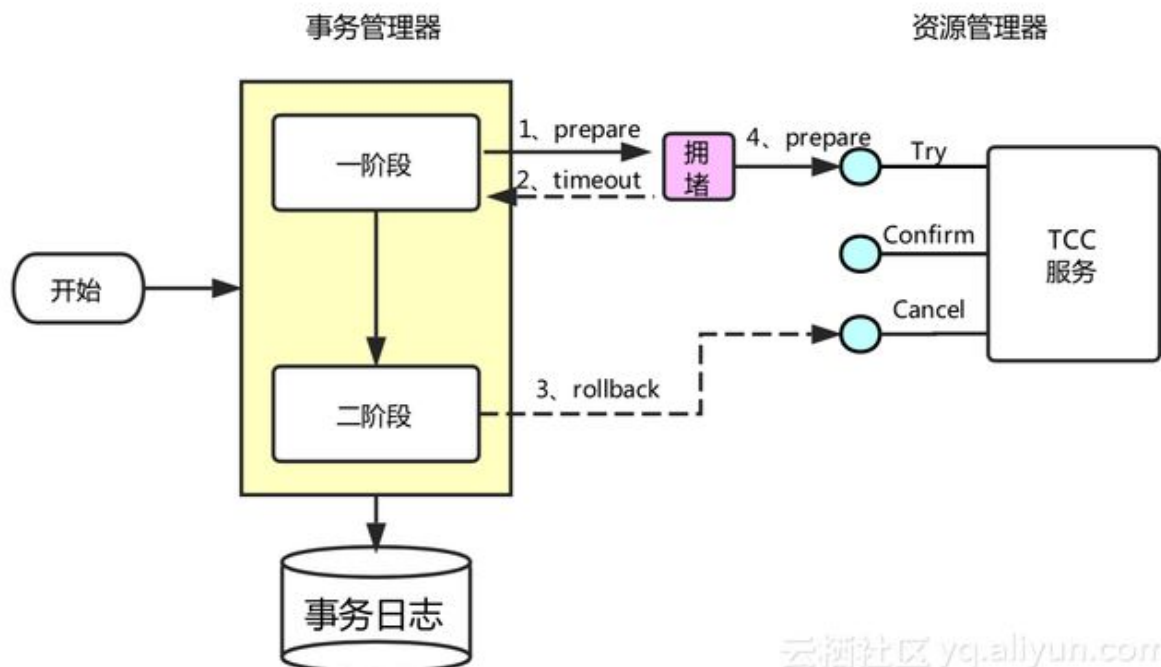
出现原因：

- Try超时（拥堵） >> 分布式事务回滚触发cancel >> Try到达

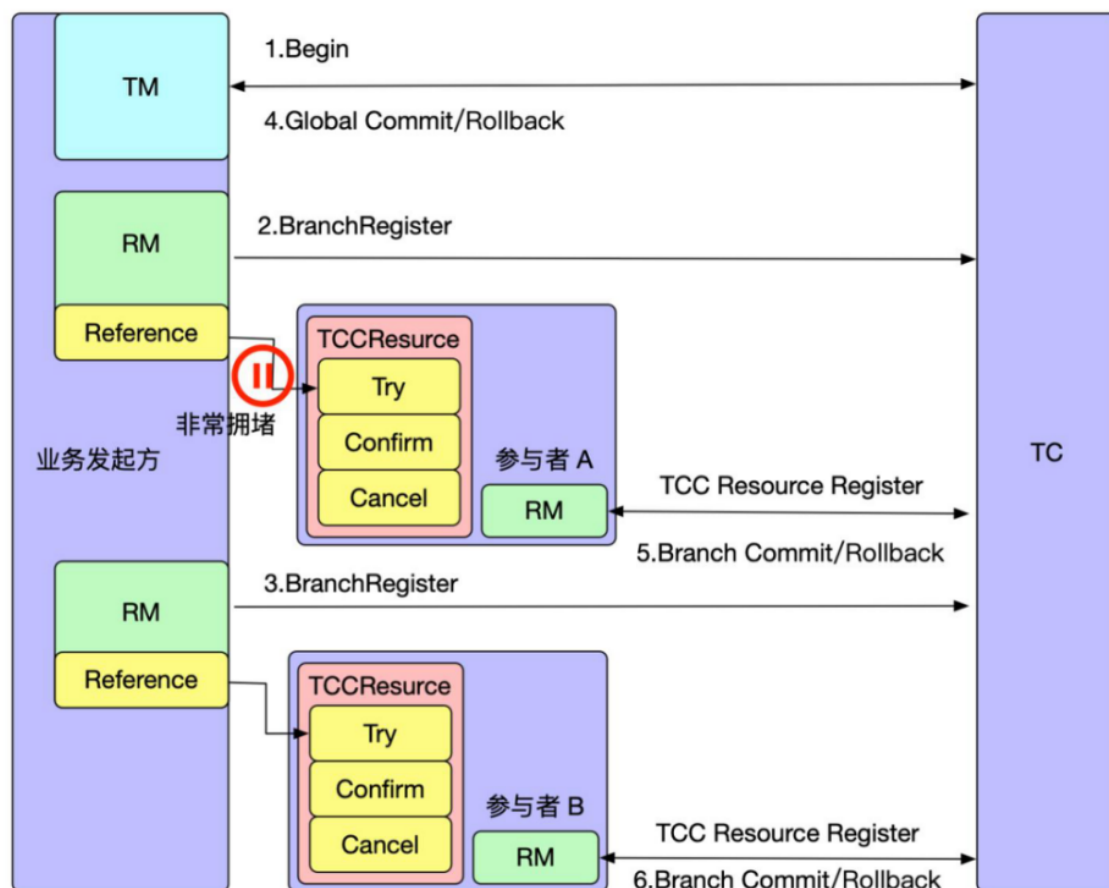
如下图所示，事务协调器在调用TCC服务的一阶段Try操作时，可能会出现因网络拥堵而导致的超时，此时事务协调器会触发二阶段回滚，调用TCC服务的Cancel操作；

在此之后，拥堵在网络上的一阶段Try数据包被TCC服务收到，出现了二阶段Cancel请求比一阶段Try请求先执行的情况；

用户在实现TCC服务时，应当允许空回滚，但是要拒绝执行空回滚之后到来的一阶段Try请求；



再来一张详细的图：



如上图所示，在执行参与者 A 的一阶段 Try 方法时，出现网路拥堵，由于 Seata 全局事务有超时限制，执行 Try 方法超时后，

TC决议全局回滚，回滚完成后如果此时 Try 请求才到达参与者 A，执行 Try 方法进行资源预留，从而造成悬挂/倒挂。

另外一种情况：try 执行的比较慢，导致调用 try 的服务超时了，这时候TC就去调了 cancel，调完 cancel 后 try 执行成功了，这时候 try 的资源无法回滚。

解决方案：一旦出现悬挂/倒挂（先执行cancel），就不能够让 try 执行成功，为啥呢？因为倒挂之后，如果 try 执行成功了，后面就没法提交/回滚了。

这里又怎么办呢？

- 可以在二阶段执行时插入一条事务控制记录，状态为已回滚，
- 这样当一阶段执行时，先检查事务控制记录，如果记录存在，就认为二阶段回滚操作已经执行，不再执行try方法；

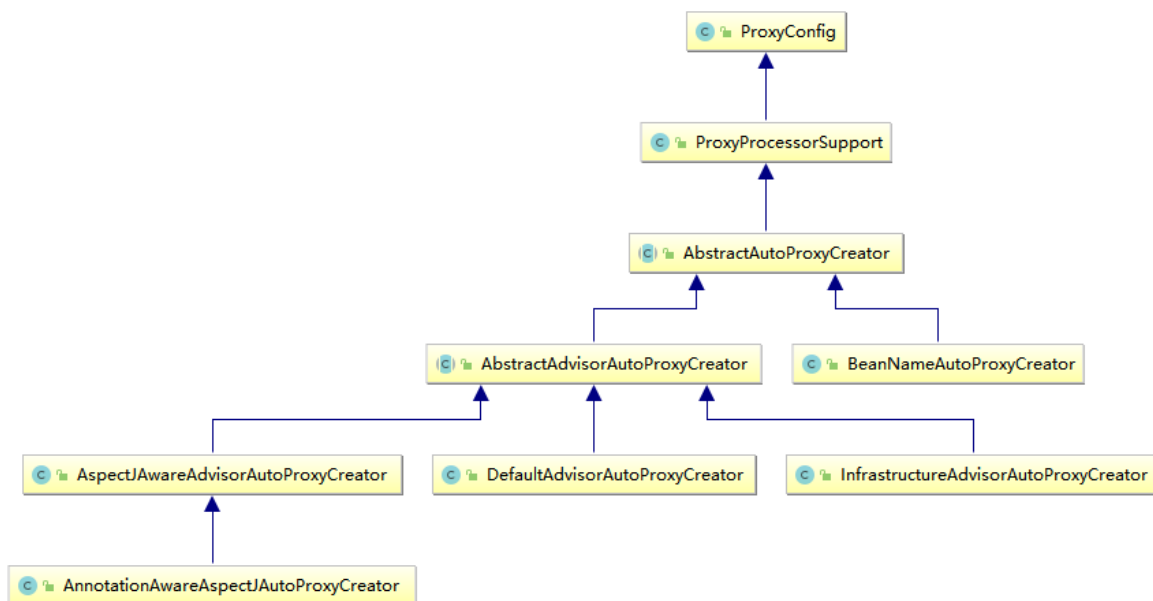
Seata TCC核心源码分析

GlobalTransactionScanner事务切面的注解扫描器

Seata全局事务发起、提交/回滚依赖于SpringAop，GlobalTransactionScanner则是aop的入口类。

一般来说,这种Scanner 继承于AbstractAutoProxyCreator, 会重写ProxyCreator的wrapIfNecessary方法。

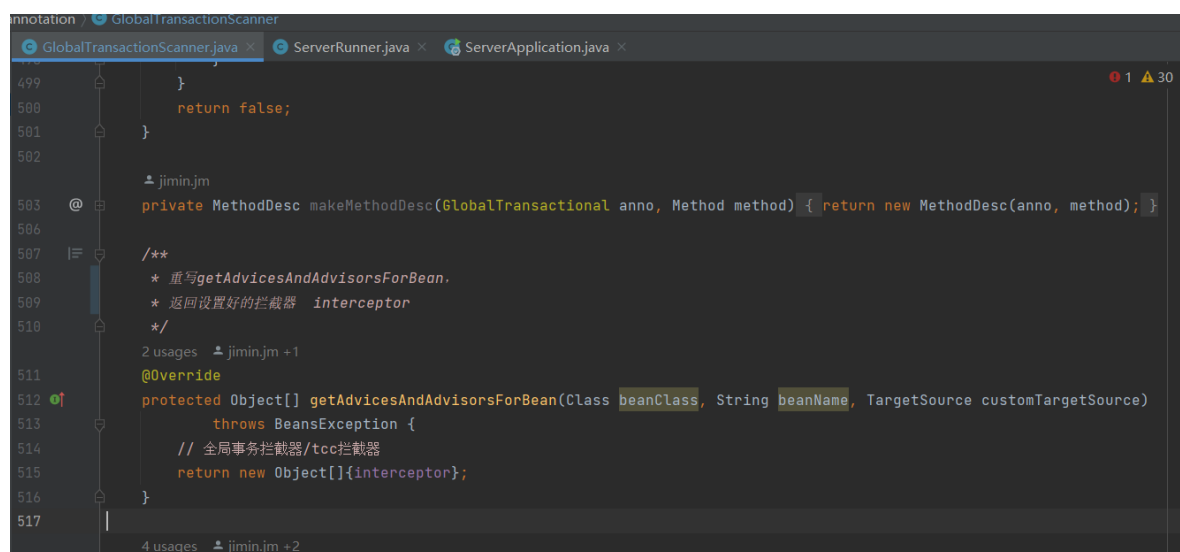
AbstractAutoProxyCreator 的继承结构如下:



注意: AbstractAutoProxyCreator 是一个抽象基类,它的如下抽象方法必须由实现子类提供:

```
// 找到某个 bean 专用的拦截器对象
protected abstract Object[] getAdvisesAndAdvisorsForBean(Class<?> beanClass,
String beanName, @Nullable TargetSource customTargetSource) throws
BeansException;
```

GlobalTransactionScanner事务切面的注解扫描器, 就实现了这个方法



以上注释, 来自尼恩的注释版本 seata 源码。

基础知识: Spring AOP自动代理创建流程

当用户获取 service bean时(也就是 `getBean("service")`), AOP通过 `BeanPostProcess` 后置处理器操作进行介入 分为2种情况:

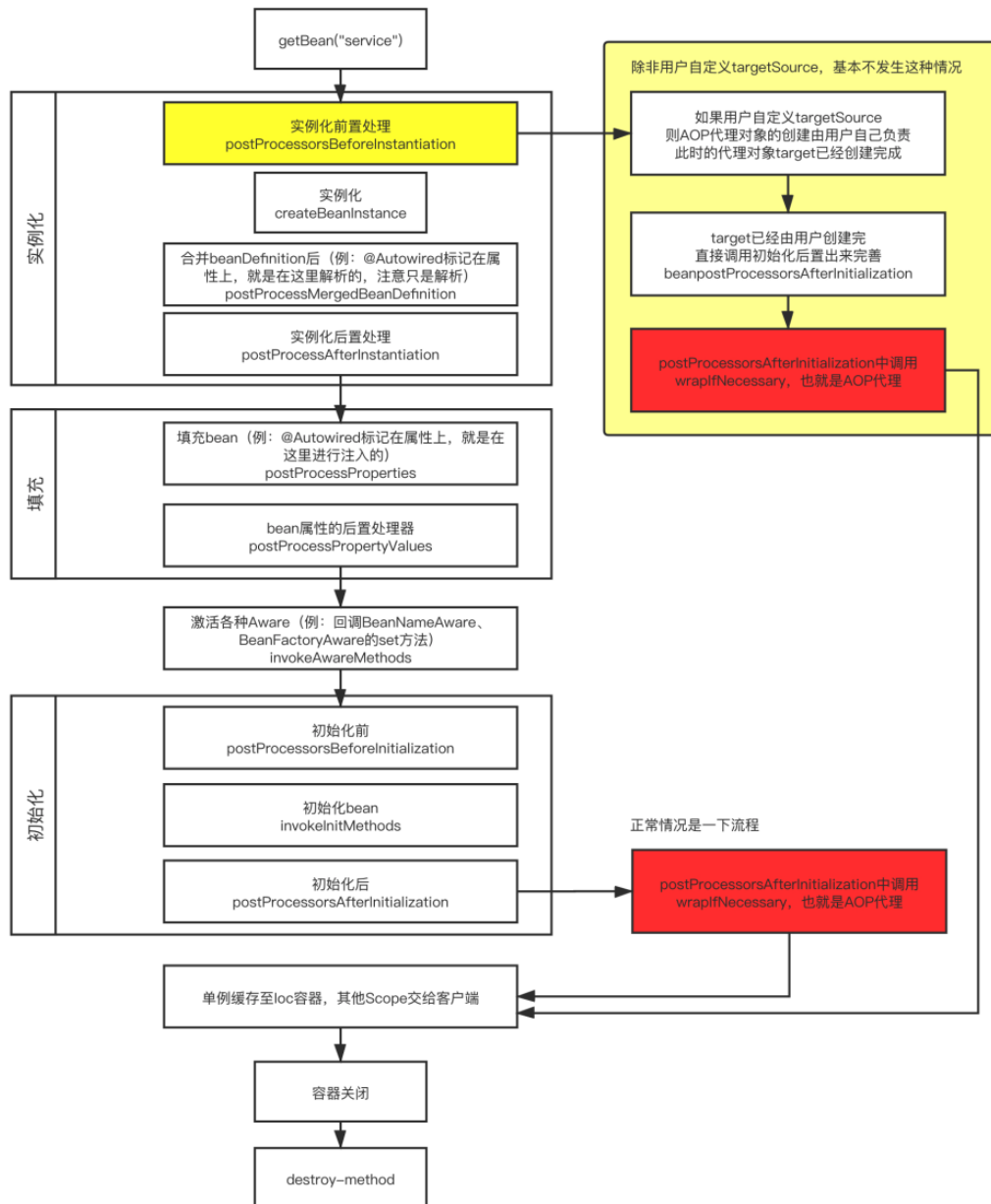
- 用户自定义了 `targetSource`，则bean的创建(实例化、填充、初始化)均由用户负责，Spring loc 不会在管该代理目标对象target

这种情况基本上不会发生，很多人用了几年Spring可能都不知道有它的存在

- 正常情况下都是Spring loc完成代理对象target的实例化、填充、初始化。然后在初始化后置处理器中进行介入，对bean也就是service进行代理

这是普遍情况

Spring AOP代理时机：



AbstractAutoProxyCreator 的 `wrapIfNecessary` 创建代理操作，所有的重点都在 `wrapIfNecessary` 做了什么操作

Spring AOP创建代理:

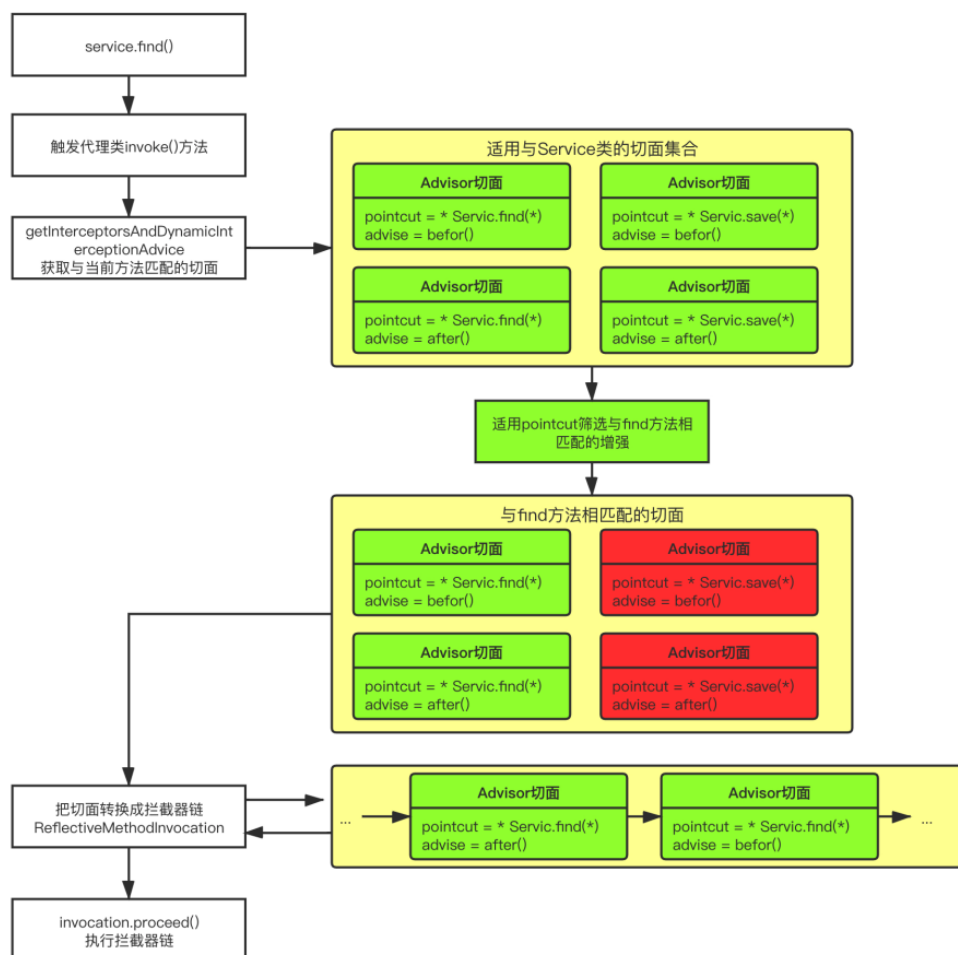


<http://log.p0rt.com/2016/04/20/aop/>

接下来，看看这些 AOP动态代理的方法调用。

创建完代理，代理如果调用方法呢？我们以Jdk动态代理为例子，方法的调用将会触发`invoke`方法。

Spring AOP代理方法调用：

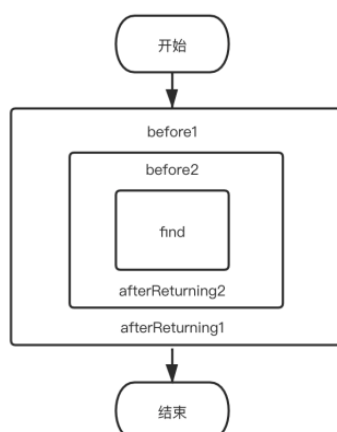


再看看 Spring AOP拦截器的执行顺序。

从上面可以看出，Spring AOP的代理invoke方法，其实是 interceptor 拦截器的执行。

Spring AOP的执行顺序，跟栈很像，后进先出

Spring AOP 拦截器链调用：



TransactionalInterceptor 源代码拦截器的创建

GlobalTransactionScanner 的wrapIfNecessary 方法，创建interceptor，生成代理对象来完成代理逻辑的处理。

```

265 @Override
266 protected Object wrapIfNecessary(Object bean, String beanName, Object cacheKey) {
267     // do checkers
268     if (!doCheckers(bean, beanName)) return bean;
269     try {
270         synchronized (PROXYED_SET) {
271             if (PROXYED_SET.contains(beanName)) {
272                 return bean;
273             }
274             interceptor = null;
275             //check TCC proxy
276             // tcc代理bean
277             // 判断接口上是否有@LocalTCC && 接口方法上有@TwoPhaseBusinessAction 注解
278             // 同时将@TwoPhaseBusinessAction注解内容解析成tccResource, 注册到缓存中, key值为TwoPhaseBusinessAction#name属性。
279             // 以供全局2阶段从缓存中获取tccResource
280             if (TCCBeanParserUtils.isTccAutoProxy(bean, beanName, applicationContext)) {
281                 // 若开启了tcc 幂等、空回滚、防悬挂控制 (TwoPhaseBusinessAction#useTCCFence为true),
282                 // 定时清理客户端的tcc_fence_log表记录
283                 // init tcc fence clean task if enable useTccFence
284                 TCCBeanParserUtils.initTccFenceCleanTask(TCCBeanParserUtils.getRemotingDesc(beanName), applicationContext);
285                 //TCC interceptor, proxy bean of sofa:reference/dubbo:reference, and LocalTCC
286                 //设置tcc代理拦截器
287                 interceptor = new TccActionInterceptor(TCCBeanParserUtils.getRemotingDesc(beanName));
288                 ConfigurationCache.addConfigListener(ConfigurationKeys.DISABLE_GLOBAL_TRANSACTION,
289                     (ConfigurationChangeListener)interceptor);
290             } else {
291                 // 非tcc模式bean
292             }
293         }

```

以上注释，来自尼恩的 注释版本 seata 源码。

从源码中看到，GlobalTransactionScanner添加了全局事务/tcc分支事务的代理拦截，

```

291 } else {
292     // 非tcc模式bean
293
294     Class<?> serviceInterface = SpringProxyUtils.findTargetClass(bean);
295     Class<?>[] interfacesIfJdk = SpringProxyUtils.findInterfaces(bean);
296     // 查找类和方法上是否有全局事务@GlobalTransactional注解
297     if (!existsAnnotation(new Class[]{serviceInterface})
298         && !existsAnnotation(interfacesIfJdk)) {
299         return bean;
300     }
301     // 设置全局事务代理拦截器。
302     // 说明@GlobalTransactional和tcc相关注解不能标准在同一类上
303     if (globalTransactionalInterceptor == null) {
304         globalTransactionalInterceptor = new GlobalTransactionalInterceptor(failureHandlerHook);
305         ConfigurationCache.addConfigListener(
306             ConfigurationKeys.DISABLE_GLOBAL_TRANSACTION,
307             (ConfigurationChangeListener)globalTransactionalInterceptor);
308     }
309     interceptor = globalTransactionalInterceptor;
310 }
311

```

以上注释，来自尼恩的 注释版本 seata 源码。

具体拦截逻辑分别在GlobalTransactionalInterceptor和TccActionInterceptor中。

全局事务处理拦截器（GlobalTransactionalInterceptor）

当代码执行到标注有@GlobalTransactional注解的类或者方法时，通过AOP创建的动态代理，会把执行流程进入GlobalTransactionalInterceptor拦截处理。


```
GlobalTransactionalInterceptor.java x MethodInterceptor.class x ServerRunner.java x ServerApplication.java x
@Override
public Object invoke(final MethodInvocation methodInvocation) throws Throwable {
    Class<?> targetClass =
        methodInvocation.getThis() != null ? AopUtils.getTargetClass(methodInvocation.getThis()) : null;
    Method specificMethod = ClassUtils.getMostSpecificMethod(methodInvocation.getMethod(), targetClass);
    if (specificMethod != null && !specificMethod.getDeclaringClass().equals(Object.class)) {
        final Method method = BridgeMethodResolver.findBridgedMethod(specificMethod);

        // 查找 GlobalTransactional 全局事务注解
        final GlobalTransactional globalTransactionalAnnotation = getAnnotation(method, targetClass, GlobalTransactional.class);
        // 查找 @GlobalLock 全局锁 注解
        final GlobalLock globalLockAnnotation = getAnnotation(method, targetClass, GlobalLock.class);
        boolean localDisable = disable || (ATOMIC_DEGRADE_CHECK.get() && degradeNum >= degradeCheckAllowTimes);
        if (!localDisable) {
            if (globalTransactionalAnnotation != null || this.aspectTransactional != null) {
                AspectTransactional transactional;
                if (globalTransactionalAnnotation != null) {
                    transactional = new AspectTransactional(globalTransactionalAnnotation.timeoutMills(),
                        globalTransactionalAnnotation.name(), globalTransactionalAnnotation.rollbackFor(),
                        globalTransactionalAnnotation.rollbackForClassName(), globalTransactionalAnnotation.noRollbackFor());
                } else {
                    transactional = this.aspectTransactional;
                }
                //执行 全局事务
                return handleGlobalTransaction(methodInvocation, transactional);
            } else if (globalLockAnnotation != null) {
                return handleGlobalLock(methodInvocation, globalLockAnnotation);
            }
        }
    }
}
```

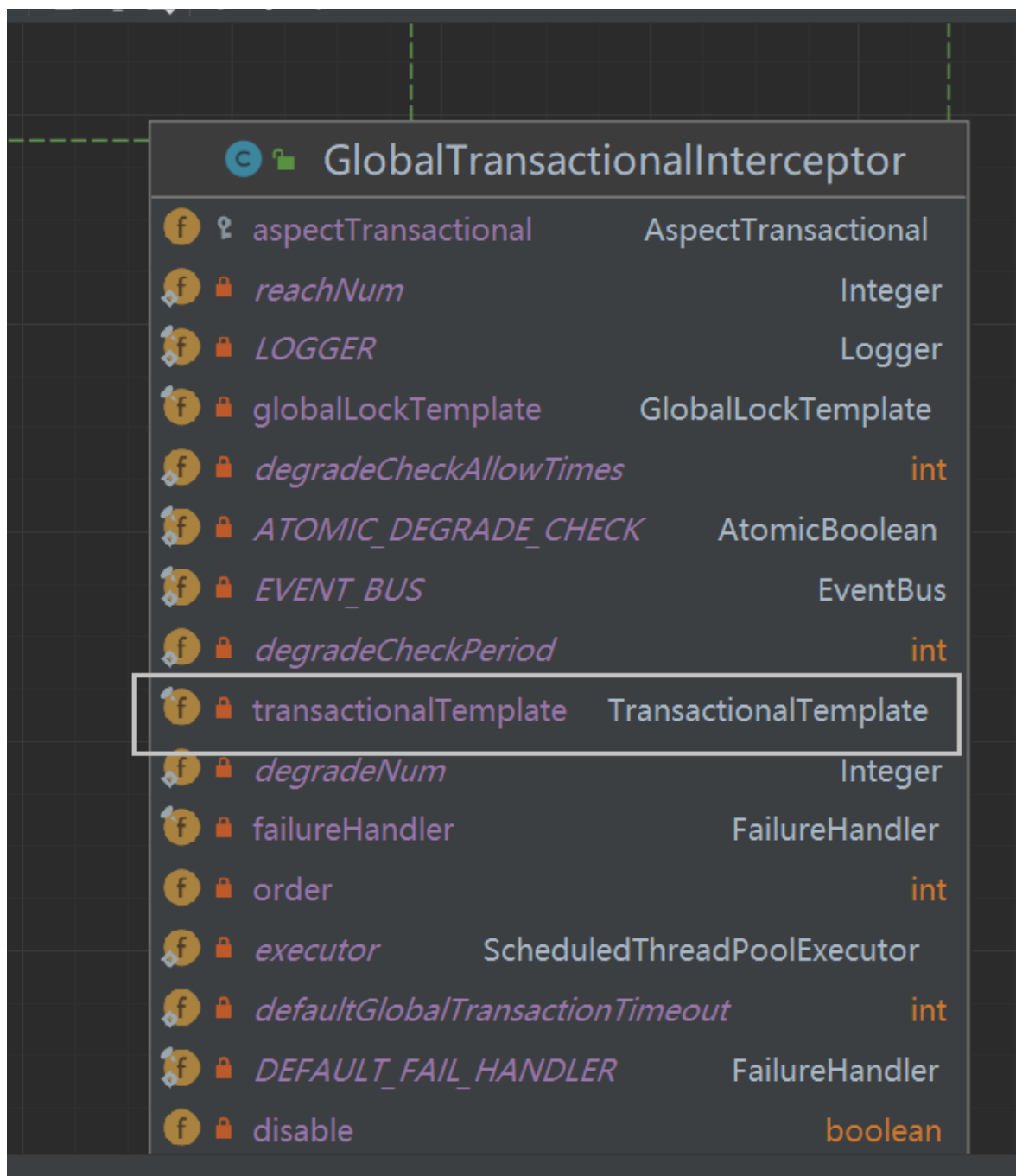
以上注释，来自尼恩的 注释版本 seata 源码。

全局事务的拦截器, 它扫描 两个注解:

- GlobalTransactional注解
- GlobalLock, 注解

如果是 GlobalTransactional 则用 transactionalTemplate 来执行真正的业务代码, 此外还从注解中拿出配置的回滚条件, 超时时间等配置, 给 transactionalTemplate 使用。

GlobalTransactionalInterceptor 有一个 transactionalTemplate模板成员。



大家看看，这个拦截器中，还有一个 globalLockTemplate，

globalLockTemplate 是在 RM 中使用的，该模板代码也很简单，就是加锁->执行->放锁，并且这里的加锁和放锁只是改变 Context 中的标志位，真正通过 RPC 进行锁确认的过程，后面会介绍。

花开两朵，各表一枝。

首先，来看看 transactionalTemplate 模板。

这里的 handleGlobalTransaction 使用 transactionalTemplate 模板封装全局事务的处理，

```

3 usages  👤 jimin.jm +9 *
Object handleGlobalTransaction(final MethodInvocation methodInvocation,
    final AspectTransactional aspectTransactional) throws Throwable {
    boolean succeed = true;
    try {
        // 使用transactionalTemplate模板封装全局事务的处理,
        👤 github-ygy +5 *
        return transactionalTemplate.execute(new TransactionalExecutor() {
            👤 jimin.jm
            @Override
            public Object execute() throws Throwable {
                return methodInvocation.proceed();
            }
        });

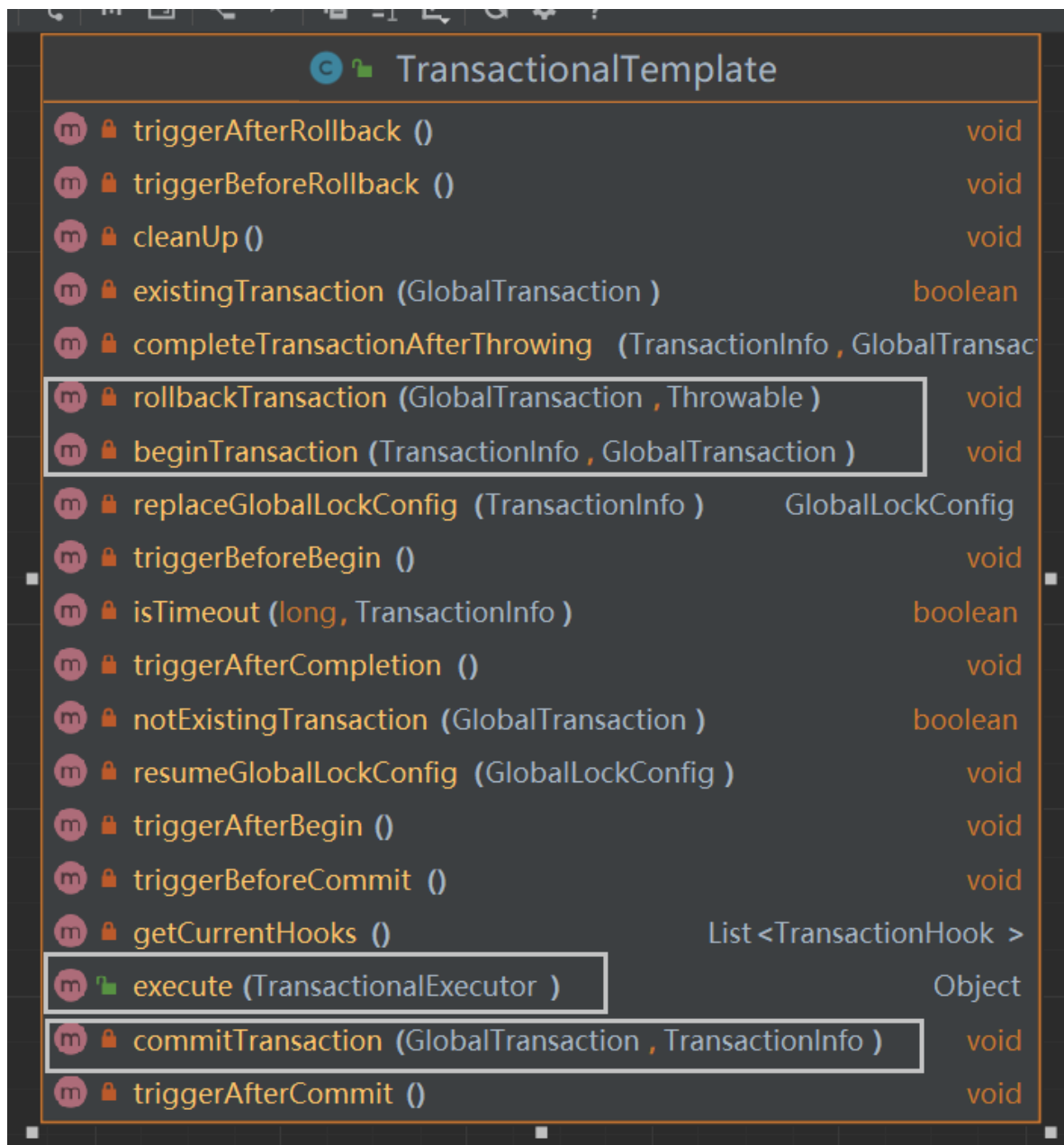
        1 usage  👤 jimin.jm +3 *
        public String name() {

            // 默认使用 方法名+参数类型名称
            String name = aspectTransactional.getName();
            if (!StringUtils.isEmpty(name)) {
                return name;
            }
            return formatMethod(methodInvocation.getMethod());
        }
        /**
         * 封装事务信息

```

TransactionalTemplate 有四个重要的模版方法

- execute 执行
- 开始事务
- 提交事务
- 回滚事务



首先，来看看Template的execute方法，这里的逻辑十分重要，主要分三步

1. 开启一个全局事务，绑定xid。
2. 执行业务代码逻辑
3. 执行全局事务提交/回滚逻辑

Template的execute方法 具体的工作流程如下:

1. 看看当前是不是已经在一个分布式事务中了, 如果是, 则复用现存的全局事务, 如果不是, 创建新的全局事务
什么时候会出现已经存在全局事务的情况呢?
假设 A 调用了 B, A 创建了全局事务 GT1,
B 也执行了上述的模板, 这时候 B 就不会创建新的全局事务, 而是使用 GT1, 这实际上是前面提到的事物的传播
2. 如果是自己创建的全局事务, 则发 RPC 开始事务, 如果不是自己创建的, 则什么都不干
3. 执行真正的业务逻辑
4. 执行回滚: 如果发生了异常, 如果自己创建全局事务, 才负责回滚, 否则就只管异常外抛

5. 执行提交: 如果没发生异常, 如果自己创建全局事务, 才负责提交, 否则就什么都不做

6. 清理工作

```
TransactionalTemplate > execute
GlobalTransactionScanner.java x GlobalTransactionalInterceptor.java x TransactionalTemplate.java x Joinpoint.class x Me
@Transactional
public Object execute(TransactionalExecutor business) throws Throwable {
    // 1. Get transactionInfo
    TransactionInfo txInfo = business.getTransactionInfo();
    if (txInfo == null) {
        throw new ShouldNeverHappenException("transactionInfo does not exist");
    }
    // 1.1 Get current transaction, if not null, the tx role is 'GlobalTransactionRole.Participant'
    // 这里会获取线程上下文中的全局事务xid,
    // 如果xid不为null, 说明此xid是通过rpc传递过来的, 那当前rm 作为此次全局事务的参与方, 创建DefaultGlobalTransaction
    // 否则, tx为null
    GlobalTransaction tx = GlobalTransactionContext.getCurrent();

    // 1.2 Handle the transaction propagation.
    // 全局事务的传播行为, 默认REQUIRED, 即继承事务
    Propagation propagation = txInfo.getPropagation();
    SuspendedResourcesHolder suspendedResourcesHolder = null;
    try {
        switch (propagation) {...}

        // 1.3 If null, create new transaction with role 'GlobalTransactionRole.Launcher'.
        if (tx == null) {
            tx = GlobalTransactionContext.createNew();
        }
    }

    //对于全局事务的发起方rm, 一个分三步
    //开启一个全局事务, 绑定xid。
    //执行业务代码逻辑
    //执行全局事务提交/回滚逻辑
    // set current tx config to holder
    GlobalLockConfig previousConfig = replaceGlobalLockConfig(txInfo);

    try {
        // 2. If the tx role is 'GlobalTransactionRole.Launcher', send the request of begin
        // else do nothing. Of course, the hooks will still be triggered.
        // 如果是全局事务的发起方 tm, 通知TC服务端注册一个全局事务
        beginTransaction(txInfo, tx);

        Object rs;
        try {
            // Do Your Business
            // 执行业务方法逻辑
            rs = business.execute();
        } catch (Throwable ex) {
            // 3. The needed business exception to rollback.
            completeTransactionAfterThrowing(txInfo, tx, ex);
            throw ex;
        }
        // 4. everything is fine, commit.
        // 全局事务提交
        commitTransaction(tx, txInfo);
    }
```

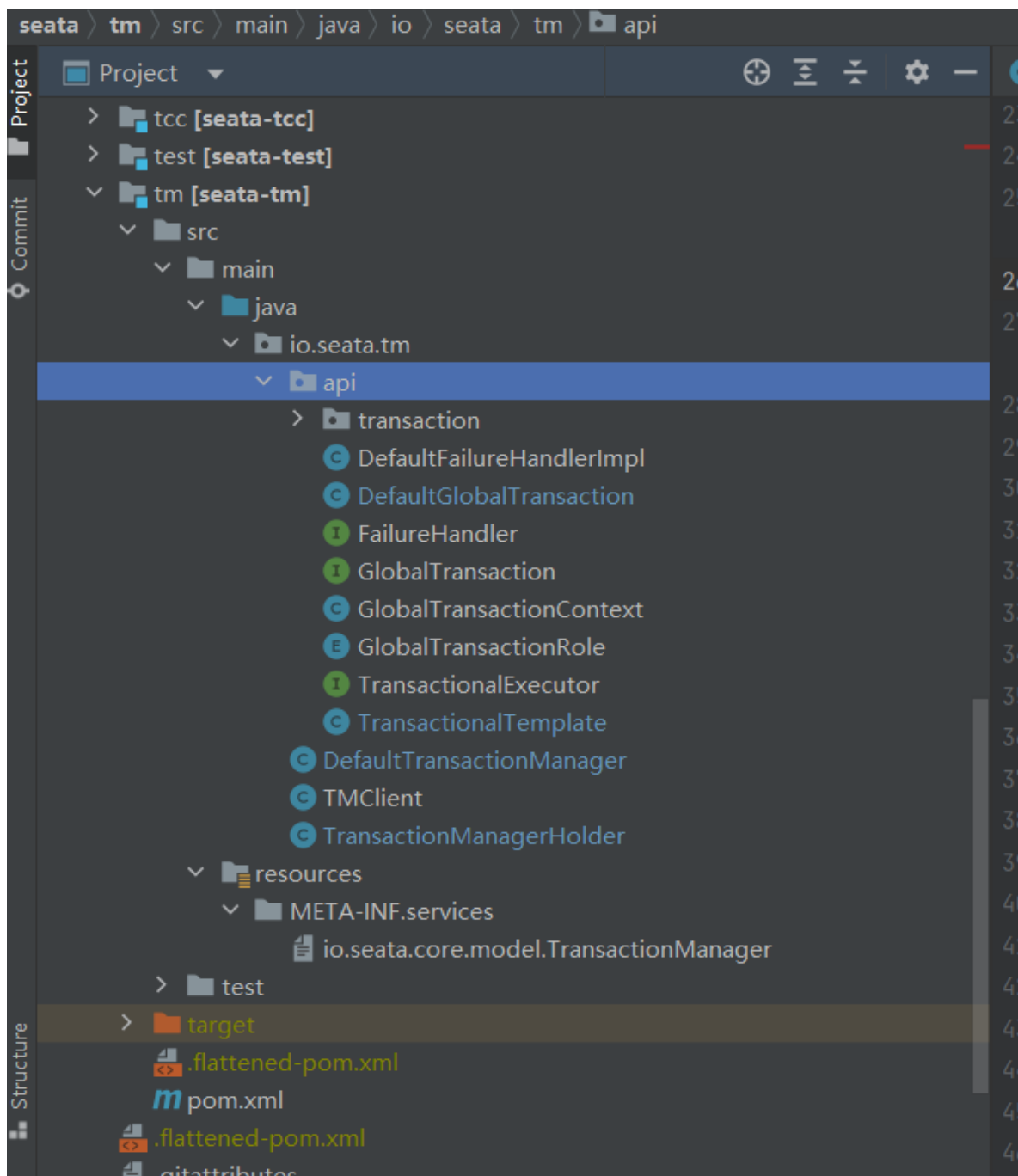
开启一个全局事务, 绑定xid。

TransactionalTemplate 的 beginTransaction

```
TransactionalTemplate.java x dependency-reduced-pom.xml x flattened-pom.xml x pom.xml (seata-all) x pom.xml (seata-build) x pom
48      throw new TransactionalExecutor.ExecutionException(tx, txe,
49          TransactionalExecutor.Code.RollbackFailure, originalException);
50  }
51
52  // 3.1 Successfully rolled back
53  throw new TransactionalExecutor.ExecutionException(tx, GlobalStatus.RollbackRetrying.equals(tx.getLocalStatus())
54      ? TransactionalExecutor.Code.RollbackRetrying : TransactionalExecutor.Code.RollbackDone, originalException);
55  }
56
57  //开始 事务
58  @ @ private void beginTransaction(TransactionInfo txInfo, GlobalTransaction tx) throws TransactionalExecutor.ExecutionExce
59      try {
60          triggerBeforeBegin();
61          tx.begin(txInfo.getTimeout(), txInfo.getName());
62          triggerAfterBegin();
63      } catch (TransactionException txe) {
64          throw new TransactionalExecutor.ExecutionException(tx, txe,
65              TransactionalExecutor.Code.BeginFailure);
66      }
67  }
68  }
```

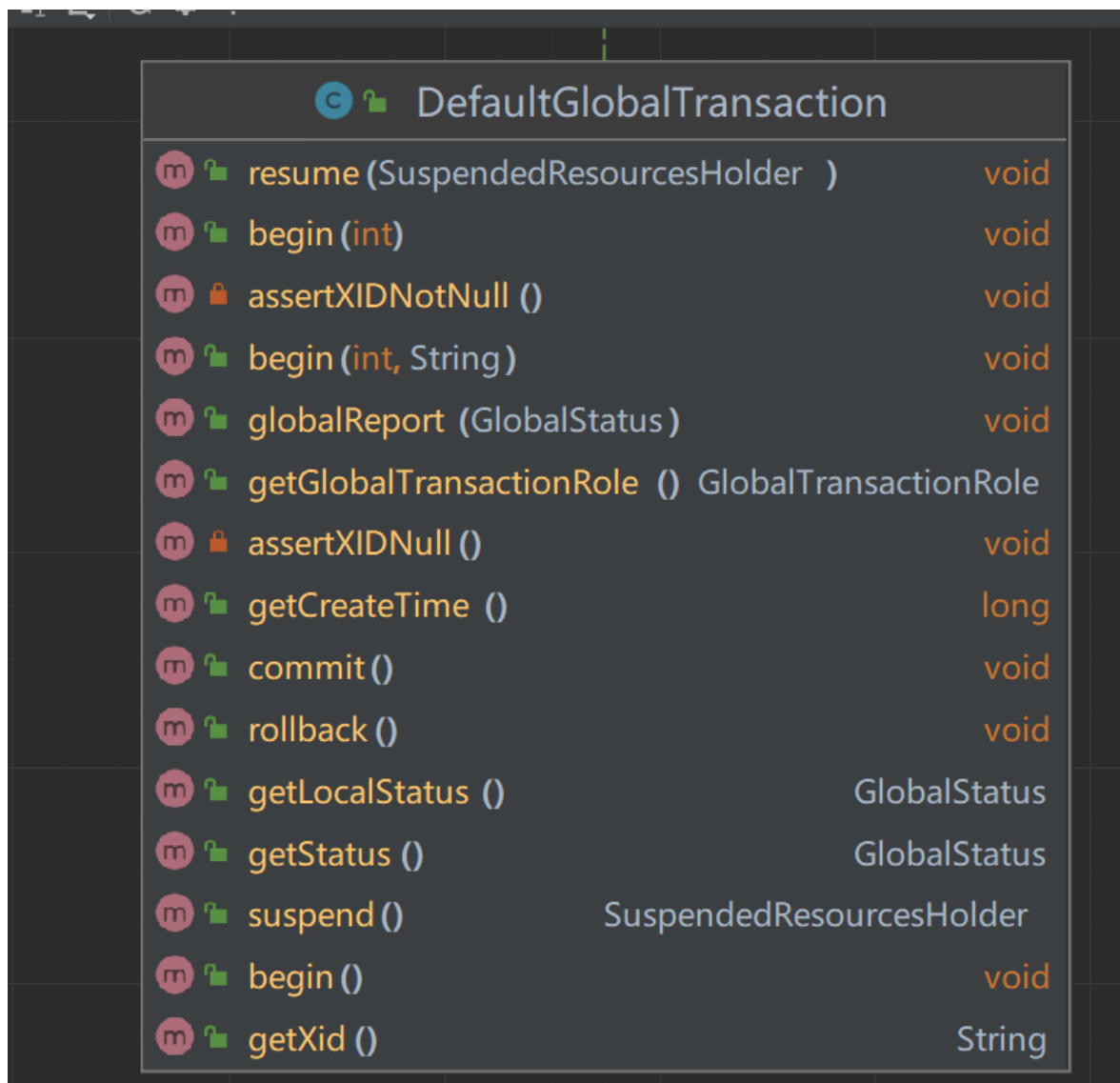
全局事务的默认实现 DefaultGlobalTransaction

TM 是一个事务管理模块, 无论是 AT 模式还是 TCC 模式, 都需要使用 TM 模块。

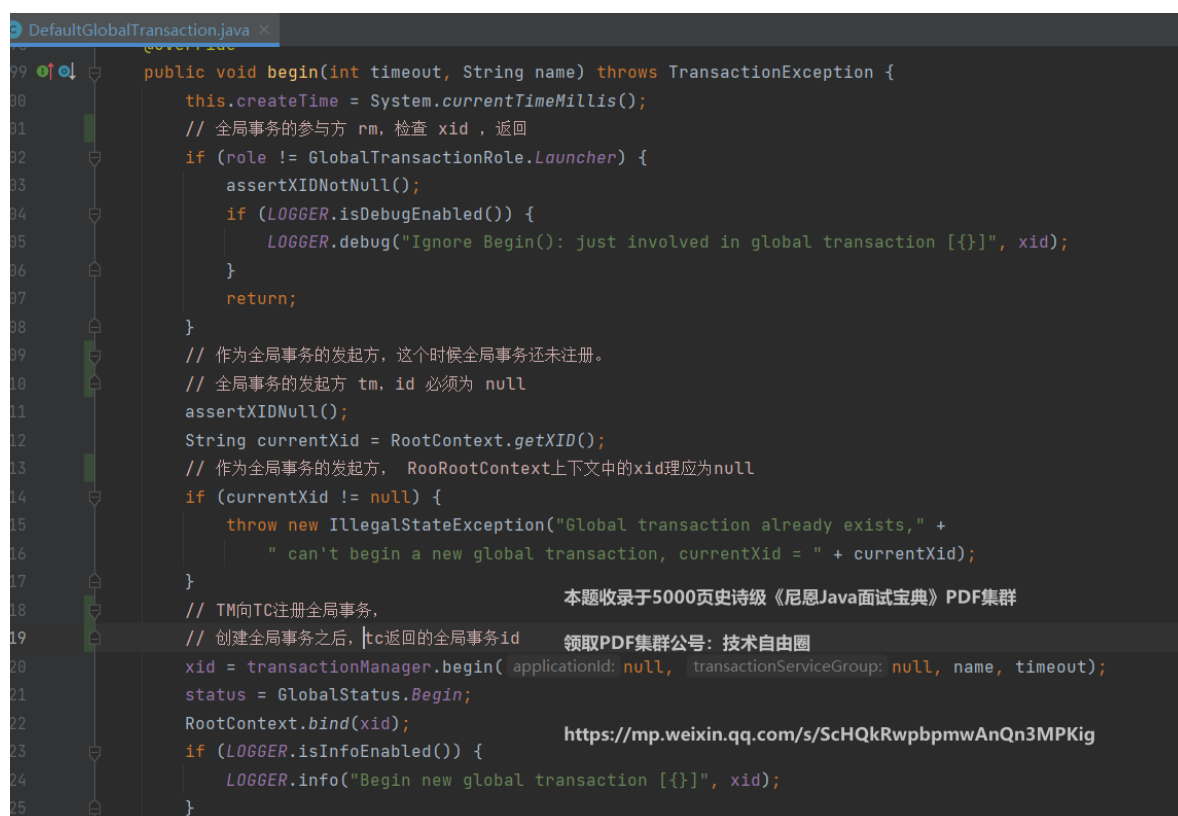


首先 TM 在启动的时候会去连接 TC Server, 然后然后通过该 TM Client 与 TC 模块进行通讯。

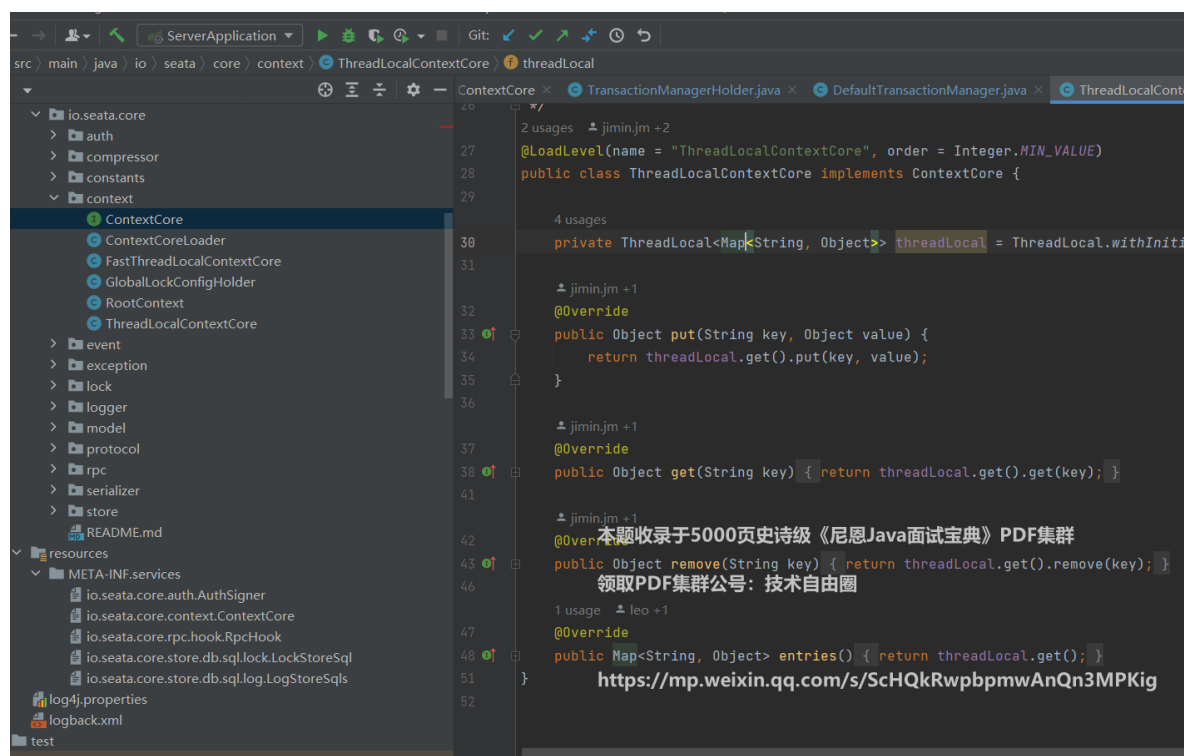
在 TM 模块中最核心的接口就是 `GlobalTransaction`, 里面包含了全局事务的创建, 提交, 回滚过程, 其实质就是向 TC 发送 RPC 请求



开始全局事务 DefaultGlobalTransaction 的begin方法

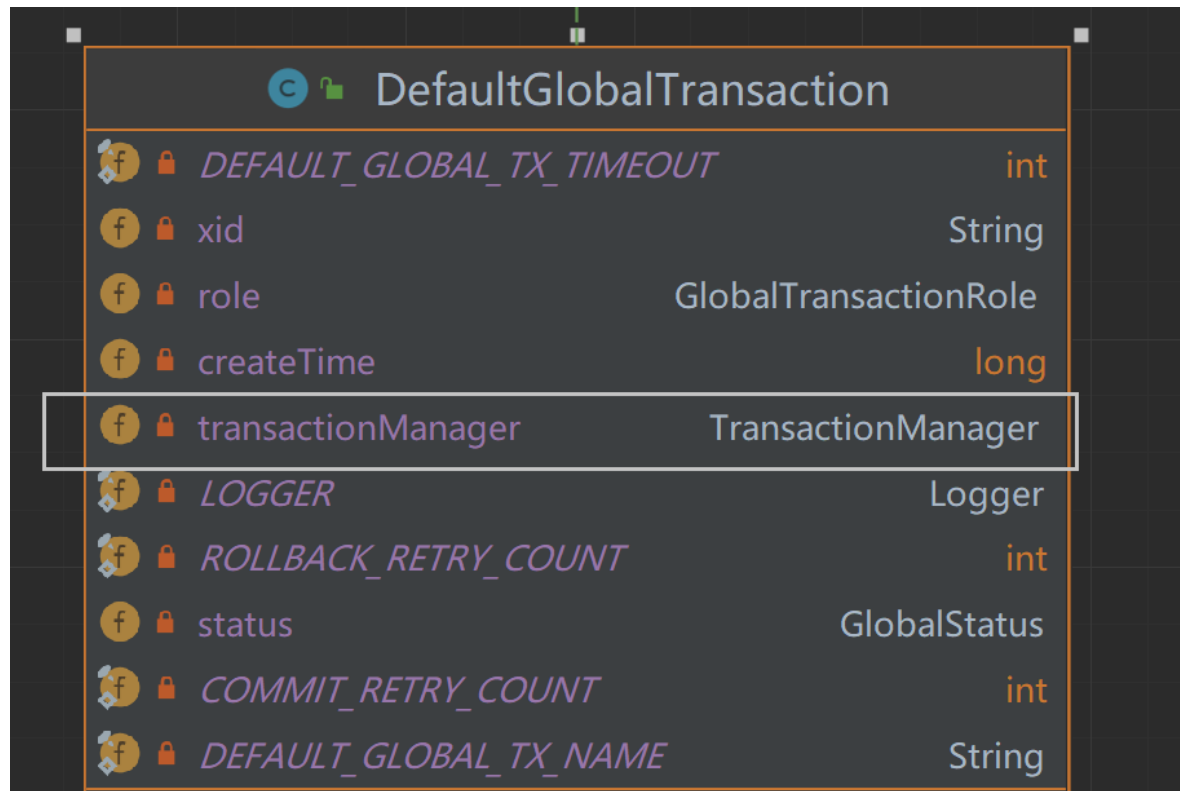


DefaultGlobalTransaction 中实际上没做什么实际的事, 它调用 transactionManager 发送消息, 然后将涉及到的全局事务 XID 保存起来, 我们看看它把数据存在什么地方了:



将 XID 存在了 ThreadLocal 中, 这样在整个 RPC 调用的上下文中都能获取到 XID。

接下来, 我们看看DefaultGlobalTransaction 的核心成员 TransactionManager, 都做了什么:



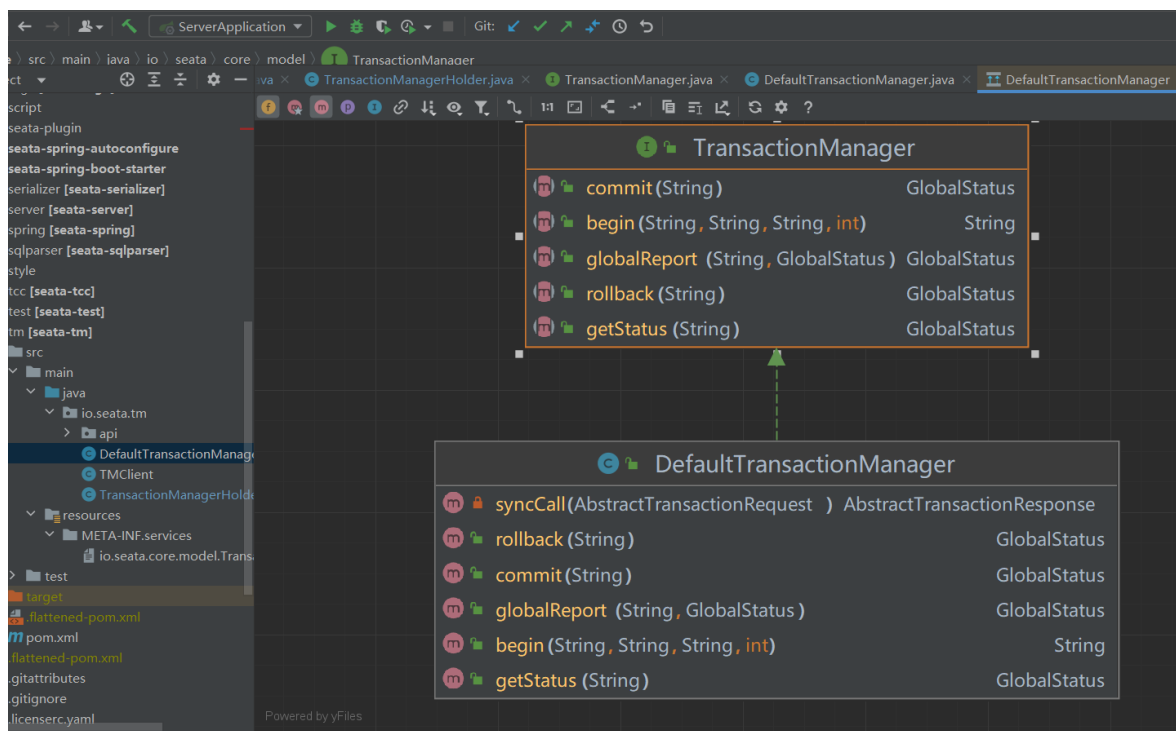
这里的TransactionManager是通过spi加载的, 一般的框架, 都会用到 java spi机制。

```
DefaultGlobalTransaction > DefaultGlobalTransaction
DefaultGlobalTransaction.java x TransactionManagerHolder.java x DefaultGlobalTransaction x GlobalTransaction

/**
 * Instantiates a new Default global transaction.
 *
 * @param xid the xid
 * @param status the status
 * @param role the role
 */
3 usages Haiqiang Shen +1
DefaultGlobalTransaction(String xid, GlobalStatus status, GlobalTransactionRole role) {
    this.transactionManager = TransactionManagerHolder.get();
    this.xid = xid;
    this.status = status;
    this.role = role;
}

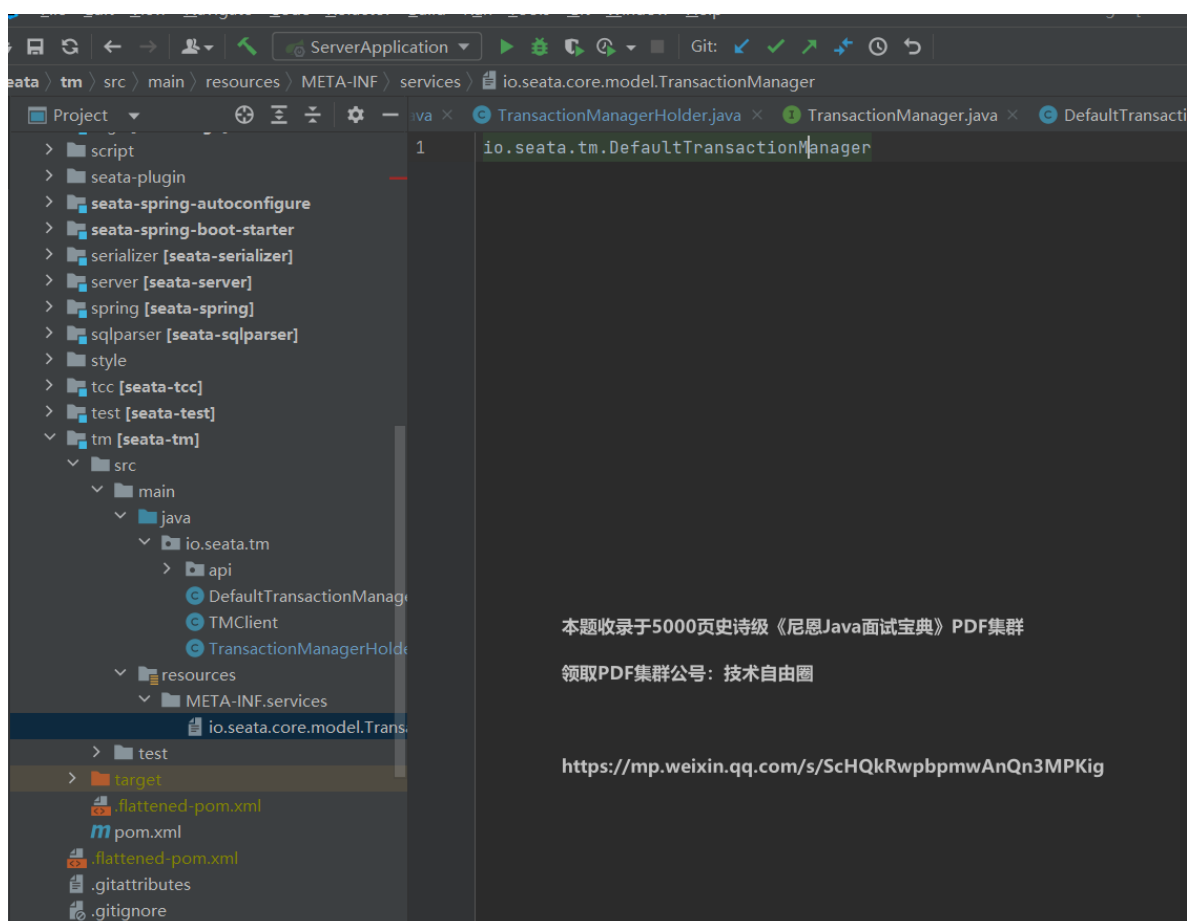
jimin.jm
```

```
DefaultGlobalTransaction.java x TransactionManagerHolder.java x DefaultGlobalTransaction x GlobalTransaction
Haiqiang Shen +1
29 public class TransactionManagerHolder {
30
31     2 usages
32     private static final Logger LOGGER = LoggerFactory.getLogger(TransactionManagerHolder.class);
33
34     3 usages Haiqiang Shen +1
35     private static class SingletonHolder {
36
37         5 usages
38         private static TransactionManager INSTANCE = null;
39
40         //这里的TransactionManager是通过spi加载的
41         // 一般的框架，都会用到 java spi机制。
42         static {
43             try {
44                 INSTANCE = EnhancedServiceLoader.load(TransactionManager.class);
45                 LOGGER.info("TransactionManager Singleton {}", INSTANCE);
46             } catch (Throwable anyEx) {
47                 LOGGER.error("Failed to load TransactionManager Singleton! ", anyEx);
48             }
49         }
50     }
51 }
```



TransactionManager默认为DefaultTransactionManager,

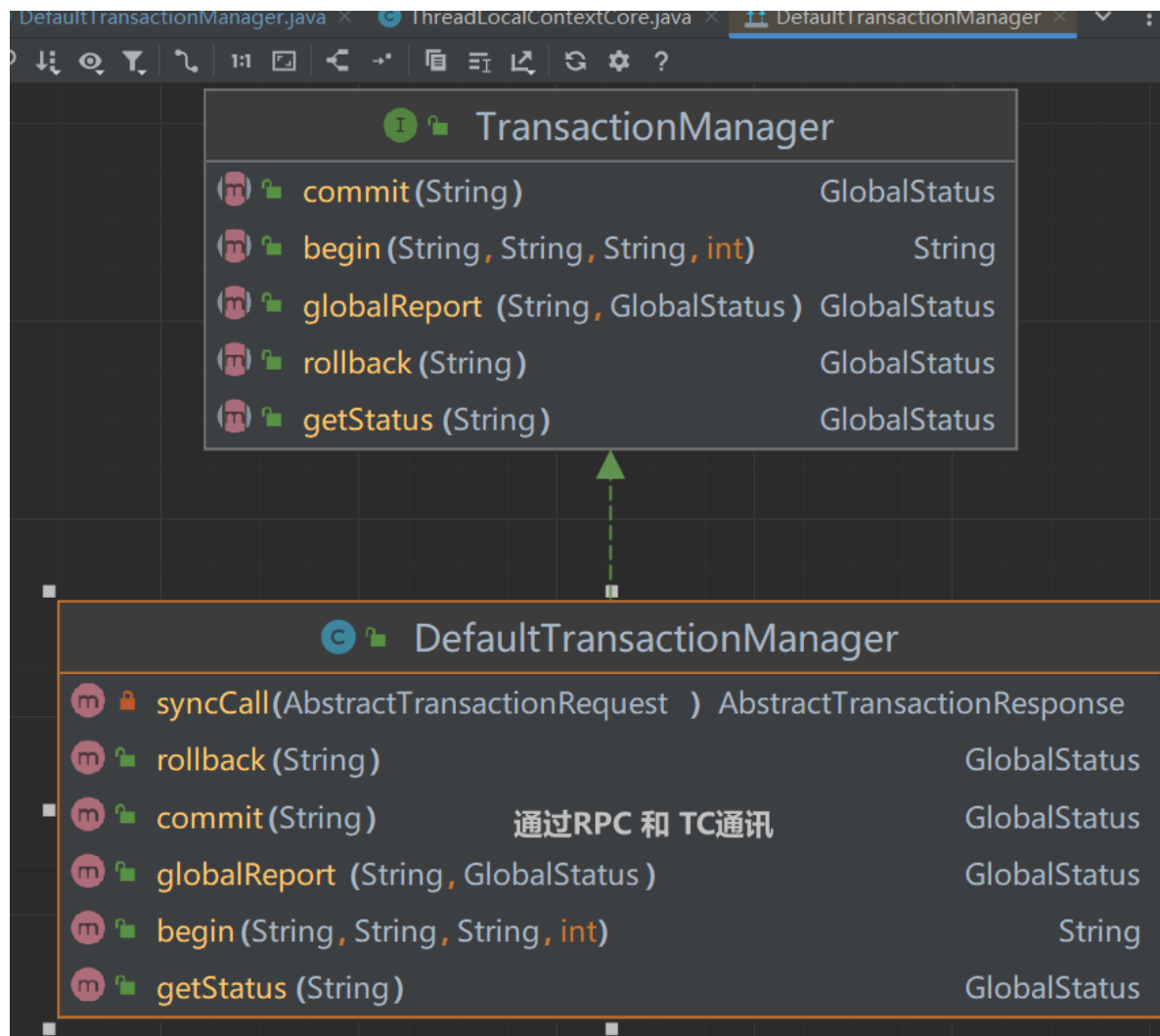
通过java spi 机制, 进行注册



DefaultTransactionManager 的几个核心方法

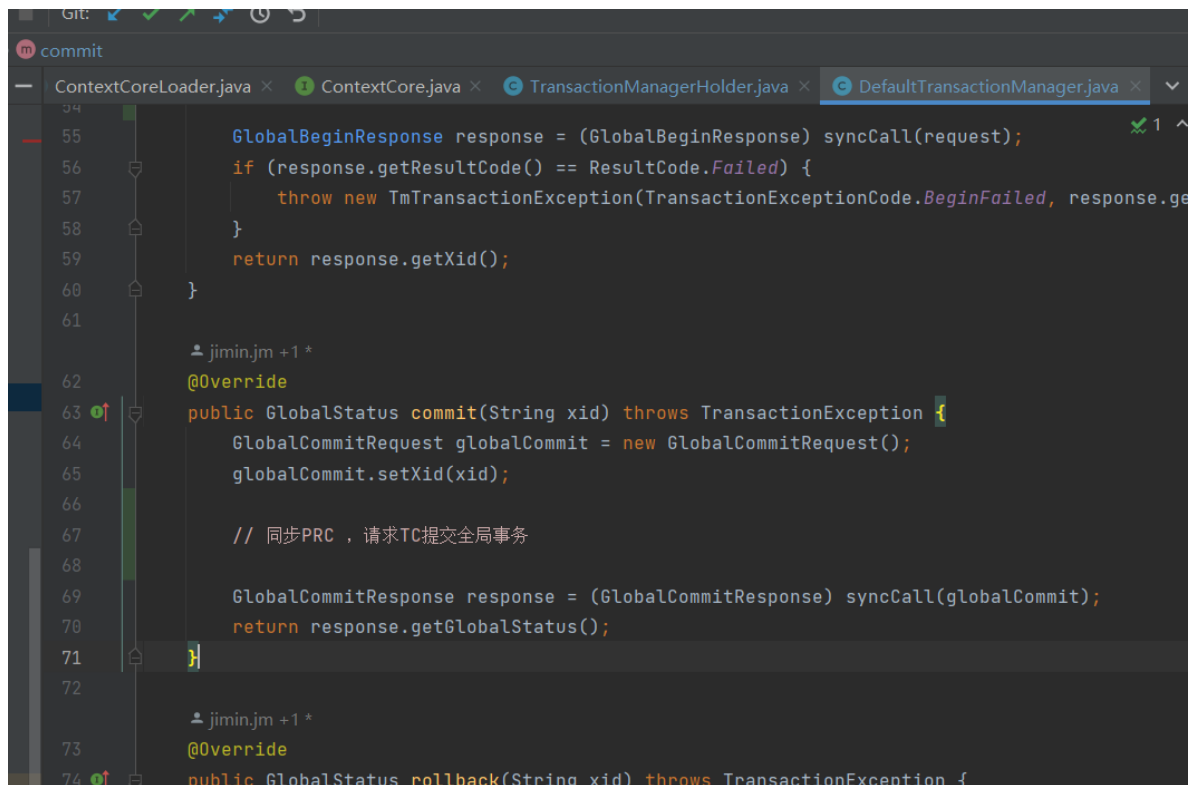
可以看到, TransactionManager 才是真正做实事的, 通过RPC 和TC进行通讯。

TransactionManager 负责RPC 消息的发送工作。



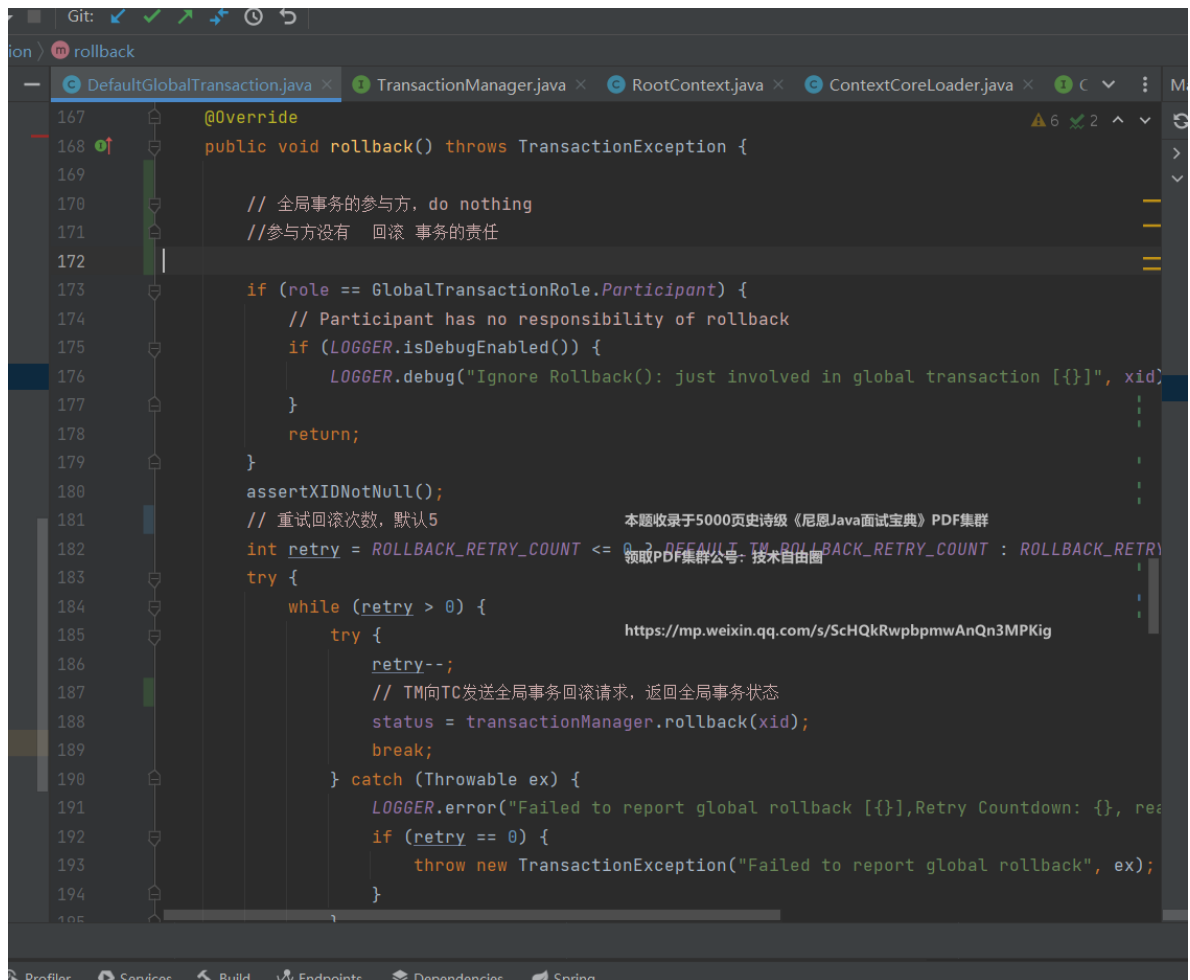
注意，是同步RPC 哦

DefaultTransactionManager 才是真正做实事的, 通过RPC 和TC进行通讯。



```
55 GlobalBeginResponse response = (GlobalBeginResponse) syncCall(request);
56 if (response.getResultCode() == ResultCode.Failed) {
57     throw new TmTransactionException(TransactionExceptionCode.BeginFailed, response.ge
58 }
59 return response.getXid();
60 }
61
62 jimin.jm +1 *
63 @Override
64 public GlobalStatus commit(String xid) throws TransactionException {
65     GlobalCommitRequest globalCommit = new GlobalCommitRequest();
66     globalCommit.setXid(xid);
67
68     // 同步PRC，请求TC提交全局事务
69
70     GlobalCommitResponse response = (GlobalCommitResponse) syncCall(globalCommit);
71     return response.getGlobalStatus();
72 }
73
74 jimin.jm +1 *
75 @Override
76 public GlobalStatus rollback(String xid) throws TransactionException {
```

全局事务回滚



```
167 @Override
168 public void rollback() throws TransactionException {
169
170     // 全局事务的参与方, do nothing
171     //参与方没有 回滚 事务的责任
172
173     if (role == GlobalTransactionRole.Participant) {
174         // Participant has no responsibility of rollback
175         if (LOGGER.isDebugEnabled()) {
176             LOGGER.debug("Ignore Rollback(): just involved in global transaction [{}]", xid);
177         }
178         return;
179     }
180     assertXIDNotNull();
181     // 重试回滚次数, 默认5
182     int retry = ROLLBACK_RETRY_COUNT <= 0 ? DEFAULT_TM_ROLLBACK_RETRY_COUNT : ROLLBACK_RETRY
183     try {
184         while (retry > 0) {
185             try {
186                 retry--;
187                 // TM向TC发送全局事务回滚请求, 返回全局事务状态
188                 status = transactionManager.rollback(xid);
189                 break;
190             } catch (Throwable ex) {
191                 LOGGER.error("Failed to report global rollback [{}], Retry Countdown: {}", retry, ex);
192                 if (retry == 0) {
193                     throw new TransactionException("Failed to report global rollback", ex);
194                 }
195             }
196         }
197     }
```

DefaultTransactionManager 才是真正做实事的, 通过RPC 和TC进行通讯。

一阶段分支事务Try处理流程（TccActionInterceptor）

再看看业务代码中tcc分支事务的流程，即TccActionInterceptor拦截处理

```
cc) TccActionInterceptor invoke
DefaultGlobalTransaction.java x TccActionInterceptor.java x ActionInterceptorHandler.java x TransactionManager.java x ContextCore.java
2 usages zhangthen
public TccActionInterceptor(RemotingDesc remotingDesc) { this.remotingDesc = remotingDesc; }

zhangthen +4 *
@Override
public Object invoke(final MethodInvocation invocation) throws Throwable {

    // 不处于全局事务||禁用||saga模式, 不执行事务的 aop 切面逻辑, 直接执行业务逻辑

    if (!RootContext.inGlobalTransaction() || disable || RootContext.inSagaBranch()) {
        //not in transaction, or this interceptor is disabled
        return invocation.proceed();
    }
    Method method = getActionInterfaceMethod(invocation);

    // 获取方法上的TwoPhaseBusinessAction 两阶段事务注解
    TwoPhaseBusinessAction businessAction = method.getAnnotation(TwoPhaseBusinessAction.class);
    //try method
    // 有businessAction注解, 执行TCC 事务 切面逻辑
    if (businessAction != null) {
        //save the xid
        String xid = RootContext.getXID();
        //save the previous branchType
        BranchType previousBranchType = RootContext.getBranchType();
        //if not TCC, bind TCC branchType
        if (BranchType.TCC != previousBranchType) {
            RootContext.bindBranchType(BranchType.TCC);
        }
    }
}
```

```
DefaultGlobalTransaction.java x TccActionInterceptor.java x ActionInterceptorHandler.java x TransactionManager.java x ContextCore.java x
92 RootContext.bindBranchType(branchType.TCC);
93 }
94 try {
95     //Handler the TCC Aspect, and return the business result
96     return actionInterceptorHandler.proceed(method, invocation.getArguments(), xid, businessAction,
97         invocation::proceed);
98 } finally {
99     //if not TCC, unbind branchType
100     if (BranchType.TCC != previousBranchType) {
101         RootContext.unbindBranchType();
102     }
103     //MDC remove branchId
104     MDC.remove(RootContext.MDC_KEY_BRANCH_ID);
105 }
106 }
107
108 //not TCC try method
109 return invocation.proceed();
110 }
111 }
```

分支注册&&上下文参数处理


```

1 usage 王良 +4 *
public Object proceed(Method method, Object[] arguments, String xid, TwoPhaseBusinessAction businessAction,
                      Callback<Object> targetCallback) throws Throwable {
    //Get action context from arguments, or create a new one and then reset to arguments
    //BusinessActionContext用来保存全局事务2阶段需要的数据,
    // 可以自己指定传入一个BusinessActionContext, 没有也会创建一个
    //如果方法入参中含有BusinessActionContext类型的参数, 则直接获取方法传入的BusinessActionContext,
    // 否则, new BusinessActionContext()
    BusinessActionContext actionContext = getOrCreateActionContextAndResetToArguments(method.getParameterTypes(), a

    //Set the xid
    actionContext.setXid(xid);
    //Set the action name
    String actionName = businessAction.name();
    actionContext.setActionName(actionName);
    //Set the delay report
    actionContext.setDelayReport(businessAction.isDelayReport());

    //Creating Branch Record
    //组装@BusinessActionContextParameter参数注解中的内容到BusinessActionContext中
    //向TC注册tcc分支事务, BusinessActionContext以json格式传到TC服务端, 获取TC返回的分支id
    String branchId = doTccActionLogStore(method, arguments, businessAction, actionContext);
    actionContext.setBranchId(branchId);
    //MDC put branchId
    MDC.put(RootContext.MDC_KEY_BRANCH_ID, branchId);
}

```

```

// save the previous action context
// BusinessActionContextUtil这个工具, 用于获取/设置分支上下文中的业务参数。
BusinessActionContext previousActionContext = BusinessActionContextUtil.getContext();
try {
    //share actionContext implicitly
    //这里设置BusinessActionContext到线程上下文后 (分支内有效)
    // 我们可以在业务代码中调用BusinessActionContextUtil#addContext添加参数, 供tcc2阶段使用
    BusinessActionContextUtil.setContext(actionContext);
    //判断是否开启Tcc 幂等、空回滚、防悬挂控制。
    // 建议开启, 否则需要自己业务代码做响应的处理
    if (businessAction.useTCCFence()) {
        try {
            // Use TCC Fence, and return the business result
            //TCC 幂等、空回滚、防悬挂控制
            return TCCFenceHandler.prepareFence(xid, Long.valueOf(branchId), actionName, targetCallback);
        } catch (SkipCallbackWrapperException | UndeclaredThrowableException e) {
            Throwable originException = e.getCause();
            if (originException instanceof FrameworkException) {
                LOGGER.error("[{}] prepare TCC fence error: {}", xid, originException.getMessage());
            }
            throw originException;
        }
    } else {
        //Execute business, and return the business result
        //执行业务逻辑处理
        return targetCallback.execute();
    }
} finally {
}

```

在分支注册的时候, 会将分支上下文中供TCC2阶段使用的业务参数序列化成json字符串一同发送到TC服务端, 执行完业务方法后, 还需要再请求一次TC更新业务参数, 也是整个json字符串传递过去的。

而在我们真实的业务场景中, 要传递给2阶段执行的参数可能是**非常多的**, 一旦事务分支较多, 这会给**网络IO**带来很大的开销。

所以其实这里我们是可以优化的, 上下文中的**参数信息**完全可以保存到客户端的数据库中 (存储在tcc_fence_log中就挺合适的), 一次数据库操作就可以完整的保存下来。

而发送给TC的数据只需要包含基本的分支信息就可以了, 也不用再多请求一次TC来更新参数内容。

TCC模式通过客户端的tcc_fence_log表来完成幂等、空回滚、防悬挂的控制。

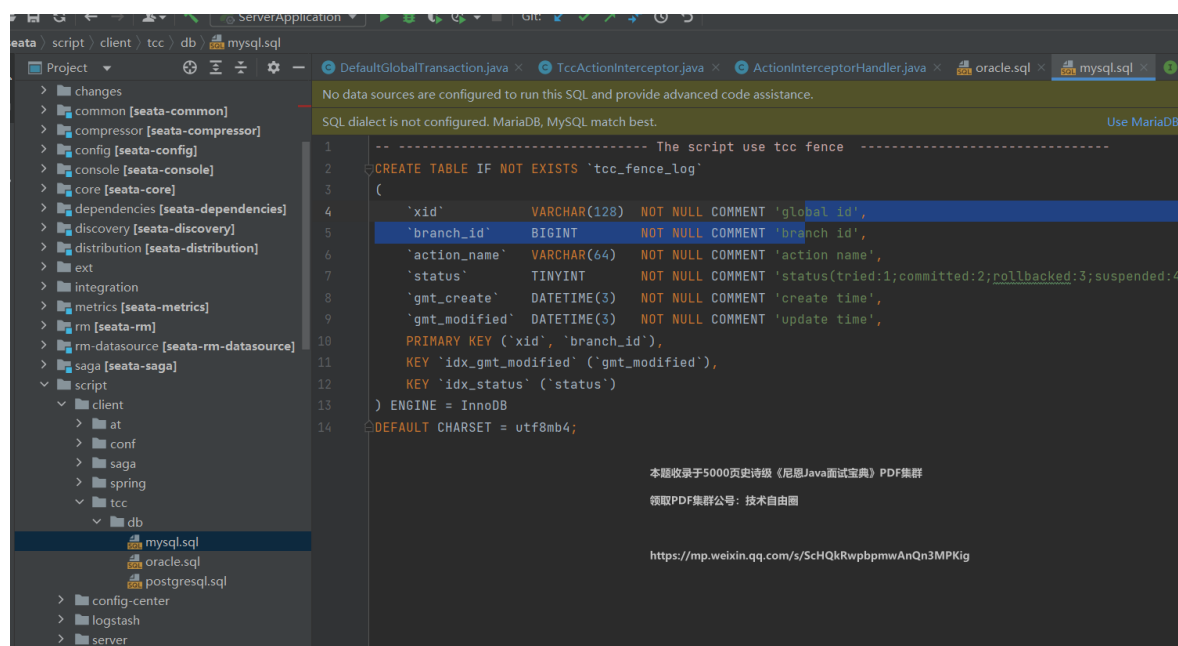
下面就具体来看下TCCFenceHandler的逻辑

tcc_fence_log

tcc_fence（tcc屏障）的前提：

- 1.开发人员需要在使用TCC模式的时候，在@TwoPhaseBusinessAction注解中设置useTCCFence=true才能使用Seata提供的功能处理幂等性、资源悬挂、空回滚等问题；
- 2.Seata解决这三个问题的方式是建立一张记录表，通过记录一阶段、二阶段处理的状态让各处理逻辑感知事务所处的状态，从而达到接口幂等性、避免资源悬挂、空回滚等问题；

tcc_fence_log表结构



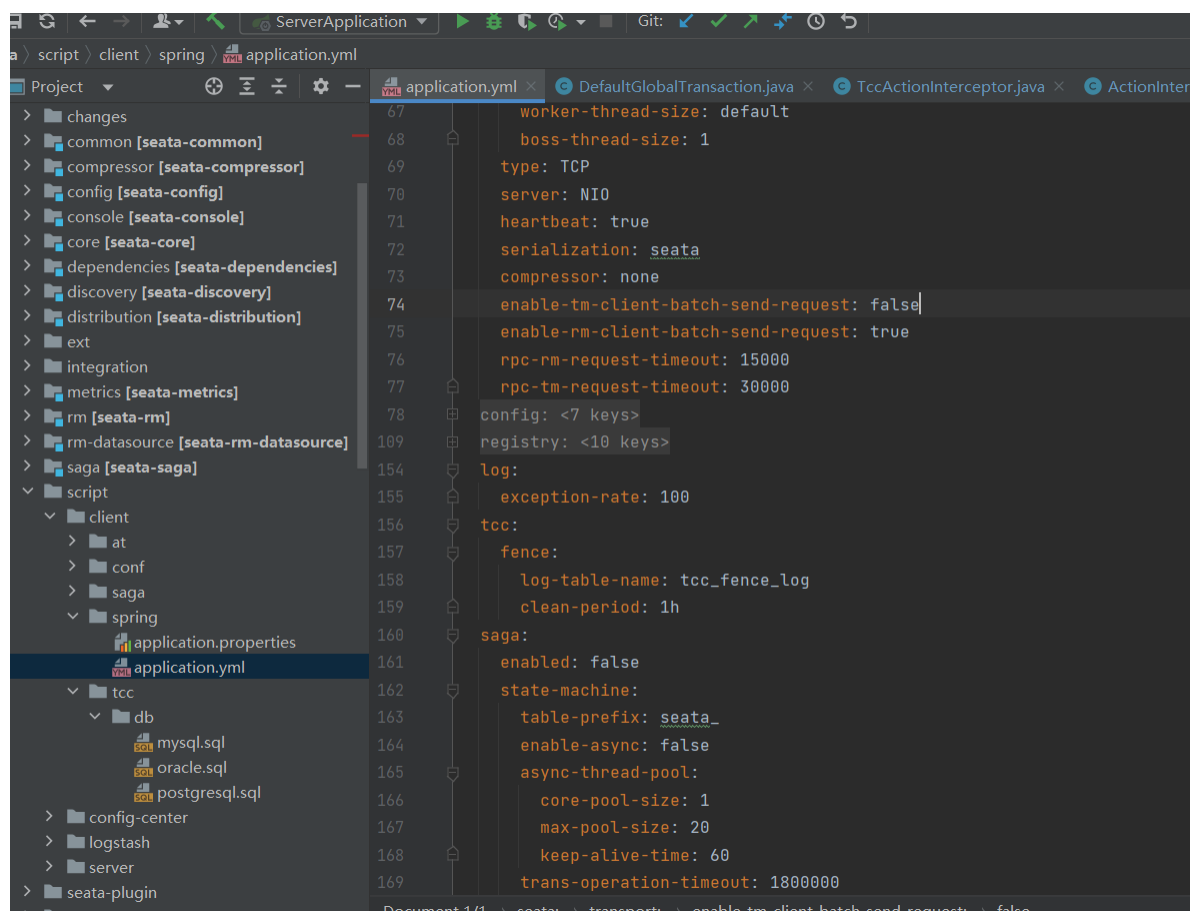
```
1  -- The script use tcc fence
2  CREATE TABLE IF NOT EXISTS `tcc_fence_log`
3  (
4      `xid` VARCHAR(128) NOT NULL COMMENT 'global id',
5      `branch_id` BIGINT NOT NULL COMMENT 'branch id',
6      `action_name` VARCHAR(64) NOT NULL COMMENT 'action name',
7      `status` TINYINT NOT NULL COMMENT 'status(tried:1;committed:2;rollbacked:3;suspended:4)',
8      `gmt_create` DATETIME(3) NOT NULL COMMENT 'create time',
9      `gmt_modified` DATETIME(3) NOT NULL COMMENT 'update time',
10     PRIMARY KEY (`xid`, `branch_id`),
11     KEY `idx_gmt_modified` (`gmt_modified`),
12     KEY `idx_status` (`status`)
13 ) ENGINE = InnoDB
14 DEFAULT CHARSET = utf8mb4;
```

可以看到这个表的核心字段就三个：

- xid-全局事务id,
- branch_id-分支事务id,
- action_name-分支名称,

主键为 (xid, branch_id)

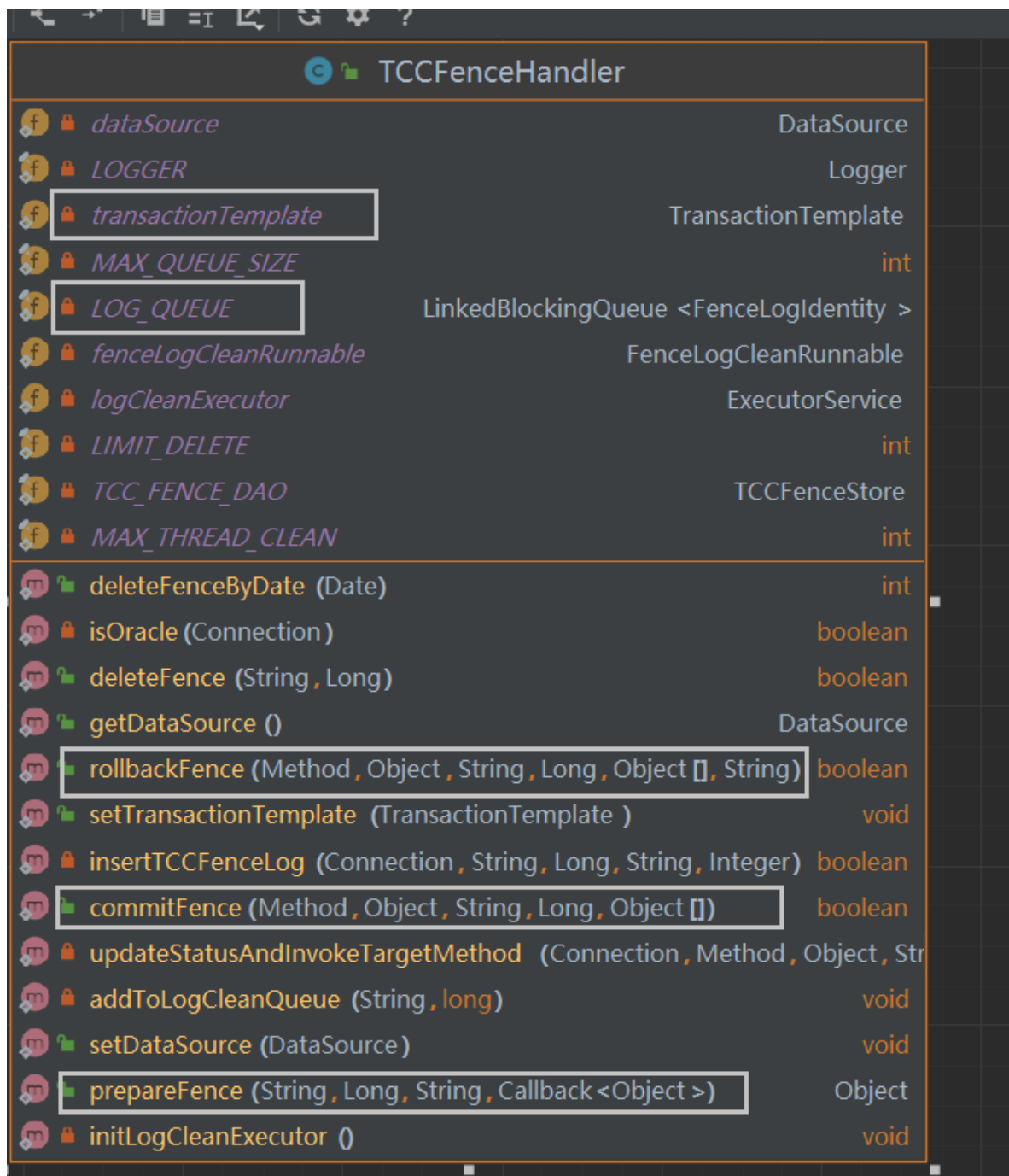
除了DB需要创建表之外，配置文件中，需要添加 tcc的配置，参考如下：



TCC 屏障处理器 TCCFenceHandler

TCCFenceHandler 如何实现幂等性

- 一阶段 TCCFenceHandler.prepareFence() :
- 二阶段 TCCFenceHandler.commitFence() :
- 二阶段 回滚 屏障 TCCFenceHandler.rollbackFence() :



一阶段 准备屏障 `TCCFenceHandler.prepareFence()`:

```

TCCFenceHandler.java
public static Object prepareFence(String xid, Long branchId, String actionName, Callback<Object> targetCallback) {

    // 这里使用spring的TransactionTemplate手工开启事务，以确保业务代码和tcc_fence_log表insert处于同一事务中

    return transactionTemplate.execute(status -> {
        try {
            Connection conn = DataSourceUtils.getConnection(dataSource);
            //insert记录到tcc_fence_log表中
            boolean result = insertTCCFenceLog(conn, xid, branchId, actionName, TCCFenceConstant.STATUS_TRIED);
            LOGGER.info("TCC fence prepare result: {}. xid: {}, branchId: {}", result, xid, branchId);
            if (result) {
                //执行业务代码处理
                return targetCallback.execute();
            } else {
                throw new TCCFenceException(String.format("Insert tcc fence record error, prepare fence failed. xid= {}, branchId= {}", xid, branchId),
                    FrameworkErrorCode.InsertRecordError);
            }
        } catch (TCCFenceException e) {
            //这里如果抛出duplikey异常, 说明:
            // 1 已经执行
            // 2 或者正在执行2阶段的cancel。
            if (e.getErrcode() == FrameworkErrorCode.DuplicateKeyException) {
                LOGGER.error("Branch transaction has already rollbacked before,prepare fence failed. xid= {},branchId= {}", xid, branchId);
                //定时清理掉这种记录
                addToLogCleanQueue(xid, branchId);
            }
            status.setRollbackOnly();
        }
    });
}

```

对于try阶段而已，只需要检查第二阶段是否已经执行完成，如果已完成，则不再执行。

这里通过往tcc_fence_log表中insert一条记录来保证此机制，

如果出现duplikey异常，说明2阶段cancel比try先执行，2阶段cancel为了支持空回滚，也会往此表中insert一条记录，而2阶段commit则是需要检查表中是否有记录，并更新状态。

二阶段 提交屏障 TCCFenceHandler.commitFence():

```

TCCFenceHandler.java
public static boolean commitFence(Method commitMethod, Object targetTCCBean, String xid, Long branchId, Object[] args) {

    // 模版模式
    // 这里使用spring的TransactionTemplate手工开启事务，以确保业务代码和tcc_fence_log表update处于同一事务中
    return transactionTemplate.execute(status -> {
        try {
            Connection conn = DataSourceUtils.getConnection(dataSource);
            // 查询一阶段提交的记录
            TCCFenceDO tccFenceDO = TCC_FENCE_DAO.queryTCCFenceDO(conn, xid, branchId);
            // 如果没查到一阶段的记录, 说明还不能调用提交逻辑
            if (tccFenceDO == null) {
                throw new TCCFenceException(String.format("TCC fence record not exists, commit fence method failed. xid= %s, branchId= %s", xid, branchId),
                    FrameworkErrorCode.RecordNotExists);
            }

            // 如果记录的状态为`TCCFenceConstant.STATUS_COMMITTED`, 说明已经提交过了, 直接返回true; 这就满足了幂等性;
            if (TCCFenceConstant.STATUS_COMMITTED == tccFenceDO.getStatus()) {
                LOGGER.info("Branch transaction has already committed before. idempotency rejected. xid: {}, branchId: {}, status: {}", xid, branchId, tccFenceDO.getStatus());
                return true;
            }

            // 如果状态是`TCCFenceConstant.STATUS_ROLLBACKED`, 说明已经调用了回滚逻辑, 那就不能执行提交;
            // 如果是`TCCFenceConstant.STATUS_SUSPENDED`, 那就说明该分布式事务暂时挂起了, 还不能执行提交;
            if (TCCFenceConstant.STATUS_ROLLBACKED == tccFenceDO.getStatus() || TCCFenceConstant.STATUS_SUSPENDED == tccFenceDO.getStatus()) {
                if (LOGGER.isWarnEnabled()) {
                    LOGGER.warn("Branch transaction status is unexpected. xid: {}, branchId: {}, status: {}", xid, branchId, tccFenceDO.getStatus());
                }
                return false;
            }

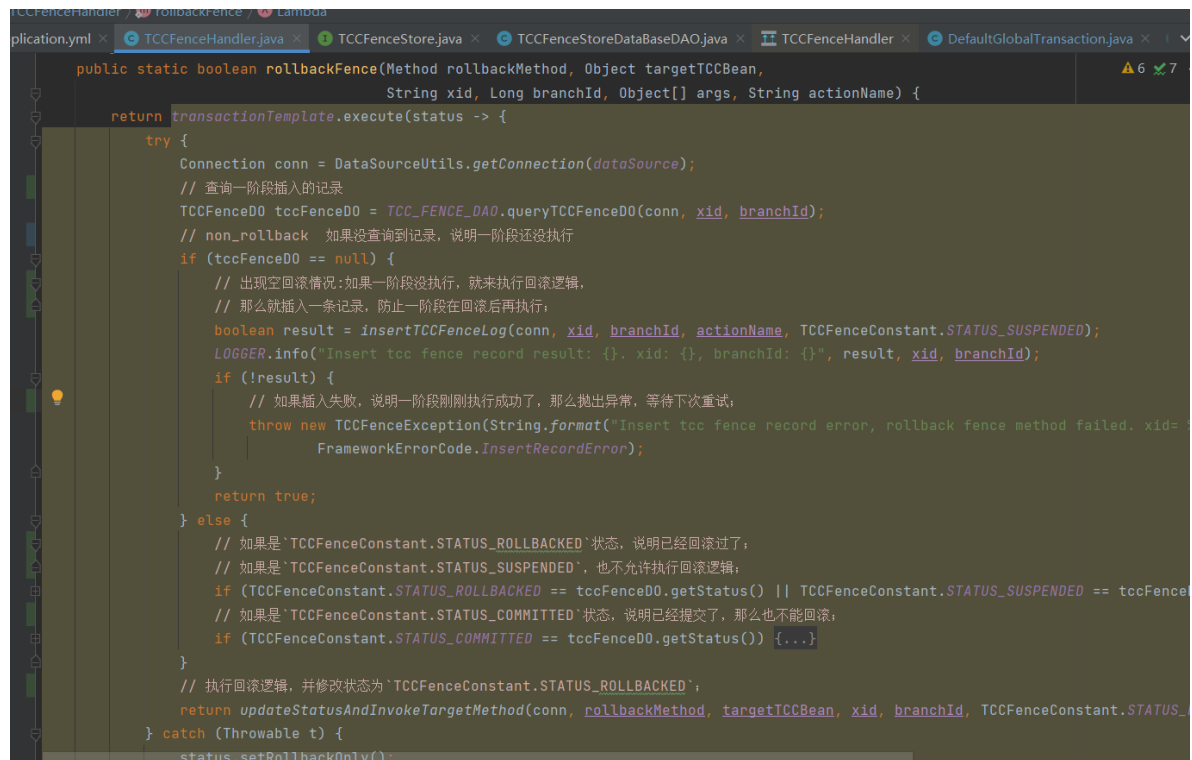
            // 这里就可以执行提交逻辑, 并修改状态为`TCCFenceConstant.STATUS_COMMITTED`;
            return updateStatusAndInvokeTargetMethod(conn, commitMethod, targetTCCBean, xid, branchId, TCCFenceConstant.STATUS_COMMITTED);
        } catch (Throwable t) {
            // 这里使用spring的TransactionTemplate手工开启事务，以确保业务代码和tcc_fence_log表update处于同一事务中
            LOGGER.error("Commit fence method failed. xid= {}, branchId= {}", xid, branchId, t);
            status.setRollbackOnly();
        }
    });
}

```

1.在提交前会先查询出一阶段插入的记录，如果没有查询到记录，说明一阶段还没有完成，那么就不能执行提交逻辑；

- 2.如果查询到记录状态是 `TCCFenceConstant.STATUS_COMMITTED`，说明已经执行过提交逻辑了，那么就不会再次执行提交逻辑，也就保证了提交的幂等性；
- 3.如果是其他状态，比如已经回滚了，或者被挂起了，那么就不能执行提交逻辑；
- 4.所有不可提交的状态都排除后，开始正式执行提交逻辑，并更新状态为 `TCCFenceConstant.STATUS_COMMITTED`，表示已经提交；

二阶段 回滚 屏障 `TCCFenceHandler.rollbackFence()`:



```
public static boolean rollbackFence(Method rollbackMethod, Object targetTCCBean,
                                   String xid, Long branchId, Object[] args, String actionName) {
    return transactionTemplate.execute(status -> {
        try {
            Connection conn = DataSourceUtils.getConnection(dataSource);
            // 查询一阶段插入的记录
            TCCFenceDO tccFenceDO = TCC_FENCE_DAO.queryTCCFenceDO(conn, xid, branchId);
            // non_rollback 如果没查询到记录，说明一阶段还没执行
            if (tccFenceDO == null) {
                // 出现空回滚情况:如果一阶段没执行，就来执行回滚逻辑。
                // 那么就插入一条记录，防止一阶段在回滚后再执行；
                boolean result = insertTCCFenceLog(conn, xid, branchId, actionName, TCCFenceConstant.STATUS_SUSPENDED);
                LOGGER.info("Insert tcc fence record result: {}, xid: {}, branchId: {}", result, xid, branchId);
                if (!result) {
                    // 如果插入失败，说明一阶段刚刚执行成功了，那么抛出异常，等待下次重试；
                    throw new TCCFenceException(String.format("Insert tcc fence record error, rollback fence method failed. xid=%s",
                                                                FrameworkErrorCode.InsertRecordError));
                }
                return true;
            } else {
                // 如果是 'TCCFenceConstant.STATUS_ROLLBACKED' 状态，说明已经回滚过了；
                // 如果是 'TCCFenceConstant.STATUS_SUSPENDED'，也不允许执行回滚逻辑；
                if (TCCFenceConstant.STATUS_ROLLBACKED == tccFenceDO.getStatus() || TCCFenceConstant.STATUS_SUSPENDED == tccFenceDO.getStatus()) {
                    // 如果是 'TCCFenceConstant.STATUS_COMMITTED' 状态，说明已经提交了，那么也不能回滚；
                    if (TCCFenceConstant.STATUS_COMMITTED == tccFenceDO.getStatus()) {...}
                }
                // 执行回滚逻辑，并修改状态为 'TCCFenceConstant.STATUS_ROLLBACKED'；
                return updateStatusAndInvokeTargetMethod(conn, rollbackMethod, targetTCCBean, xid, branchId, TCCFenceConstant.STATUS_ROLLBACKED);
            }
        } catch (Throwable t) {
            status.setRollbackOnly();
        }
    });
}
```

- 1.回滚前会查询一阶段保存的记录，如果没有查询到一阶段保存的记录，那么说明出现了空回滚的情况；
- 2.出现空回滚情况就立马插入一条记录，防止一阶段插入记录成功，也就预防了资源悬挂；
- 3.如果空回滚能够插入记录，那么一阶段必然执行失败，不会出现资源悬挂问题，也就说明什么都没做，回滚成功；如果空回滚插入记录失败，说明一阶段刚刚执行成功，那么我们直接抛异常，回滚失败，等待下次重试；
- 4.如果能够查到一阶段的记录，并且状态是 `TCCFenceConstant.STATUS_ROLLBACKED` 或 `TCCFenceConstant.STATUS_SUSPENDED`，说明已经执行过回滚动作了，那么就不能再执行了，直接返回，满足幂等性；
- 5.如果一阶段记录的状态是 `TCCFenceConstant.STATUS_COMMITTED`，说明已经执行过提交逻辑了，那么就不能再回滚了；
- 6.如果满足执行回滚的条件，那么调用回滚逻辑，并修改状态为 `TCCFenceConstant.STATUS_ROLLBACKED`，完成回滚动作；

二阶段处理流程

首先要明确的是二阶段无论是commit还是rollback都是由TC服务端发起的请求，对于client端来说，一阶段事务结束，业务流程就已经结束了。

2阶段处理已经处于处理tc请求的异步线程中了，而非业务线程中。

客户端会初始化一个RMClient的RPC客户端，且同时会添加一个监听器RmMessageListener，监听器将监听来自seata的server发送的RPC消息。

```
src/main/java/io/seata/rm/RMClient
DefaultRMHandler.java x AbstractRMHandler.java x RMClient.java x AbstractNettyRemotingClient.java x AbstractException
changes
common [seata-common]
compressor [seata-compressor]
config [seata-config]
console [seata-console]
core [seata-core]
dependencies [seata-dependencies]
discovery [seata-discovery]
distribution [seata-distribution]
ext
integration
metrics [seata-metrics]
rm [seata-rm]
src
  main
    java
      io.seata.rm
        AbstractResourceManage
        AbstractRMHandler
        DefaultResourceManager
        DefaultRMHandler
        RMClient
target
  flattened-pom.xml
  pom.xml
rm-datasource [seata-rm-datasource]
saga [seata-saga]
script
  client
  at
  conf
  saga

package io.seata.rm;

import io.seata.core.rpc.netty.RmNettyRemotingClient;

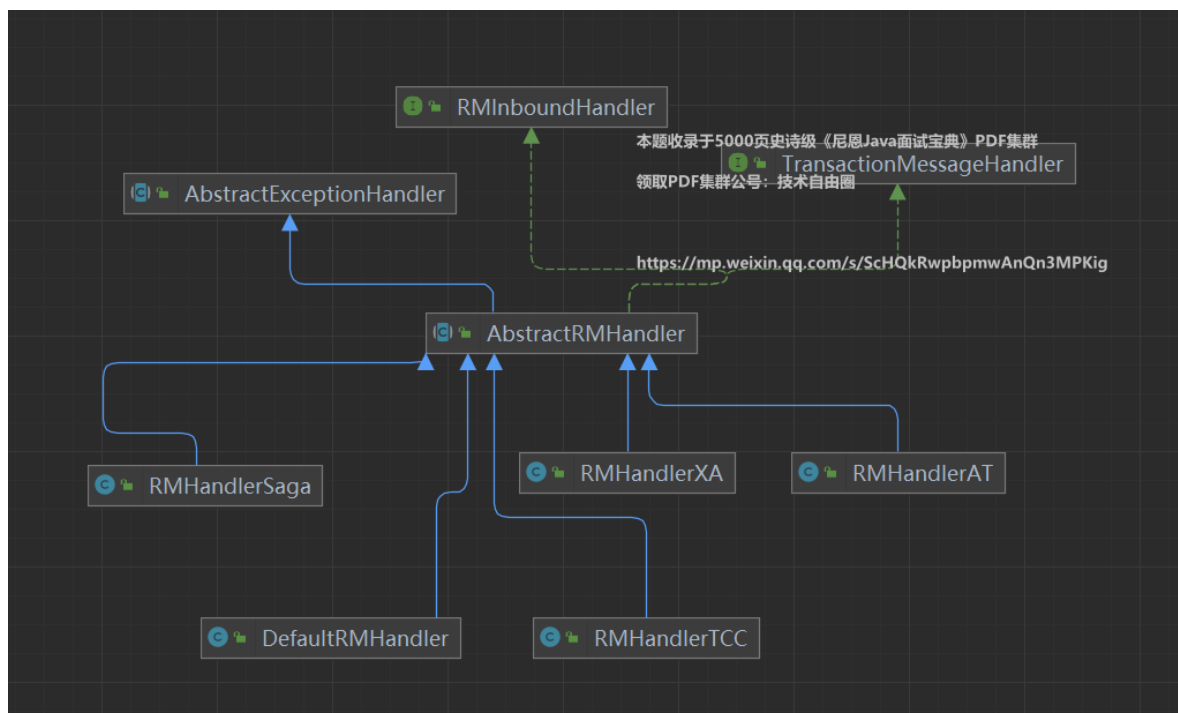
/**
 * The Rm client Initiator.
 *
 * @author slievrlly
 */
public class RMClient {

    /**
     * Init.
     *
     * @param applicationId the application id
     * @param transactionServiceGroup the transaction service group
     */
    public static void init(String applicationId, String transactionServiceGroup) {

        RmNettyRemotingClient rmNettyRemotingClient = RmNettyRemotingClient.getInstance(applicationId,
        rmNettyRemotingClient.setResourceManager(DefaultResourceManager.get());
        //添加一个监听器RmMessageListener，监听器将监听来自seata的server发送的RPC消息
        rmNettyRemotingClient.setTransactionMessageHandler(DefaultRMHandler.get());
        rmNettyRemotingClient.init();
    }
}
```

分支提交/回滚请求是由RM来处理的，委托给DefaultRMHandler处理。这里的DefaultRMHandler.get()将返回一个DefaultRMHandler的单例对象，

分别对应不同的事务模式，三种处理器和DefaultRMHandler一样都继承了AbstractRMHandler。



而AbstractRMHandler包含了两个核心方法，doBranchCommit和doBranchRollback，分别用于二阶段分支事务的提交和分支事务的回滚。

```
DefaultRMHandler > handle
application.yml x TCCFenceHandler.java x DefaultRMHandler.java x TCCFenceStore.java x TCCFenceStoreDataBaseDAO.java x
44 protected DefaultRMHandler() { initRMHandlers(); }
47
48 1 usage  zhangthen *
49 protected void initRMHandlers() {
50     //通过spi加载所有的RMHandler放到map中, key值为BranchType, value为具体handler对象
51
52     List<AbstractRMHandler> allRMHandlers = EnhancedServiceLoader.loadAll(AbstractRMHandler.class);
53     if (CollectionUtils.isNotEmpty(allRMHandlers)) {
54         for (AbstractRMHandler rmHandler : allRMHandlers) {
55             allRMHandlersMap.put(rmHandler.getBranchType(), rmHandler);
56         }
57     }
58 }
59 //处理分支commit请求, 对应BranchCommitRequest
60 //返回分支处理结果
61 zhangthen +1 *
62 @Override
63 public BranchCommitResponse handle(BranchCommitRequest request) {
64     MDC.put(RootContext.MDC_KEY_XID, request.getXid());
65     MDC.put(RootContext.MDC_KEY_BRANCH_ID, String.valueOf(request.getBranchId()));
66     //调用具体的handler处理请求
67     return getRMHandler(request.getBranchType()).handle(request);
68 }
69
70 zhangthen +1
71 @Override
72 public BranchRollbackResponse handle(BranchRollbackRequest request) {
73     MDC.put(RootContext.MDC_KEY_XID, request.getXid());
74     MDC.put(RootContext.MDC_KEY_BRANCH_ID, String.valueOf(request.getBranchId()));
75     return getRMHandler(request.getBranchType()).handle(request);
76 }
```

不同的分支类型对应不同的RmHandler。

DefaultRMHandler 实现了 TransactionMessageHandler，并且组合了三种handler

- 1) RMHandlerAT
- 2) RMHandlerTCC
- 3) RMHandlerSaga

getRMHandler 方法获取对应的 handler


```
DefaultRMHandler.java x AbstractRMHandler.java x RMClient.java x AbstractNettyRemotingClient.java x AbstractExceptionHandle
MDC.put(RootContext.MDC_KEY_BRANCH_ID, String.valueOf(request.getBranchId()));
return getRMHandler(request.getBranchType()).handle(request);
}

github-ygy
@Override
public void handle(UndoLogDeleteRequest request) { getRMHandler(request.getBranchType()).handle(r

// 根据分支类型获取对应的Rmhandler
3 usages zhangthen
protected AbstractRMHandler getRMHandler(BranchType branchType) {
    return allRMHandlersMap.get(branchType);
}

4 usages zhangthen
@Override
```

```
Application
AbstractRMHandler > handle
DefaultRMHandler.java x AbstractRMHandler.java x RMClient.java x AbstractNettyRemotingClient.java x AbstractExceptionHandle
45 private static final Logger LOGGER = LoggerFactory.getLogger(AbstractRMHandler.class);
46
1 override zhangthen +2 *
47 @Override
48 public BranchCommitResponse handle(BranchCommitRequest request) {
49     //分支提交 响应
50
51     BranchCommitResponse response = new BranchCommitResponse();
52     //模板方法 执行, 正常/异常情况响应赋值
53     zhangthen +2 *
54     exceptionHandleTemplate(new AbstractCallback<BranchCommitRequest, BranchCommitResponse>() {
55         zhangthen +1 *
56         @Override
57         public void execute(BranchCommitRequest request, BranchCommitResponse response)
58             throws TransactionException {
59             //分支提交
60             doBranchCommit(request, response);
61         }
62     }, request, response);
63     return response;
64 }

AbstractRMHandler > handle
DefaultRMHandler.java x AbstractRMHandler.java x RMClient.java x AbstractNettyRemotingClient.java x AbstractExceptionHandle
1 override zhangthen +2 *
@Override
public BranchRollbackResponse handle(BranchRollbackRequest request) {
    //分支回滚响应
    BranchRollbackResponse response = new BranchRollbackResponse();
    //模板方法执行, 正常/异常情况响应赋值
    zhangthen +2 *
    exceptionHandleTemplate(new AbstractCallback<BranchRollbackRequest, BranchRollbackResponse>() {
        zhangthen +1 *
        @Override
        public void execute(BranchRollbackRequest request, BranchRollbackResponse response)
            throws TransactionException {
            //分支回滚
            doBranchRollback(request, response);
        }
    }, request, response);
    return response;
}
```


Commit处理流程

AbstractRMHandler 的 doBranchCommit 方法如下

```
AbstractRMHandler / handle / anonymous AbstractCallback / execute
DefaultRMHandler.java x AbstractRMHandler.java x RMClient.java x AbstractNettyRemotingClient.java x AbstractExceptionHandler
92 * @param request the request
93 * @param response the response
94 * @throws TransactionException the transaction exception
95 */
1 usage jimin +1 *
96 @
97 protected void doBranchCommit(BranchCommitRequest request, BranchCommitResponse response)
98     throws TransactionException {
99     String xid = request.getXid();
100     Long branchId = request.getBranchId();
101     String resourceId = request.getResourceId();
102     //一阶段初始化好的上下文参数内容
103     String applicationData = request.getApplicationData();
104     if (LOGGER.isInfoEnabled()) {
105         LOGGER.info("Branch committing: " + xid + " " + branchId + " " + resourceId + " " + applicationData);
106     }
107     //委托给ResourceManager
108     BranchStatus status = getResourceManager().branchCommit(request.getBranchType(), xid, branchId, resourceId,
109         applicationData);
110     response.setXid(xid);
111     response.setBranchId(branchId);
112     response.setBranchStatus(status);
113     if (LOGGER.isInfoEnabled()) {
114         LOGGER.info("Branch commit result: " + status);
115     }
116 }
117 }
```

这个方法，委托给了 ResourceManager

ResourceManager也是通过spi加载的，委托给DefaultResourceManager，最终定位到TCCResourceManager

```
DefaultRMHandler.java x AbstractRMHandler.java x ResourceManagerInbound.java x RMClient.java x AbstractNettyRemotingClient.java
36 * @param applicationData Application data bind with this branch.
37 * @return Status of the branch after committing.
38 * @throws TransactionException Any exception that fails this will be wrapped with TransactionException
39 * out.
40 */
41 6 implementations zhangthen
42 BranchStatus branchCommit(BranchType branchType, String xid, long branchId, String resourceId, String applicationData)
43 /**
44 * Rollback a branch transaction.
45 */
46 Choose Implementation of branchCommit (6 Methods Found)
47 DataSourceManager (io.seata.rm.datasource) seata-rm-datasource
48 DefaultResourceManager (io.seata.rm) seata-rm
49 MockDataSourceManager in LocalTransactionWithGlobalLockDataSourceBasicTest seata-test
50 ResourceManagerXA (io.seata.rm.datasource.xa) seata-rm-datasource
51 SagaResourceManager (io.seata.saga.rm) seata-saga-rm
52 TCCResourceManager (io.seata.rm.tcc) seata-tcc
53 * out.
54 */
55 6 implementations zhangthen
56 BranchStatus branchRollback(BranchType branchType, String xid, long branchId, String resourceId, String applicationData)
```

```
TCCResourceManager \ branchCommit
DefaultRMHandler.java x AbstractRMHandler.java x ResourceManagerInbound.java x TCCResourceManager.java x RMClient.java
zhangthen +7 *
@Override
public BranchStatus branchCommit(BranchType branchType, String xid, long branchId, String resourceId,
                                String applicationData) throws TransactionException {
    //通过resource名称获取到具体的TCCResource对象,
    // 在客户端GlobalTransactionScanner扫描的时候,
    // 将各分支方法的@TwoPhaseBusinessAction解析成TCCResource注册到map中
    TCCResource tccResource = (TCCResource)tccResourceCache.get(resourceId);
    if (tccResource == null) {
        throw new ShouldNeverHappenException(String.format("TCC resource is not exist, resourceId: %s",
                                                                resourceId));
    }
    //目标对象
    Object targetTCCBean = tccResource.getTargetBean();
    //指定的commit方法
    Method commitMethod = tccResource.getCommitMethod();
    if (targetTCCBean == null || commitMethod == null) {
        throw new ShouldNeverHappenException(String.format("TCC resource is not available, resourceId: %s",
                                                                resourceId));
    }
    try {
        //BusinessActionContext
        //将TC server端返回的applicationData、xid等组装成BusinessActionContext对象, 供2阶段commit方法使用
        BusinessActionContext businessActionContext = getBusinessActionContext(xid, branchId, resourceId,
                                                                                applicationData);
        //支持在try方法上的@TwoPhaseBusinessAction中commitArgsClasses + commit方法参数注解@BusinessActionContextParameter
        //以@BusinessActionContextParameter的value属性作为key, 从businessActionContext中获取到json string, 再反序列化
        Object[] args = this.getTwoPhaseCommitArgs(tccResource, businessActionContext);
        Object ret;
        boolean result;
        // add idempotent and anti hanging
        // 幂等、空回滚、防悬挂处理
        // 幂等、空回滚、防悬挂处理
        if (Boolean.TRUE.equals(businessActionContext.getActionContext(Constants.USE_TCC_FENCE))) {
            try {
                result = TCCFenceHandler.commitFence(commitMethod, targetTCCBean, xid, branchId, args);
            } catch (SkipCallbackWrapperException | UndeclaredThrowableException e) {
                throw e.getCause();
            }
        } else {
            //直接执行commit方法
            ret = commitMethod.invoke(targetTCCBean, args);
            if (ret != null) {
                if (ret instanceof TwoPhaseResult) {
                    result = ((TwoPhaseResult)ret).isSuccess();
                } else {
                    result = (boolean)ret;
                }
            } else {
                result = true;
            }
        }
        LOGGER.info("TCC resource commit result : {}, xid: {}, branchId: {}, resourceId: {}", result, xid, branchId, resourceId);
        return result ? BranchStatus.PhaseTwo_Committed : BranchStatus.PhaseTwo_CommitFailed_Retryable;
    } catch (Throwable t) {
        String msg = String.format("commit TCC resource error, resourceId: %s, xid: %s.", resourceId, xid);
        LOGGER.error(msg, t);
    }
}
```

这里，调用了二阶段 提交屏障 TCCFenceHandler.commitFence()，代码参见前面，不做赘述

Rollback处理流程

AbstractRMHandler 的 doBranchRollback方法如下

```
actRMHandler > doBranchCommit
defaultRMHandler.java x AbstractRMHandler.java x ResourceManagerInbound.java x TCCResourceManager.java x RMClient.java
* @param response the response
* @throws TransactionException the transaction exception
*/
1 usage jimin +1*
@
protected void doBranchRollback(BranchRollbackRequest request, BranchRollbackResponse response)
    throws TransactionException {
    String xid = request.getXid();
    long branchId = request.getBranchId();
    String resourceId = request.getResourceId();
    //一阶段初始化好的上下文参数内容
    String applicationData = request.getApplicationData();
    if (LOGGER.isInfoEnabled()) {
        LOGGER.info("Branch Rollbacking: " + xid + " " + branchId + " " + resourceId);
    }
    //委托给ResourceManager
    BranchStatus status = getResourceManager().branchRollback(request.getBranchType(), xid, branchId, resourceId, applicationData);
    response.setXid(xid);
    response.setBranchId(branchId);
    response.setBranchStatus(status);
    if (LOGGER.isInfoEnabled()) {
        LOGGER.info("Branch Rollbacked result: " + status);
    }
}

/**
 * get resource manager implement
 *
 * @return resource manager
 */
}
```

这个方法，委托给了 ResourceManager

ResourceManager也是通过spi加载的，委托给DefaultResourceManager，最终定位到TCCResourceManager

```
TCCResourceManager > branchRollback
DefaultRMHandler.java x AbstractRMHandler.java x ResourceManagerInbound.java x TCCResourceManager.java x RMClient.java
@Override
public BranchStatus branchRollback(BranchType branchType, String xid, long branchId, String resourceId,
    String applicationData) throws TransactionException {
    //通过resource名称获取到具体的TCCResource对象，
    //在客户端GlobalTransactionScanner扫描的时候就将各分支方法的@TwoPhaseBusinessAction解析成TCCResource注册到map中

    TCCResource tccResource = (TCCResource)tccResourceCache.get(resourceId);
    if (tccResource == null) {
        throw new ShouldNeverHappenException(String.format("TCC resource is not exist, resourceId: %s", resourceId));
    }
    Object targetTCCBean = tccResource.getTargetBean(); //目标对象
    Method rollbackMethod = tccResource.getRollbackMethod(); //指定的rollback方法
    if (targetTCCBean == null || rollbackMethod == null) {
        throw new ShouldNeverHappenException(String.format("TCC resource is not available, resourceId: %s", resourceId));
    }
    try {
        //BusinessActionContext
        //将TC server端返回的applicationData、xid等组装成BusinessActionContext对象，供2阶段rollback方法使用
        BusinessActionContext businessActionContext = getBusinessActionContext(xid, branchId, resourceId, applicationData);
        //支持在try方法上的@TwoPhaseBusinessAction中rollbackArgsClasses + rollback方法参数注解的@BusinessActionContextParameter
        //以@BusinessActionContextParameter的value属性作为key，从businessActionContext中获取到json string，再反序列为Object[]
        Object[] args = this.getTwoPhaseRollbackArgs(tccResource, businessActionContext);
        Object ret;
        boolean result;
        // add idempotent and anti hanging | 幂等、空回滚、防悬挂处理
        if (Boolean.TRUE.equals(businessActionContext.getActionContext(Constants.USE_TCC_FENCE))) {
            try {
                result = TCCFenceHandler.rollbackFence(rollbackMethod, targetTCCBean, xid, branchId, args, tccResource.getActionName());
            } catch (Exception e) {
                // ...
            }
        }
    } catch (Exception e) {
        // ...
    }
}
```

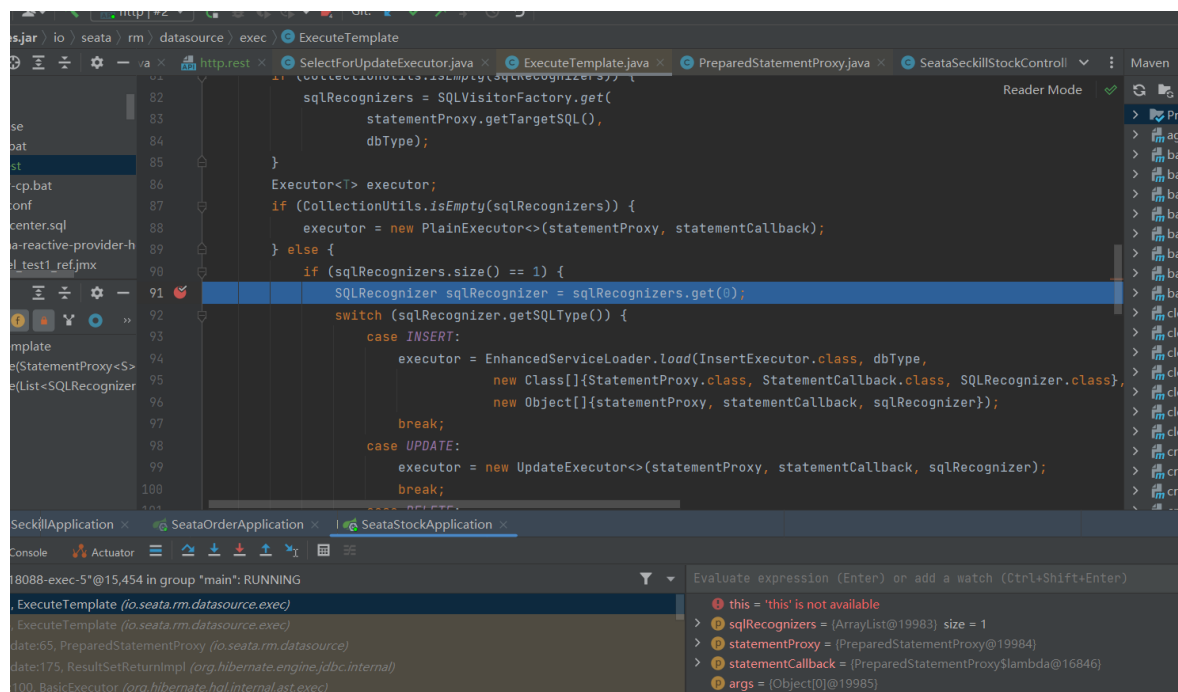
这里，调用了二阶段回滚屏障 `TCCFenceHandler.rollbackFence()`：，代码参见前面，不做赘述

TCC幂等&&空回滚&&防悬挂控制总结

1. 一阶段try: 执行前insert一条初始分支记录, 如果插入成功, 说明第二阶段还没有执行, 可以继续执行第一阶段。如果插入失败, 则说明第二阶段已经执行或正在执行, 抛出异常
2. 二阶段commit: 对于commit, 一定要保证commit方法在 Try 方法之后执行。
 1. 首先判断分支记录是否存在, 如果不存在, 直接抛出异常, 等待重试
 2. 如果记录存在且状态为已提交, 说明是重复提交请求, 直接返回成功
 3. 如果记录存在且状态为初始化, 执行commit方法逻辑, 并更新分支记录状态为已提交
3. 二阶段rollback: 对于rollback, 要支持空回滚, 防止事务悬挂
 1. 首先判断分支记录是否存在, 如果不存在, 则认为 Try 方法还没执行, 即是空回滚, 应该先插入一条事务记录, 确保后续的 Try 方法不会再次执行。
 2. 如果insert成功, 说明try未执行, 空回滚成功
 3. 如果insert失败, 说明try正在执行, 抛出异常, 等待TC重试
 4. 如果分支记录存在并且为已回滚状态, 说明是重复rollback请求, 直接返回成功
 5. 如果分支记录存在并且为初始化状态, 执行rollback方法逻辑, 并更新分支记录状态为已回滚

源码解读: @GlobalLock 注解 + select for update语句执行过程

执行过程在 StatementProxy 类, 会通过 ExecuteTemplate 去执行



在执行过程中如果执行 SQL 是 select for update, 则会使用 SelectForUpdateExecutor 类,

如果执行方法中带有 @GlobalTransactional or @GlobalLock注解, 则会检查是否有全局锁, 如果当前存在全局锁, 则会回滚本地事务, 通过 while 循环不断地重新竞争获取本地锁和全局锁。

源码位置: io.seata.rm.datasource.exec.SelectForUpdateExecutor#doExecute

```
c:\main> java> io> seata> rm> datasource> exec> SelectForUpdateExecutor> doExecute
ResourceManager.java x postgresql.sql x SelectForUpdateExecutor.java x ConnectionProxy.java x RMClient.java x AbstractNettyR
while (true) { // 死循环
    try {
        // #870
        // execute return Boolean
        // executeQuery return ResultSet
        rs = statementCallback.execute(statementProxy.getTargetStatement(), args);

        // Try to get global lock of those rows selected
        TableRecords selectPKRows = buildTableRecords(getTableMeta(), selectPKSQL, paramAppenderList);
        String lockKeys = buildLockKey(selectPKRows);
        if (StringUtils.isNullOrEmpty(lockKeys)) {
            break;
        }

        if (RootContext.inGlobalTransaction() || RootContext.requireGlobalLock()) {
            // Do the same thing under either @GlobalTransactional or @GlobalLock,
            // that only check the global lock here.
            //检查全局锁, 是不是 存在
            statementProxy.getConnectionProxy().checkLock(lockKeys);
        } else {
            throw new RuntimeException("Unknown situation!");
        }
        break;
    } catch (LockConflictException lce) {
        if (sp != null) {
            conn.rollback(sp); //回滚本地事务
        } else {
            conn.rollback();
        }
        // trigger retry
    }
}
```

源码中有注释: // Just check lock without requiring lock by now. 仅仅检查, 不抢占锁

```
main> java> io> seata> rm> datasource> ConnectionProxy> checkLock
sourceManager.java x postgresql.sql x SelectForUpdateExecutor.java x ConnectionProxy.java x RMClient.java x
2 usages skyex
public boolean isGlobalLockRequire() { return context.isGlobalLockRequire(); }

/**
 * Check lock.
 *
 * @param lockKeys the lockKeys
 * @throws SQLException the sql exception
 */
2 usages 2 overrides jimin.jm +3 *
public void checkLock(String lockKeys) throws SQLException {
    if (StringUtils.isBlank(lockKeys)) {
        return;
    }

    // Just check lock without requiring lock by now. 仅仅检查, 不抢占锁
    try {
        //检查全局锁, 是不是 存在

        boolean lockable = DefaultResourceManager.get().lockQuery(BranchType.AT,
            getDataSourceProxy().getResourceId(), context.getXid(), lockKeys);
        if (!lockable) {
            throw new LockConflictException(String.format("get lock failed, lockKey: %s", lockKeys));
        }
    } catch (TransactionException e) {
        recognizeLockKeyConflictException(e, lockKeys);
    }
}
```

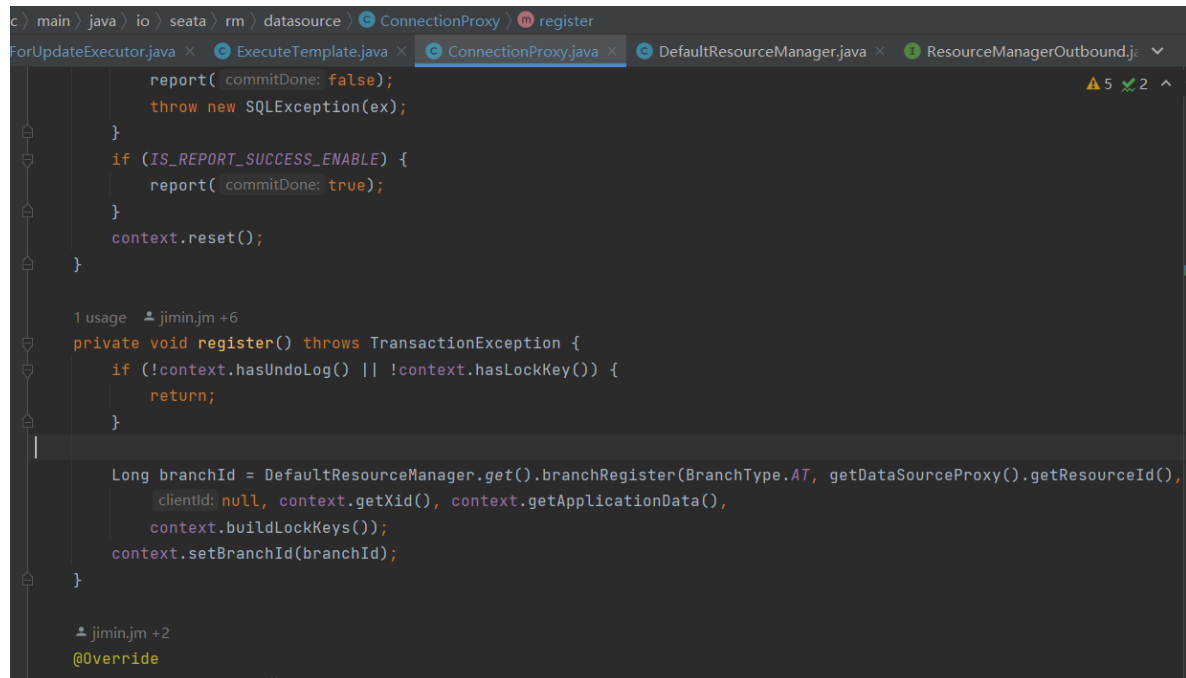
源码解读：提交过程

提交过程在 ConnectionProxy#doCommit方法中。

1) 如果执行方法中带有@GlobalTransactional注解，则会在注册分支时候获取全局锁：

- 请求 TC 注册分支

源码位置：io.seata.rm.datasource.ConnectionProxy#register



```
report( commitDone: false);
throw new SQLException(ex);
}
if (IS_REPORT_SUCCESS_ENABLE) {
    report( commitDone: true);
}
context.reset();
}

1 usage  jimin.jm +6
private void register() throws TransactionException {
    if (!context.hasUndoLog() || !context.hasLockKey()) {
        return;
    }

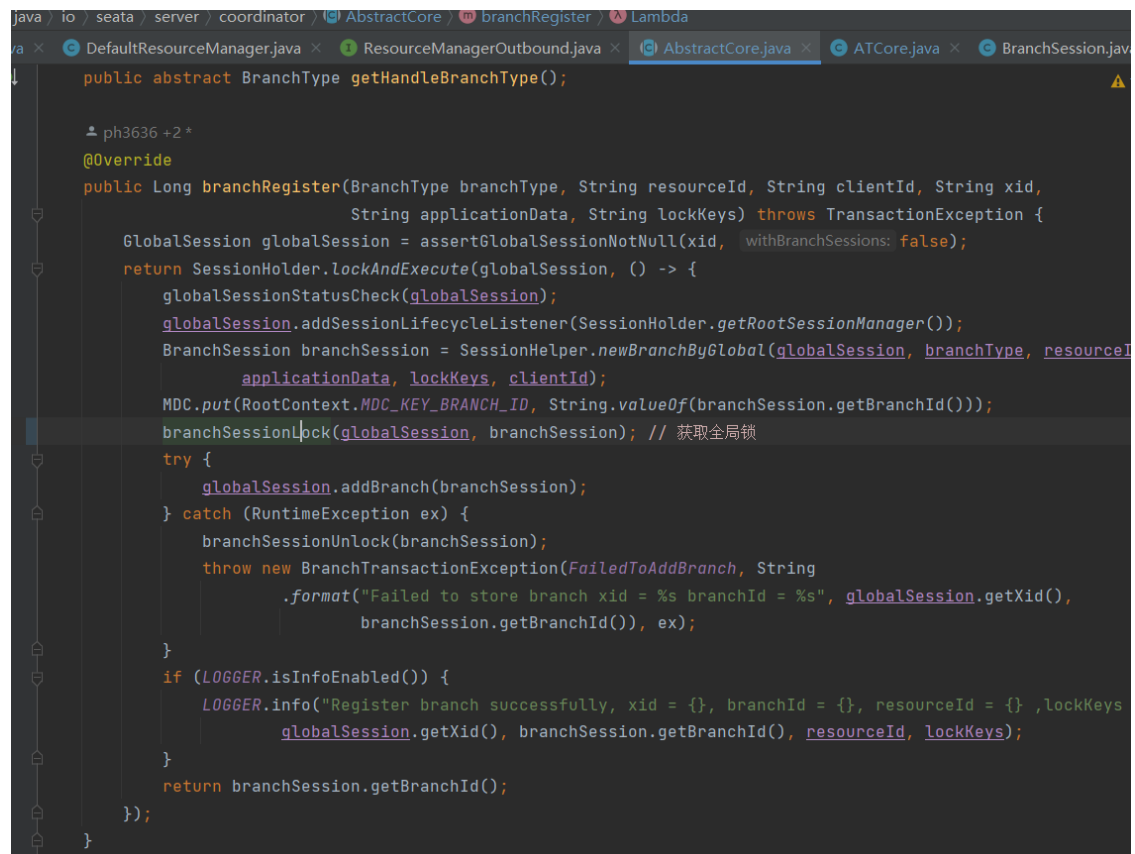
    Long branchId = DefaultResourceManager.get().branchRegister(BranchType.AT, getDataSourceProxy().getResourceId(),
        clientId: null, context.getXid(), context.getApplicationData(),
        context.buildLockKeys());
    context.setBranchId(branchId);
}

jimin.jm +2
@Override
public void rollback() throws SQLException {
```

- TC 注册分支的时候，获取全局锁

来到 Client 端（RM）注册分支的底层方法：

io.seata.server.coordinator.AbstractCore#branchRegister



```
public abstract BranchType getHandleBranchType();

ph3636 +2*
@Override
public Long branchRegister(BranchType branchType, String resourceId, String clientId, String xid,
    String applicationData, String lockKeys) throws TransactionException {
    GlobalSession globalSession = assertGlobalSessionNotNull(xid, withBranchSessions: false);
    return SessionHolder.lockAndExecute(globalSession, () -> {
        globalSessionStatusCheck(globalSession);
        globalSession.addSessionLifecycleListener(SessionHolder.getRootSessionManager());
        BranchSession branchSession = SessionHelper.newBranchByGlobal(globalSession, branchType, resourceId,
            applicationData, lockKeys, clientId);
        MDC.put(RootContext.MDC_KEY_BRANCH_ID, String.valueOf(branchSession.getBranchId()));
        branchSessionLock(globalSession, branchSession); // 获取全局锁
        try {
            globalSession.addBranch(branchSession);
        } catch (RuntimeException ex) {
            branchSessionUnlock(branchSession);
            throw new BranchTransactionException(FailedToAddBranch, String
                .format("Failed to store branch xid = %s branchId = %s", globalSession.getXid(),
                    branchSession.getBranchId()), ex);
        }
        if (LOGGER.isInfoEnabled()) {
            LOGGER.info("Register branch successfully, xid = {}, branchId = {}, resourceId = {}, lockKeys = {} ",
                globalSession.getXid(), branchSession.getBranchId(), resourceId, lockKeys);
        }
        return branchSession.getBranchId();
    });
}
```

源码位置：io.seata.server.transaction.at.ATCore#branchSessionLock


```
java io.seata.server.transaction.at ATCore branchSessionLock
a x DefaultResourceManager.java x ResourceManagerOutbound.java x AbstractCore.java x ATCore.java x BranchSession.java
1 usage ph3636
@Override
public BranchType getHandleBranchType() { return BranchType.AT; }

1 usage FUNKYE +3 *
@Override
protected void branchSessionLock(GlobalSession globalSession, BranchSession branchSession)
    throws TransactionException {
    String applicationData = branchSession.getApplicationData();
    boolean autoCommit = true;
    boolean skipCheckLock = false;
    if (StringUtils.isNotBlank(applicationData)) {...}
    try {
        // 获取全局锁
        if (!branchSession.lock(autoCommit, skipCheckLock)) {
            throw new BranchTransactionException(LockKeyConflict,
                String.format("Global lock acquire failed xid = %s branchId = %s", globalSession.getXid(),
                    branchSession.getBranchId()));
        }
    } catch (StoreException e) {
        if (e.getCause() instanceof BranchTransactionException) {
            throw new BranchTransactionException(((BranchTransactionException)e.getCause()).getCode(),
                String.format("Global lock acquire failed xid = %s branchId = %s", globalSession.getXid(),
                    branchSession.getBranchId()));
        }
        throw e;
    }
}
```

该方法的主要逻辑：

1. 使用 xid 获取到全局会话。
2. 锁定会话并在事务上下文进行操作。
3. 全局会话状态检查。
4. 全局会话添加会话生命周期监听器。
5. 新建全局事务分支，获得分支会话对象。
6. **锁定分支会话。**
7. 将分支会话添加到全局会话中，如果出现异常，解锁分支并抛出异常。
8. 打印日志并返回分支id。

2) 如果执行方法中带有@GlobalLock注解，在提交前会查询全局锁是否存在，如果存在则抛异常：

源码位置： io.seata.rm.datasource.ConnectionProxy#processLocalCommitWithGlobalLocks

```
ForUpdateExecutor.java × ExecuteTemplate.java × ConnectionProxy.java × DefaultResourceManager.java ×
    processLocalCommitWithGlobalLocks();
} else {
    targetConnection.commit();
}
}

1 usage 1 skyesx *
private void processLocalCommitWithGlobalLocks() throws SQLException {
    //用`ConnectionProxy#checkLock`判断有冲突的全局行锁是否存在
    // 也就是，本事务所操作的行记录构造成行锁记录是否跟在 TC 侧已存在的全局行锁记录有重复
    checkLock(context.buildLockKeys());
    try {
        targetConnection.commit();
    } catch (Throwable ex) {
        throw new SQLException(ex);
    }
    context.reset();
}
```

globallock 检测+本地事务提交场景，用 `ConnectionProxy#checkLock` 判断有冲突的全局行锁是否存在（本事务所操作的行记录构造成行锁记录是否跟在 TC 侧已存在的全局行锁记录有重复），

```
/**
 * Check lock.
 *
 * @param lockKeys the lockKeys
 * @throws SQLException the sql exception
 */
2 usages 2 overrides 1 jimin.jm +3 *
public void checkLock(String lockKeys) throws SQLException {
    if (StringUtils.isBlank(lockKeys)) {
        return;
    }
    // Just check lock without requiring lock by now. 仅仅检查，不抢占锁
    try {
        //检查全局锁，是不是 存在

        boolean lockable = DefaultResourceManager.get().lockQuery(BranchType.AT,
            getDataSourceProxy().getResourceId(), context.getXid(), lockKeys);
        if (!lockable) {
            throw new LockConflictException(String.format("get lock failed, lockKey: %s", lockKeys));
        }
    } catch (TransactionException e) {
        recognizeLockKeyConflictException(e, lockKeys);
    }
}
```

如果不存在全局锁，则通过 `targetConnection.commit()` 提交本地事务；

这个过程中只是检查，并不会在 TC 中添加全局行锁记录。

终于明白：GlobalLock 注解的价值

在全面使用 Seata 分布式事务场景，开启全局事务 `@GlobalTransactional` 注解可以实现 RC（读已提交），解决脏读问题。

但是，在 Seata 分布式事务+ 独立事务场景，分布式事务的隔离级别是 RU，会发生脏读和脏写。

为什么 Seata 还要设计多处一个 @GlobalLock 注解呢？可以在 Seata 分布式事务+ 独立事务场景，查询全局锁是否存在，解决脏读和脏写问题。

因为并不是所有的数据库操作都需要开启全局事务，而开启全局事务是一个比较重的操作，需要向 TC 发起开启全局事务等 RPC 过程，而 @GlobalLock 注解只会在执行过程中查询全局锁是否存在，不会去开启全局事务，

因此，在不需要全局事务，而又需要检查全局锁避免脏读脏写时，使用 @GlobalLock 注解是一个更加轻量的操作。

XID 如何进行远程传递

至此，我们知道了 TM 是如何觉察到全局事务需求(GlobalTransactional 注解)，如何创建全局事务(RPC 调用 TC 接口)，如何通过模板方法模式完成对业务的封装(GlobalTransactionTemplate)。

那么还有一个问题，事务信息是怎么传递的呢，TM 怎么将全局事务 XID 传递给 RPC 的提供者的呢？

RPC 框架会拦截 RPC 的调用过程，在 RPC 请求中加一个隐藏属性来存储 XID，RPC 的提供方从请求中获取到该隐藏属性，然后存储在事务 Context 的 ThreadLocal 中，

这部分，根据 RPC 框架的不同，需要不同的实现，但是本质上都是一样的，

我们以 Dubbo 为例，看一看它是这么做的：

```
@Activate(group = {Constants.PROVIDER, Constants.CONSUMER}, order = 100)
public class TransactionPropagationFilter implements Filter {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(TransactionPropagationFilter.class);

    @Override
    public Result invoke(Invoker<?> invoker, Invocation invocation) throws
        RpcException {
        String xid = RootContext.getXID();
        String rpcXid =
            RpcContext.getContext().getAttachment(RootContext.KEY_XID);
        if (LOGGER.isDebugEnabled()) {
            LOGGER.debug("xid in RootContext[" + xid + "] xid in RpcContext[" +
                rpcXid + "]");
        }
        boolean bind = false;
        if (xid != null) {
            // 如果当前存在 xid 说明是 RPC 发起方，将 xid 存在 RPC context 中，它会随着
            RPC request 一起发送
            RpcContext.getContext().setAttachment(RootContext.KEY_XID, xid);
        } else {
            // 否则他就是 RPC 提供方，它从 rpc context 中拿到 xid，并设置到事务
            context 中
            if (rpcXid != null) {
                RootContext.bind(rpcXid);
                bind = true;
                if (LOGGER.isDebugEnabled()) {
                    LOGGER.debug("bind[" + rpcXid + "] to RootContext");
                }
            }
        }
    }
}
```

```

    }
}
try {
    return invoker.invoke(invocation);

} finally {
    // 最后清除事务 context 中绑定的 xid，防止 ThreadLocal 内容污染下次调用
    if (bind) {
        String unbindXid = RootContext.unbind();
        if (LOGGER.isDebugEnabled()) {
            LOGGER.debug("unbind[" + unbindXid + "] from RootContext");
        }
        if (!rpcXid.equalsIgnoreCase(unbindXid)) {
            LOGGER.warn("xid in change during RPC from " + rpcXid + " to " + unbindXid);

            if (unbindXid != null) {
                RootContext.bind(unbindXid);
                LOGGER.warn("bind [" + unbindXid + "] back to RootContext");
            }
        }
    }
}
}
}
}

```

Dubbo 是通过一个 Filter 的概念来表示 AOP 特性的，Filter 的注入基于 Dubbo 的 SPI，而 Seata 这里所做的就是实现了一个 Dubbo Filter，把事务 Context 和 RPCContext 中的数据做一下绑定。

其他 RPC 框架的支持方案基本类似, 这里就不再赘述。

,值得一提的是,如果全局事务中的各个微服务是通过 HTTP 请求来互相调用的话,可以将 XID 存储在 HTTP Header 中,在官方的提供的 Sample 中有一份样例代码:

```
// 实现 servlet filter，这样服务提供者能从 Http request 中获取 xid 并绑定进事务 Context
@Component
public class SeataFilter implements Filter {
    @Override
    public void doFilter(ServletRequest servletRequest, ServletResponse
servletResponse, FilterChain filterChain) throws IOException, ServletException {
        HttpServletRequest req = (HttpServletRequest) servletRequest;
        String xid = req.getHeader(RootContext.KEY_XID.toLowerCase());
        boolean isBind = false;
        if (StringUtils.isNotBlank(xid)) {
            RootContext.bind(xid);
            isBind = true;
        }
        try {
            filterChain.doFilter(servletRequest, servletResponse);
        } finally {
            if (isBind) {
                RootContext.unbind();
            }
        }
    }
}
```

```

}
// 实现 ClientHttpRequestInterceptor, 服务请求方讲 xid 塞入 http request header
public class SeataRestTemplateInterceptor implements
ClientHttpRequestInterceptor {

    public ClientHttpResponse intercept(HttpRequest httpRequest, byte[] bytes,
ClientHttpRequestExecution clientHttpRequestExecution) throws IOException {
        HttpRequestWrapper requestWrapper = new HttpRequestWrapper(httpRequest);
        String xid = RootContext.getXID();
        if (StringUtils.isEmpty(xid)) {
            requestWrapper.getHeaders().add(RootContext.KEY_XID, xid);
        }

        return clientHttpRequestExecution.execute(requestWrapper, bytes);
    }
}
// 找到所有 RestTemplate 将 SeataRestTemplateInterceptor 注入
@Configuration
public class SeataRestTemplateAutoConfiguration {
    @Autowired(required = false)
    private Collection<RestTemplate> restTemplatees;
    @Autowired
    private SeataRestTemplateInterceptor seataRestTemplateInterceptor;

    @PostConstruct
    public void init() {
        if (this.restTemplatees != null) {
            Iterator var1 = this.restTemplatees.iterator();
            while (var1.hasNext()) {
                RestTemplate restTemplate = (RestTemplate) var1.next();
                List<ClientHttpRequestInterceptor> interceptors = new
ArrayList(restTemplate.getInterceptors());
                interceptors.add(this.seataRestTemplateInterceptor);
                restTemplate.setInterceptors(interceptors);
            }
        }
    }
}

```

seata 压力测试

特别说明，尼恩太忙，没有时间准备压测的环境和实验

首先给两个网上的其他小伙伴的压测数据

压力测试 参考实验1： AT 20 tps

[Seata入门系列\(26\)-Seata压力测试-CSDN博客](#)

压力测试 参考实验 2: TCC 100tps, AT 20 tps

[论Seata是否适合高并发下的分布式事务解决方案（附测试结果） seata并发量-CSDN博客](#)

在单机环境下使用 jmeter 进行压测,

Seata AT模式: 在100并发下成功数是 23、29, 成功率不到30%, 在1000并发下成功数是276、260, 成功率同样是不到30%

Seata TCC模式: 在100并发下成功数 98、80, 500并发下成功数是304, 在1000并发下成功数是481

Seata相关的面试题以及参考答案

阿里面试: Seata 如何实现 RC? 保证事务的隔离性?

尼恩说在前面

在40岁老架构师 尼恩的[读者交流群](#)(50+)中, 最近有小伙伴拿到了一线互联网企业如阿里、滴滴、极兔、有赞、希音、百度、网易、美团的面试资格, 遇到很多很重要的面试题:

Seata 如何实现 RC? 保证事务的隔离性?

最近有小伙伴在面试阿里, 又遇到了相关的面试题。小伙伴懵了, 因为没有遇到过, 所以支支吾吾的说了几句, 面试官不满意, 面试挂了。

所以, 尼恩给大家做一下系统化、体系化的梳理, 使得大家内力猛增, 可以充分展示一下大家雄厚的“技术肌肉”, 让面试官爱到“不能自己、口水直流”, 然后实现“offer直提”。

当然, 这道面试题, 以及参考答案, 也会收入咱们的《[尼恩Java面试宝典PDF](#)》V140版本, 供后面的小伙伴参考, 提升大家的 3高 架构、设计、开发水平。

《尼恩 架构笔记》《尼恩高并发三部曲》《尼恩Java面试宝典》的PDF, 请到文末公号【技术自由圈】获取

Seata 事务隔离级别解读

先来回顾一下, 数据库事务的隔离级别, 目前数据库事务的隔离级别一共有 4 种, 由低到高分别为:

事务的四个隔离级别:

- **未提交读 (READ UNCOMMITTED)**: 所有事务都可以看到其他事务未提交的修改。一般很少使用;
- **读已提交 (READ COMMITTED)**: Oracle默认隔离级别, 事务之间只能看到彼此已提交的变更修改;
- **可重复读 (REPEATABLE READ)**: MySQL默认隔离级别, 同一事务中的多次查询会看到相同的数据行; 可以解决不可重复读, 但可能出现幻读;

- **可串行化 (SERIALIZABLE)**：最高的隔离级别，事务串行的执行，前一个事务执行完，后面的事务会执行。读取每条数据都会加锁，会导致大量的超时和锁争用问题；

隔离级别	脏读可能性	不可重复读可能性	幻读可能性	加锁读
READ UNCOMMITTED	Yes	Yes	Yes	No
READ COMMITTED	No	Yes	Yes	No
REPEATABLE READ	No	No	Yes	No
SERIALIZABLE	No	No	No	Yes

数据库一般默认的隔离级别为 读已提交 RC，比如 Oracle，

也有一些数据的默认隔离级别为 可重复读 RR，比如 Mysql。"可重复读" (Repeatable Read) 这个级别确保了对同一字段的多次读取结果是一致的，除非数据是被本身事务自己所修改。它能够防止脏读、不可重复读，但可能会遇到幻读的情况。

参考 文章：

[Mysql如何实现RR级隔离时，不会幻读？](#)

MySQL默认的Repeatable Read隔离级别，被改成了RC，具体请参考 《尼恩Java 面试宝典》 MYSQL 专题：

专题23: Mysql 面试题 (卷王专供+ 史上最全 + 2023面试必备) -V106-from-尼恩Java面试宝典.md - Typora

文件(F) 编辑(E) 段落(P) 格式(O) 视图(V) 主题(T) 帮助(H)

文件

大纲

美团面试题：一个SQL，怎么分析加了哪些锁？
背景说明：
加锁的场景分析
回顾一下，InnoDB 三种行锁：
两条最为基础的规则
实例分析
场景1：唯一索引等值查询
场景1.1、查询的记录存在
场景1.2、查询的记录不存在
场景2：唯一索引范围查询
场景3：非唯一索引等值查询
场景3.1、查询的记录存在
场景3.2、查询的记录不存在
场景4：非唯一索引范围查询
阿里二面：为什么MySQL默认的Repeatable Read隔离级别，被改成了RC？
ISO SQL定义的标准隔离级别有四种
RR在高并发场景的性能损耗
RR在高并发场景的性能损耗
总之：
聊聊：count(1)、count(*) 与 count(列名)的区别？
聊聊：exist和in的区别？
聊聊：truncate、delete与drop区别？
聊聊：union与union all的区别？
聊聊：group by 和 distinct 的区别？
聊聊：Blob和text有什么区别？
聊聊：数据库的三大范式？
聊聊：什么是存储过程？有哪些优缺点？

阿里二面：为什么MySQL默认的Repeatable Read隔离级别，被改成了RC？

前段时间，一位小伙伴面试了阿里，遇到了一个 MySQL 日志的面试题：
为什么MySQL默认的Repeatable Read隔离级别，被改成了RC？
这个问题，背后的原理比较复杂，而且又很重要。
现在把这个 题目，以及参考答案，收入咱们的《尼恩Java面试宝典》，供后面的小伙伴参考，前车之鉴啊

ISO SQL定义的标准隔离级别有四种

我们知道，我们可以通过这个命令查看数据库当前的隔离级别，MySQL 默认隔离级别是RR。

```
select @@tx_isolation;
```

ANSI/ISO SQL定义的标准隔离级别有四种，从高到底依次为：
可序列化(Serializable)、可重复读(Repeatable Reads)、提交读(Read Committed)、未提交读(Read Uncommitted)。
这四个隔离级别可以解决脏读、不可重复读、幻象读这三类问题。总结如下

事务隔离级别	脏读	不可重复读	幻读
读未提交 RU	允许	允许	允许
读已提交 RC	不允许	允许	允许
可重复读 RR	不允许	不允许	允许

一般而言，数据库的读已提交（READ COMMITTED）能够满足业务绝大部分场景了。

但是 seata 却做不到 RC，只能做到RU。

Seata 的事务是一个全局事务，它包含了若干个分支本地事务，在全局事务执行过程中（全局事务还没执行完），某个本地事务提交了，如果 Seata 没有采取任务措施，则会导致已提交的本地事务被读取，造成脏读，如果数据在全局事务提交前已提交的本地事务被修改，则会造成脏写。

由此可以看出，传统意义的脏读是读到了未提交的数据，Seata 脏读是读到了全局事务下未提交的数据，全局事务可能包含多个本地事务，某个本地事务提交了不代表全局事务提交了。

page:118/133 of 疯狂创客圈Java架构班配套讲义：跟着40岁老架构师-尼恩，狠卷3高架构，穿透3高架构，走向技术自由！

从这个意义上说，seata 的隔离级别是 读未提交。而实际上，这当中又有绝大多数的应用场景，实际上工作在读未提交的隔离级别下同样没有问题。

如何实现 读已提交？

默认情况下，Seata 的全局事务是工作在读未提交隔离级别的，保证绝大多数场景的高效性。

但是，在极端场景下，应用如果需要达到全局的读已提交，Seata 也提供了全局锁机制实现全局事务读已提交 RC。

Seata 的事务隔离级别

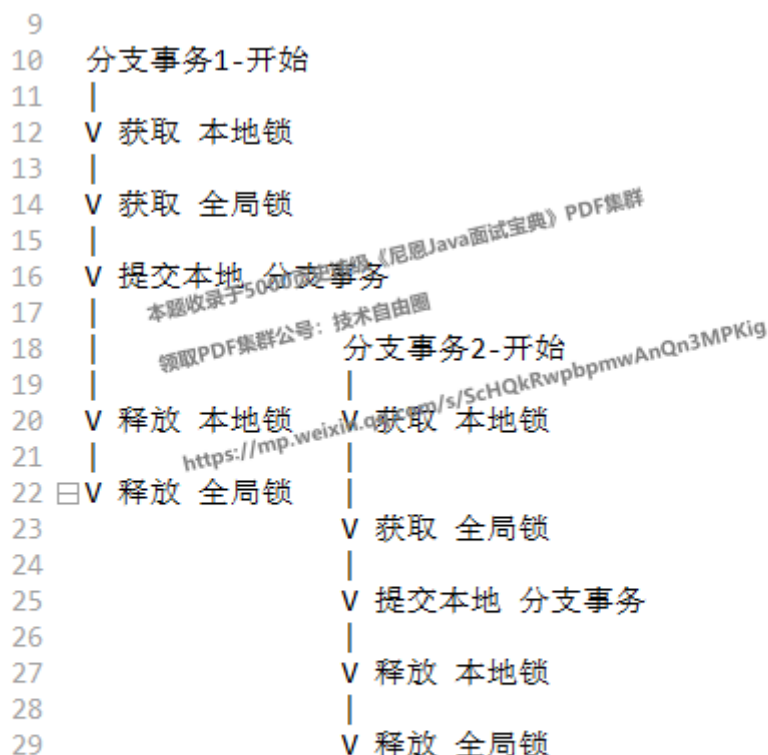
首先看结论：

- Seata 事务 和 Seata 事务 之间，是 RC 的隔离级别，实现了可重复度，没有脏读问题
- Seata 事务 和 独立事务 之间，是 RU 的隔离级别，存在 脏读 和脏写问题

接下来，我们一点一点的分析问题，然后找出解决方案。

AT 模式下，会使用 Seata 内部数据源代理 DataSourceProxy，全局锁的实现就是隐藏在这个代理中。

接下来看看，seata at分别在执行、提交的过程都做了什么。



如上所示，一个分布式事务的锁获取流程是这样的

1. 先获取到本地锁即可修改本地数据，但不能直接提交本地事务
2. 为何不能直接提交本地事务呢？因为还未获取全局锁
3. 本地锁和全局锁都获得了，才具有全局事务写隔离的保障，本地事务才可以提交，之后释放本地锁
4. 当分布式事务执行 2 阶段提交后释放全局锁；这样就可以让其它事务获取全局锁，并提交它们对本地数据的修改了。

至此已明确 Seata 通过将传统 XA 方案的 2 个阶段的本地 DB 锁，拆分成了 本地锁(DB 锁)+ 全局锁的模式。

如果一个应用所有的事务都用Seata 分布式事务，性能一定是很低的。 具体请参见 尼恩《Seata 学习圣经 的性能部分内容》。

所以，在大部分业务场景，并不是所有的 操作都要用分布式操作，并不是所有的 操作都要用Seata分布式事务，所以生产场景基本上会是 **Seata事务 + 独立事务** 组合模式。

那么，在 **Seata事务 + 独立事务** 场景下，Seata 的全局锁对于 外部的 独立事务失效了，对于外部事务，本质上是Seata默认的全局事务是读未提交，能读到seata 未提交（Seata分支已提交的分支事务的数据）的数据，于是，本地事务会发生 脏读。

如何实现 **Seata事务 + 独立事务** 场景的 RC？如何解决 独立本地事务的 脏读问题？

Seata事务 + 独立事务 场景的 脏读、脏写

AT 模式是一种非侵入式的分布式事务解决方案，Seata 在内部做了对数据库操作的代理层，我们使用 Seata AT 模式时，实际上用的是 Seata 自带的数据源代理 DataSourceProxy。

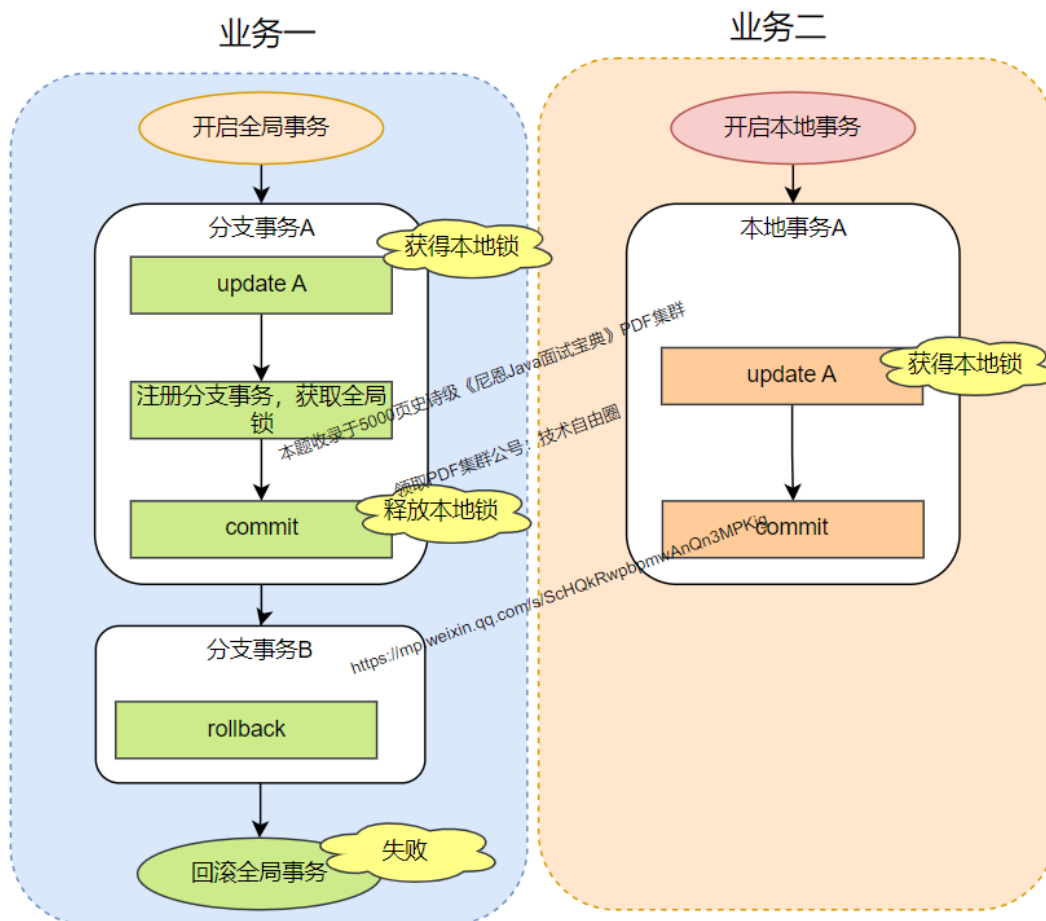
Seata 在这层DataSourceProxy 代理中加入了很多逻辑，比如：

- 插入回滚 undo_log 日志，
- 检查全局锁等。

为什么要检查全局锁呢，这是由于 Seata AT 模式的事务隔离是建立在支事务的本地隔离级别基础之上的，

在数据库本地隔离级别读已提交或以上的前提下，Seata 设计了由TC维护的全局写排他锁，来保证事务间的写隔离，实现分布式事务的 RC。

先来看一下Seata事务 + 独立事务 场景，使用 Seata AT 模式是怎么产生脏读的：



业务一开启seata 分布式事务，其中包含分支事务A（修改 A）和分支事务 B（修改 B），

业务二开启本地事务，仅仅修改 A，

其中业务一执行分支事务 A 先获取本地锁，

业务二则等待业务一执行完分支事务 A 之后，获得本地锁修改 A 并提交，

业务一在执行分支事务时发生异常了，由于分支事务 A 的数据被业务二修改，导致业务一的全局事务无法回滚，需要人工干预。

这里发生了两种错误的现象：

- 业务2发生了脏读
- 业务1 发生了脏写

如何防止脏读、脏写？ 帮助 Seata+ 独立事务 场景实现 RC。

方案有两种：

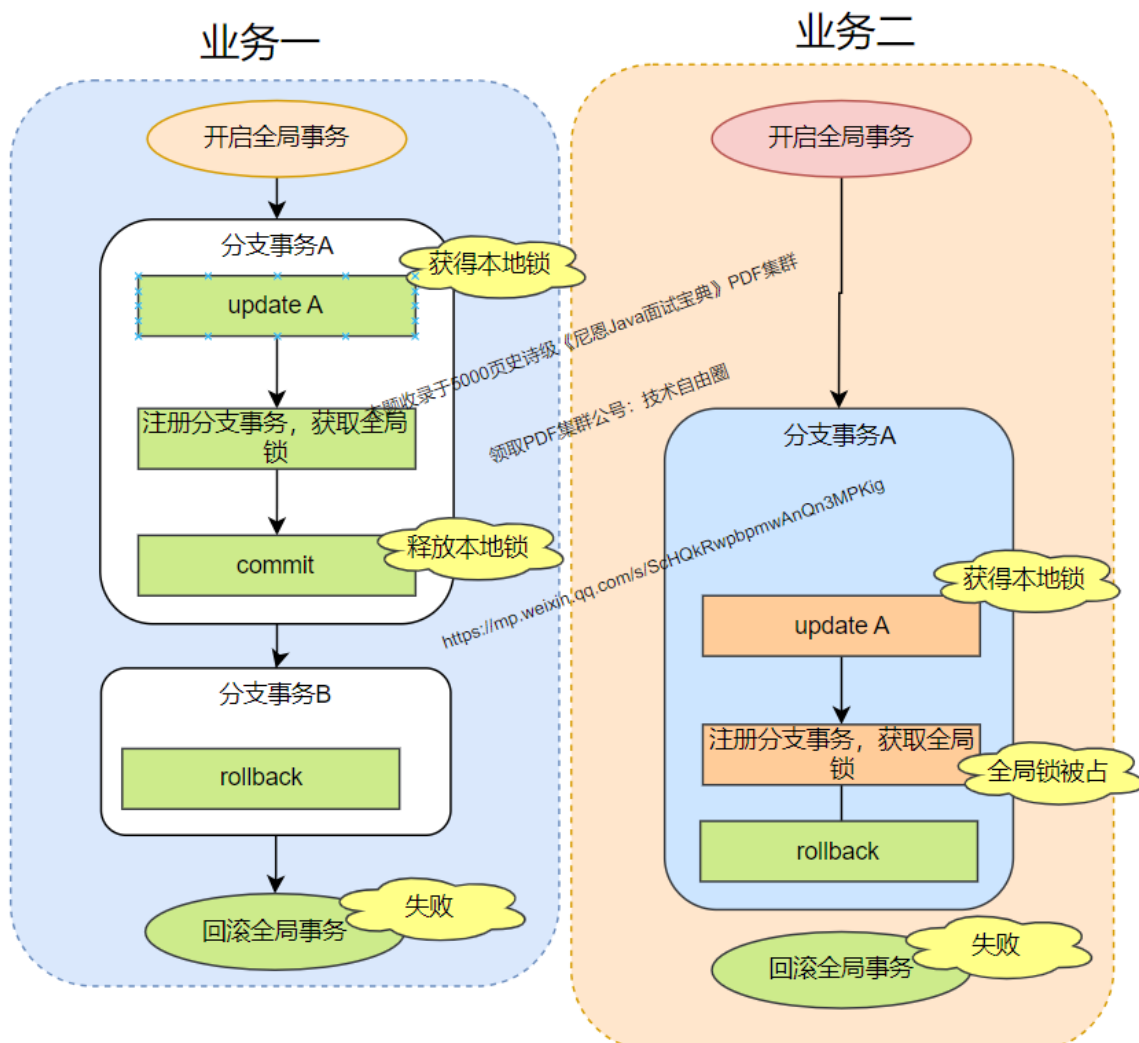
- 独立事务 升级为分布式事务
- 业务二查询 A 时加 @GlobalLock 注解 + select for update语句

如何防止脏读/脏写

Seata AT 模式的脏读是指在全局事务未提交前，被其它业务读到已提交的分支事务的数据，本质上是 Seata默认的全局事务是读未提交。

那么怎么避免脏读、脏写现象呢？

方案一：独立事务执行时加 @GlobalTransactional注解：

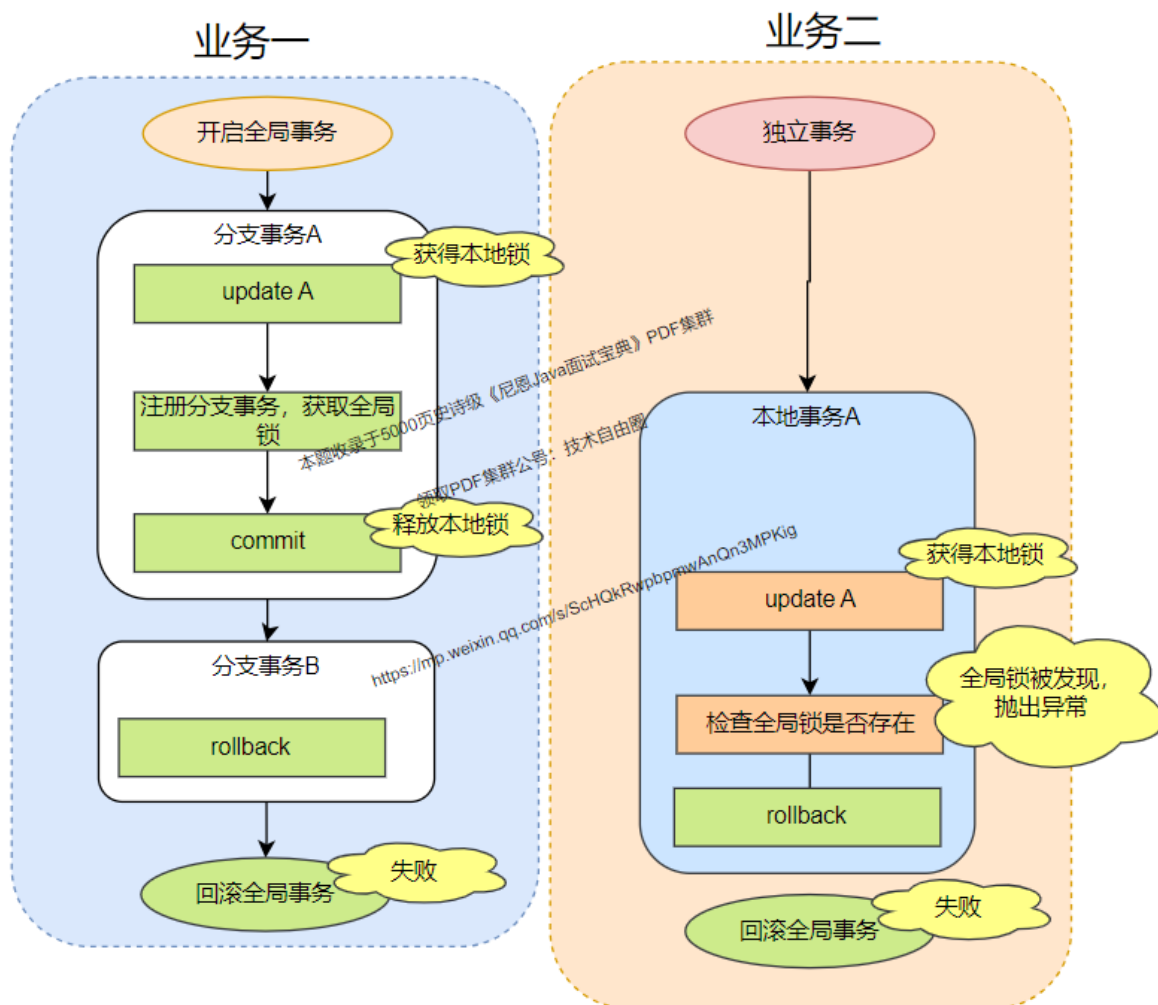


业务二在执行全局事务过程中，分支事务 A 提交前注册分支事务获取全局锁时，发现业务业务一全局锁还没执行完，因此业务二提交不了，抛异常回滚，所以不会发生脏写。

方案二：独立事务业务二执行时加 @GlobalLock注解：

全局事务 很重。

因此，在不需要全局事务、而又需要检查全局锁避免脏读脏写的场景，可以使用@GlobalLock注解实现隔离的功能，@GlobalLock是一个更加轻量的操作。



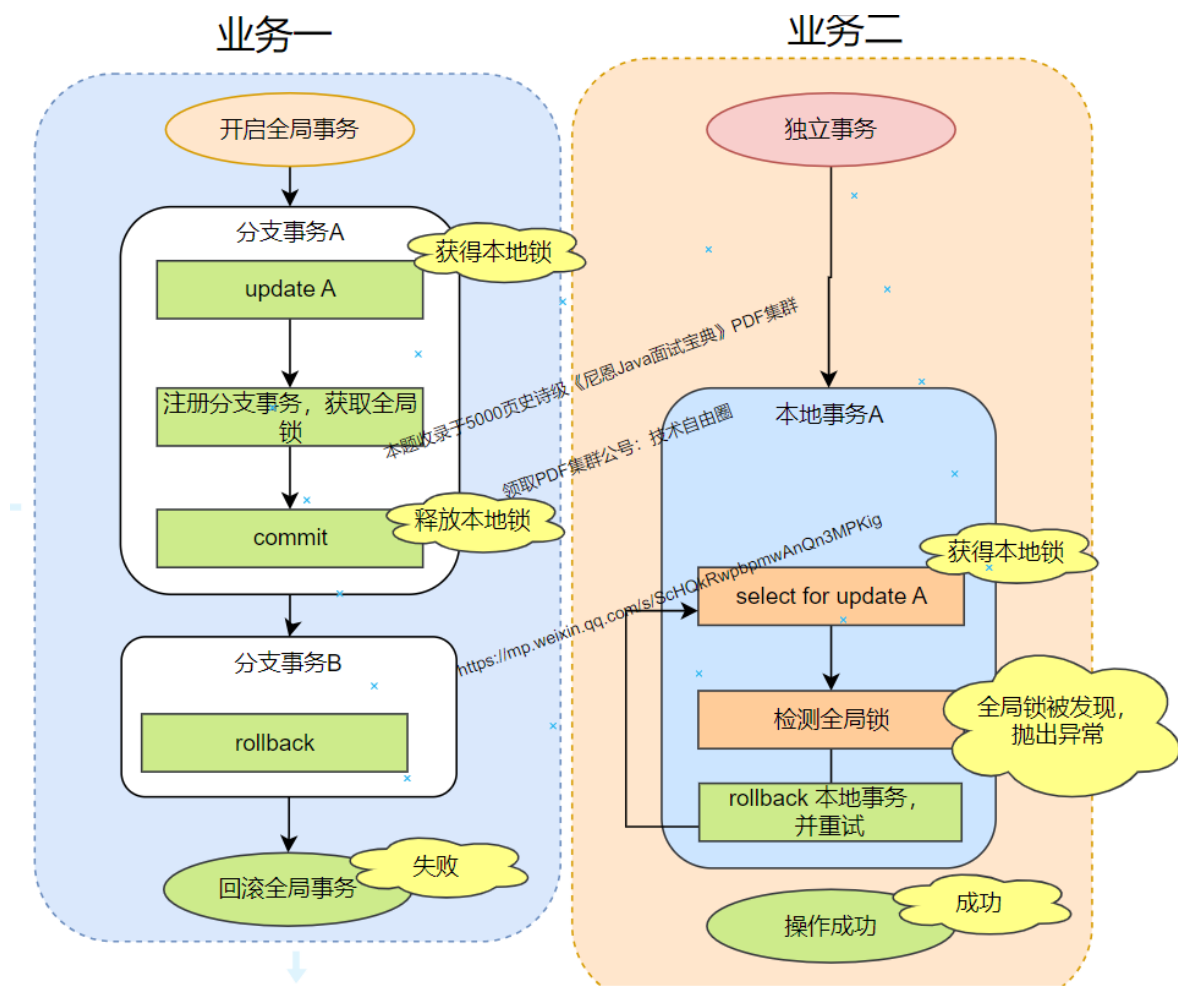
与 @GlobalTransactional 注解效果类似，只不过 业务二 不需要开启全局事务，只在本地事务提交前，检查全局锁是否存在。如果存在，就抛出异常，从而本地事务回滚。

@GlobalTransactional 注解 开启全局事务是一个比较重的操作，需要向 TC 发起开启全局事务等 RPC 过程，而 @GlobalLock 注解只会在执行过程中查询全局锁是否存在，不会去开启全局事务，

方案三：业务二执行时加 @GlobalLock 注解 + select for update 语句：

如果独立事务希望最终能够提交，在发现全局锁的过程中，独立事务希望等待和重试，则可以使用下面的方案：

加 @GlobalLock 注解 + select for update 语句



如果加了select for update语句, 则会在 update 前检查全局锁是否存在, 只有当全局锁释放之后, 业务二才能开始执行 updateA 操作。

加select for update语句会在执行 SQL 前检查全局锁是否存在, 只有当全局锁完成之后, 才能继续执行 SQL, 这样就彻底防止了脏读+脏写, 也能保护 独立事务不会 直接回滚。

所以@GlobalLock 注解 加 select for update 也有个好处, 就是可以重试。

说在最后：有问题找老架构取经

以上的内容, 如果大家能对答如流, 如数家珍, 基本上 面试官会被你 震惊到、吸引到。

最终, 让面试官爱到 “不能自己、口水直流”。offer, 也就来了。

在面试之前, 建议大家系统化的刷一波 5000页《[尼恩Java面试宝典PDF](#)》, 里边有大量的大厂真题、面试难题、架构难题。很多小伙伴刷完后, 吊打面试官, 大厂横着走。

在刷题过程中, 如果有啥问题, 大家可以来找 40岁老架构师尼恩交流。

另外, 如果没有面试机会, 可以找尼恩来改简历、做帮扶。

遇到职业难题, 找老架构取经, 可以省去太多的折腾, 省去太多的弯路。

尼恩指导了大量的小伙伴上岸, 前段时间, 刚指导一个40岁+被裁小伙伴, 拿到了一个年薪100W的 offer。

狠狠卷, 实现 “offer自由” 很容易的, 前段时间一个武汉的跟着尼恩卷了2年的小伙伴, 在极度严寒/痛苦被裁的环境下, offer拿到手软, 实现真正的 “offer自由”。

面试真题： 怎么使用Seata框架，来保证事务的隔离性？

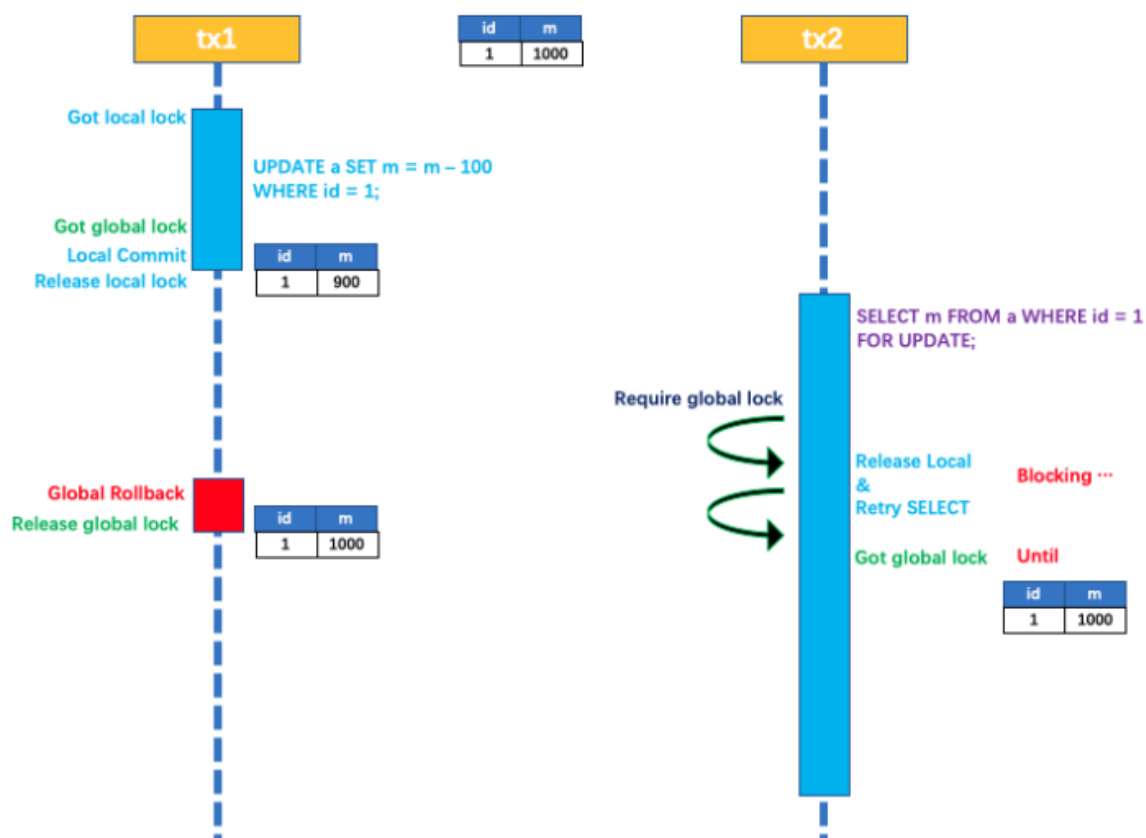
Seata隔离级别

由于Seata一阶段直接提交了本地事务，因此会造成隔离性问题，因此Seata的默认隔离级别为 Read Uncommitted。

然而Seata也支持Read Committed的隔离级别，该如何如何实现呢？

Seata由于一阶段RM自动提交本地事务的原因，默认隔离级别为Read Uncommitted。如果希望隔离级别为Read Committed，那么可以使用 `SELECT...FOR UPDATE` 语句。

Seata引擎重写了 `SELECT...FOR UPDATE` 语句执行逻辑，`SELECT...FOR UPDATE` 语句的执行会申请全局锁，如果全局锁被其他事务持有，则释放本地锁（回滚 `SELECT...FOR UPDATE` 语句的本地执行）并重试。这个过程中，查询是被 block 住的，直到全局锁拿到，即读取的相关数据是已提交的才返回。



SELECT FOR UPDATE.PNG

出于总体性能上的考虑，Seata 目前的方案并没有对所有 SELECT 语句都进行代理，仅针对 FOR UPDATE 的 SELECT 语句。

因 seata一阶段本地事务已提交，为防止其他事务脏读脏写需要加强隔离。

【1】脏读 select语句加 for update（会去检查全局锁），代理方法增加 `@GlobalLock`或 `@GlobalTransaction`；

【2】脏写 必须使用 `@GlobalTransaction`；

【3】注：如果你查询的业务接口没有 `GlobalTransactional` 包裹，也就是这个方法上压根没有分布式事务的需求，这时你可以在方法上标注 `@GlobalLock`注解，并且在查询语句上加 `for update`。如果你查询的接口在事务链路上外层有 `GlobalTransactional`注解，那么你查询的语句只要加 `for update`就行。

设计这个注解的原因是在没有这个注解之前，需要查询分布式事务读已提交的数据，但业务本身不需要分布式事务。若使用 GlobalTransactional注解就会增加一些没用的额外的 rpc开销比如 begin 返回xid，提交事务等。GlobalLock简化了 rpc过程，使其做到更高的性能。

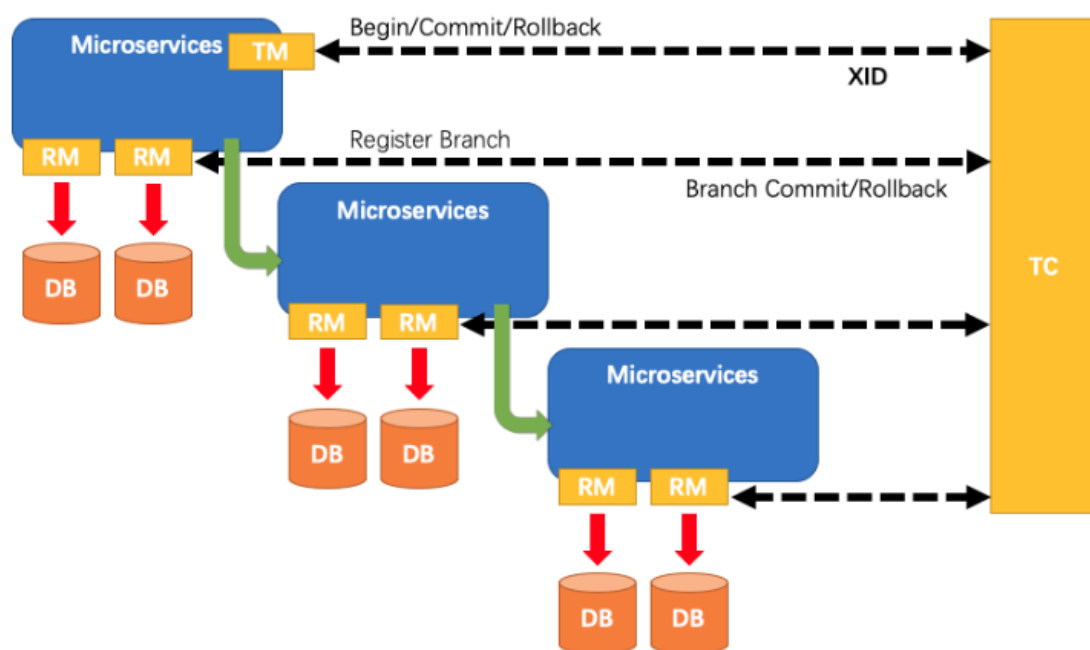
默认spring只在发生未被捕获的runtimeexception时才回滚。

面试题：说说，Seata AT模式的事务流程

Seata的分布式事务解决方案是业务层面的解决方案，只依赖于单台数据库的事务能力。Seata框架中一个分布式事务包含3中角色：

- **Transaction Coordinator (TC)**：事务协调器，维护全局事务的运行状态，负责协调并驱动全局事务的提交或回滚。
- **Transaction Manager™**：控制全局事务的边界，负责开启一个全局事务，并最终发起全局提交或全局回滚的决议。
- **Resource Manager (RM)**：控制分支事务，负责分支注册、状态汇报，并接收事务协调器的指令，驱动分支（本地）事务的提交和回滚。

其中，TM是一个分布式事务的发起者和终结者，TC负责维护分布式事务的运行状态，而RM则负责本地事务的运行。如下图所示：



下面是一个分布式事务在Seata中的执行流程：

1. TM 向 TC 申请开启一个全局事务，全局事务创建成功并生成一个全局唯一的 XID。
2. XID 在微服务调用链路的上下文中传播。
3. RM 向 TC 注册分支事务，接着执行这个分支事务并提交（重点：RM在第一阶段就已经执行了本地事务的提交/回滚），最后将执行结果汇报给TC。
4. TM 根据 TC 中所有的分支事务的执行情况，发起全局提交或回滚决议。
5. TC 调度 XID 下管辖的全部分支事务完成提交或回滚请求。

回顾：Seata执行流程

下面是一个Seata中一个分布式事务执行的详细过程：

1. 首先TM 向 TC 申请开启一个全局事务，全局事务创建成功并生成一个全局唯一的 XID。

2. XID 在微服务调用链路的上下文中传播。
3. RM 开始执行这个分支事务，RM首先解析这条SQL语句，生成对应的UNDO_LOG记录。下面是一条UNDO_LOG中的记录：

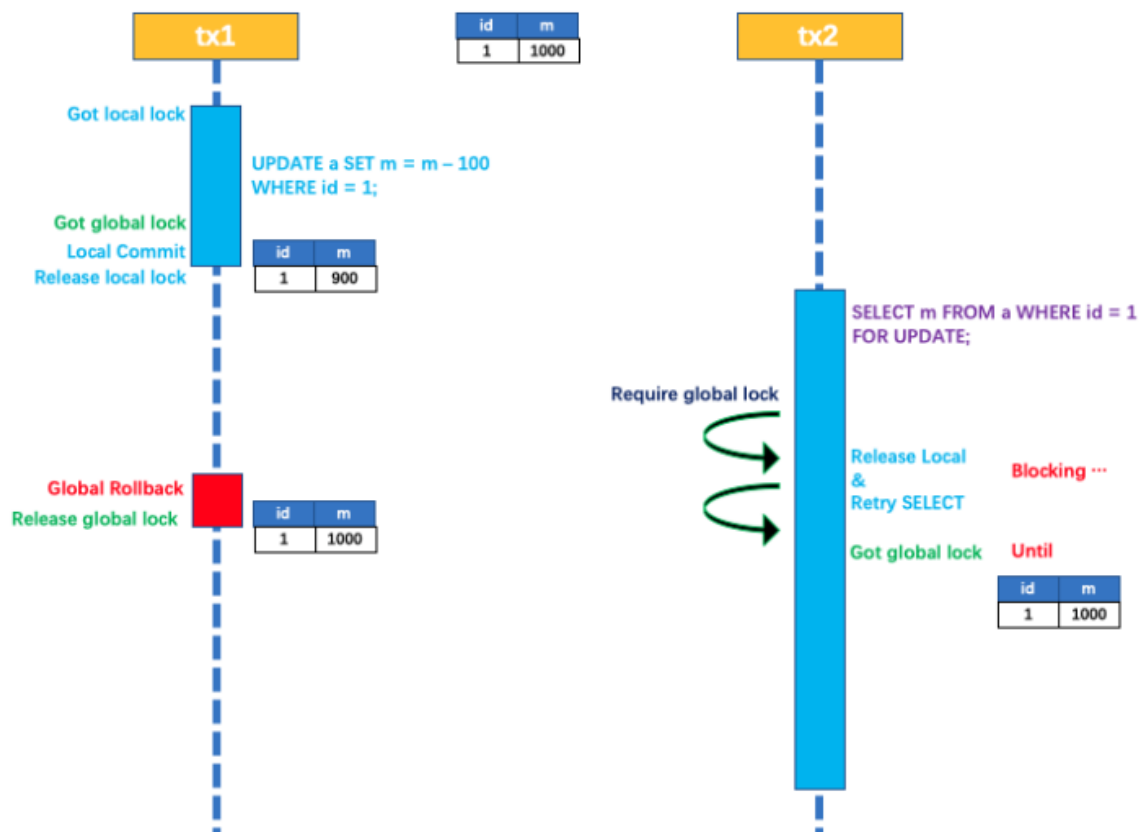
```
{
  "branchId": 641789253,
  "undoItems": [{
    "afterImage": {
      "rows": [{
        "fields": [{
          "name": "id",
          "type": 4,
          "value": 1
        }, {
          "name": "name",
          "type": 12,
          "value": "GTS"
        }, {
          "name": "since",
          "type": 12,
          "value": "2014"
        }
      ]
    },
    "tableName": "product"
  },
  "beforeImage": {
    "rows": [{
      "fields": [{
        "name": "id",
        "type": 4,
        "value": 1
      }, {
        "name": "name",
        "type": 12,
        "value": "TXC"
      }, {
        "name": "since",
        "type": 12,
        "value": "2014"
      }
    ]
  },
  "tableName": "product"
},
  "sqlType": "UPDATE"
}],
  "xid": "xid:xxx"
}
```

可以看到，UNDO_LOG表中记录了分支ID，全局事务ID，以及事务执行的redo和undo数据以供二阶段恢复。

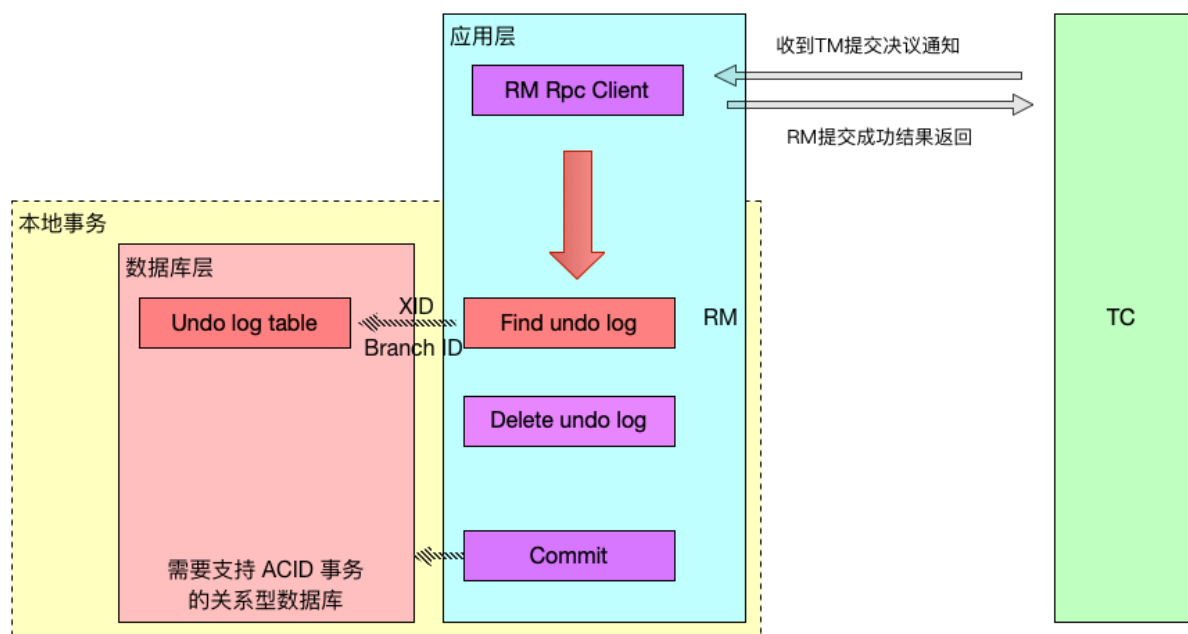
1. RM在同一个本地事务中执行业务SQL和UNDO_LOG数据的插入。

在提交这个本地事务前，RM会向TC申请关于这条记录的全局锁。

如果申请不到，则说明有其他事务也在对这条记录进行操作，因此它会在一段时间内重试，重试失败则回滚本地事务，并向TC汇报本地事务执行失败。如下图所示：

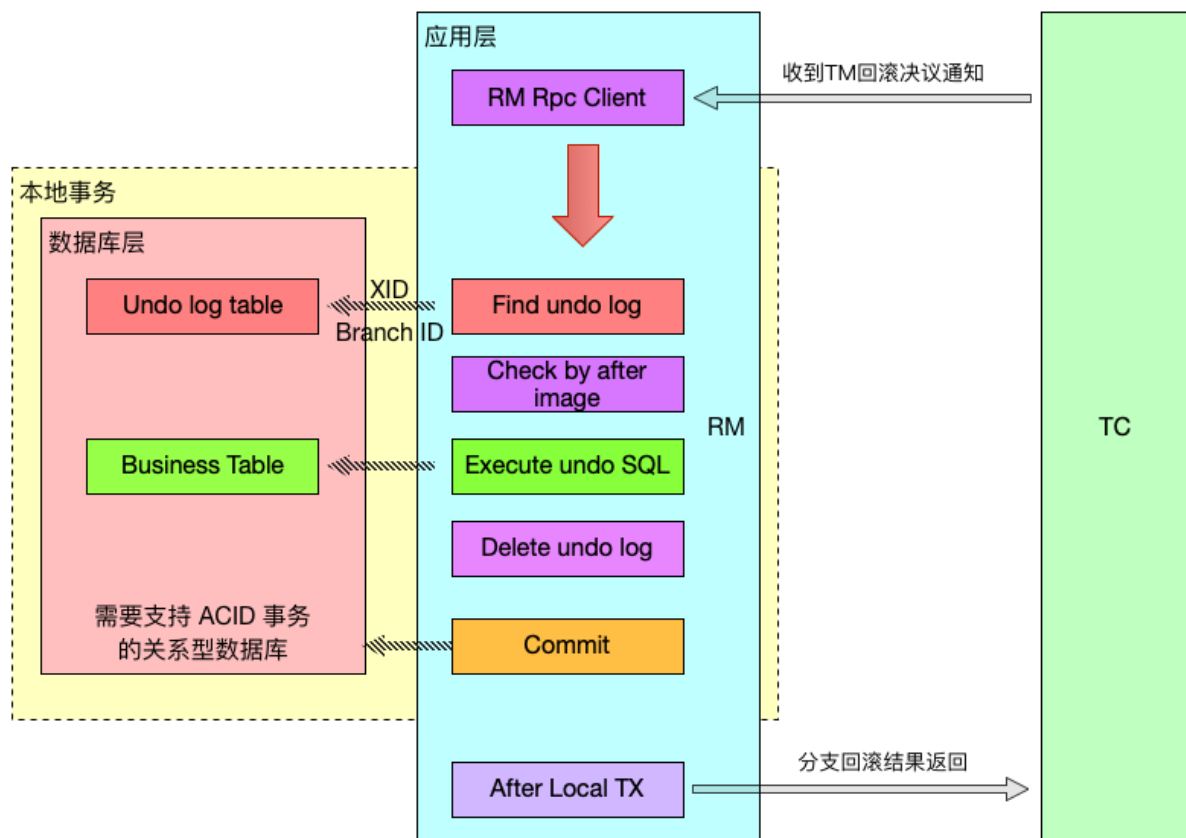


1. RM在事务提交前，申请到了相关记录的全局锁，因此直接提交本地事务，并向TC汇报本地事务执行成功。此时全局锁并没有释放，全局锁的释放取决于二阶段是提交命令还是回滚命令。
2. TC根据所有的分支事务执行结果，向RM下发提交或回滚命令。
3. RM如果收到TC的提交命令，首先立即释放相关记录的全局锁，然后把提交请求放入一个异步任务的队列中，马上返回提交成功的结果给 TC。异步队列中的提交请求真正执行时，只是删除相应 UNDO LOG 记录而已。



提交.png

1. RM如果收到TC的回滚命令，则会开启一个本地事务，通过 XID 和 Branch ID 查找到相应的 UNDO LOG 记录。将 UNDO LOG 中的后镜与当前数据进行比较，如果有不同，说明数据被当前全局事务之外的动作做了修改。这种情况，需要根据配置策略来做处理。否则，根据 UNDO LOG 中的前镜像和业务 SQL 的相关信息生成并执行回滚的语句并执行，然后提交本地事务达到回滚的目的，最后释放相关记录的全局锁。



回滚.png

为什么Seata在第一阶段就直接提交了分支事务？

Seata能够在第一阶段直接提交事务，原因是啥？

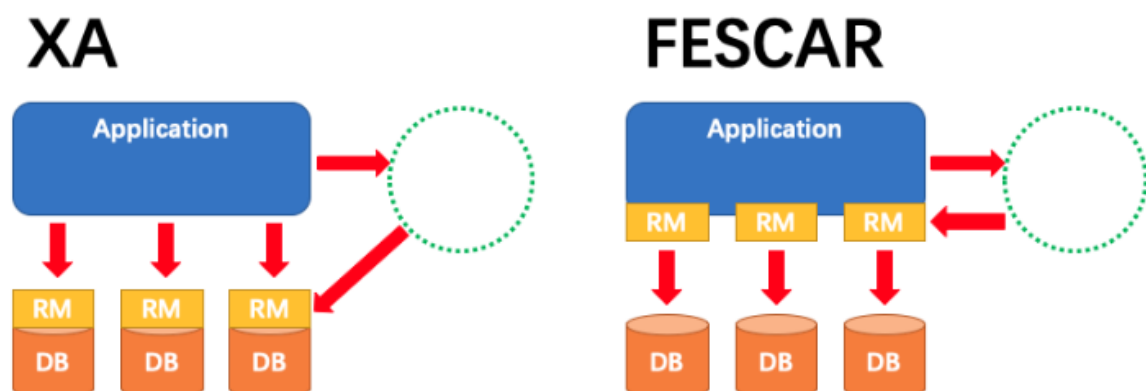
因为Seata框架为每一个RM维护了一张UNDO_LOG表（这张表需要客户端自行创建），其中保存了每一次本地事务的回滚数据。

因此，二阶段的回滚并不依赖于本地数据库事务的回滚，而是RM直接读取这张UNDO_LOG表，并将数据库中的数据更新为UNDO_LOG中存储的历史数据。

Seata的本地事务提交，并不是发生在第二个阶段，为啥？

如果第二阶段是提交命令，那么RM事实上并不会对数据进行提交（因为一阶段已经提交了），而事实上，seata是发起一个异步请求，删除UNDO_LOG中关于本事务的记录。

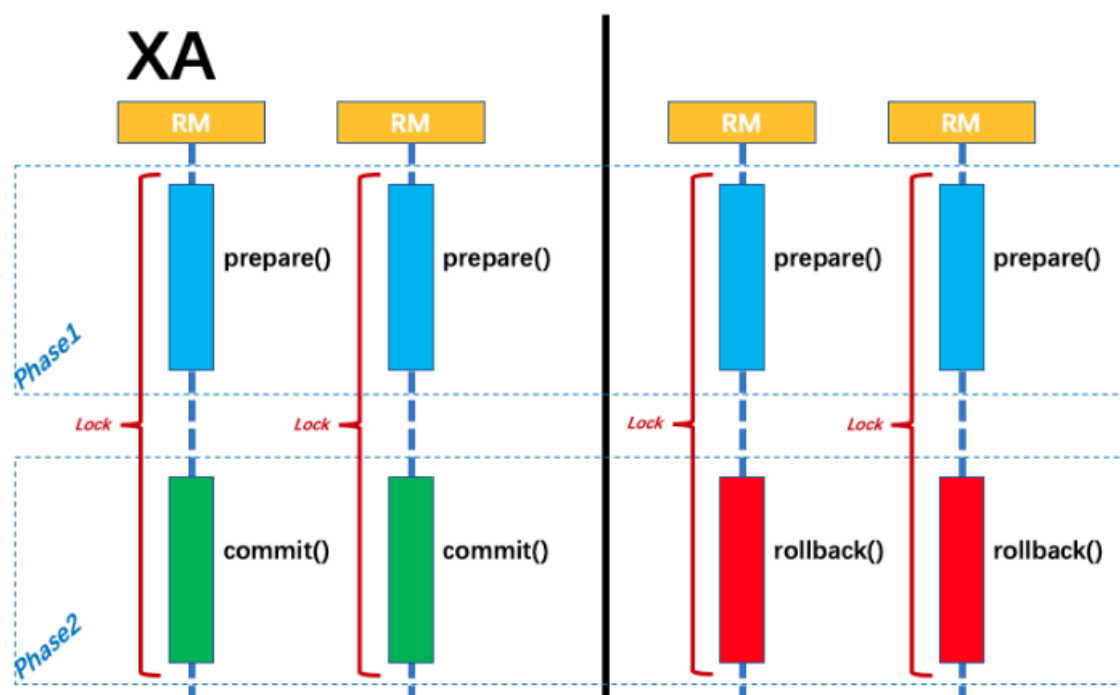
面试真题：比较一下，经典XA和Seata AT模式的区别？



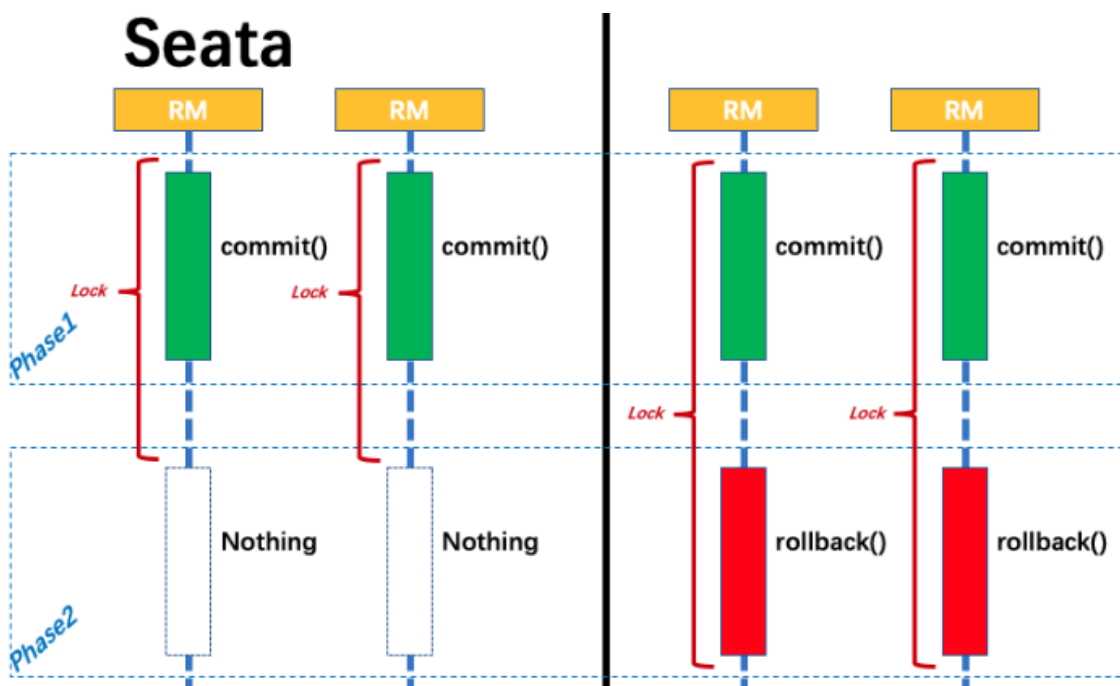
注：Seata的曾用名名为FESCAR。

如图所示，XA 方案的 RM 实际上是在数据库层，RM 本质上就是数据库自身（通过提供支持 XA 的驱动程序来供应用使用）。而 Seata 的 RM 是以二方包的形式作为中间件层部署在应用程序这一侧的，不依赖与数据库本身对协议的支持，当然也不需要数据库支持 XA 协议。这点对于微服务化的架构来说是非常重要的：应用层不需要为本地事务和分布式事务两类不同场景来适配两套不同的数据库驱动。

另外，XA方案无论 Phase2 的决议是 commit 还是 rollback，事务性资源的锁都要保持到 Phase2 完成才释放。而对于Seata，将锁分为了本地锁和全局锁，本地锁由本地事务管理，在分支事务Phase1结束时就直接释放。而全局锁由TC管理，在决议 Phase2 全局提交时，全局锁马上可以释放。只有在决议全局回滚的情况下，全局锁 才被持有至分支的 Phase2 结束。因此，Seata对于资源的占用时间要少的多。对比如下图所示：

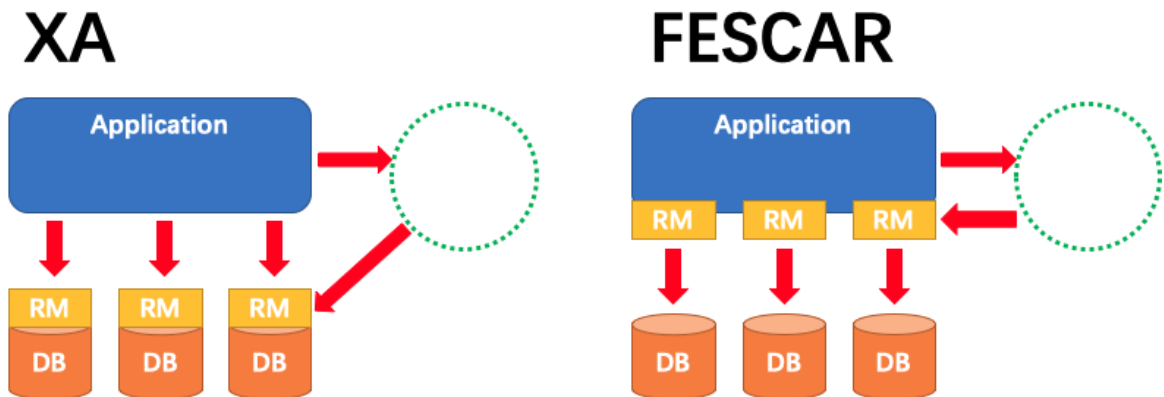


XA锁资源.PNG



Seata 的方案其实一个 XA 两阶段提交的改进版，具体区别如下：

架构的层面：

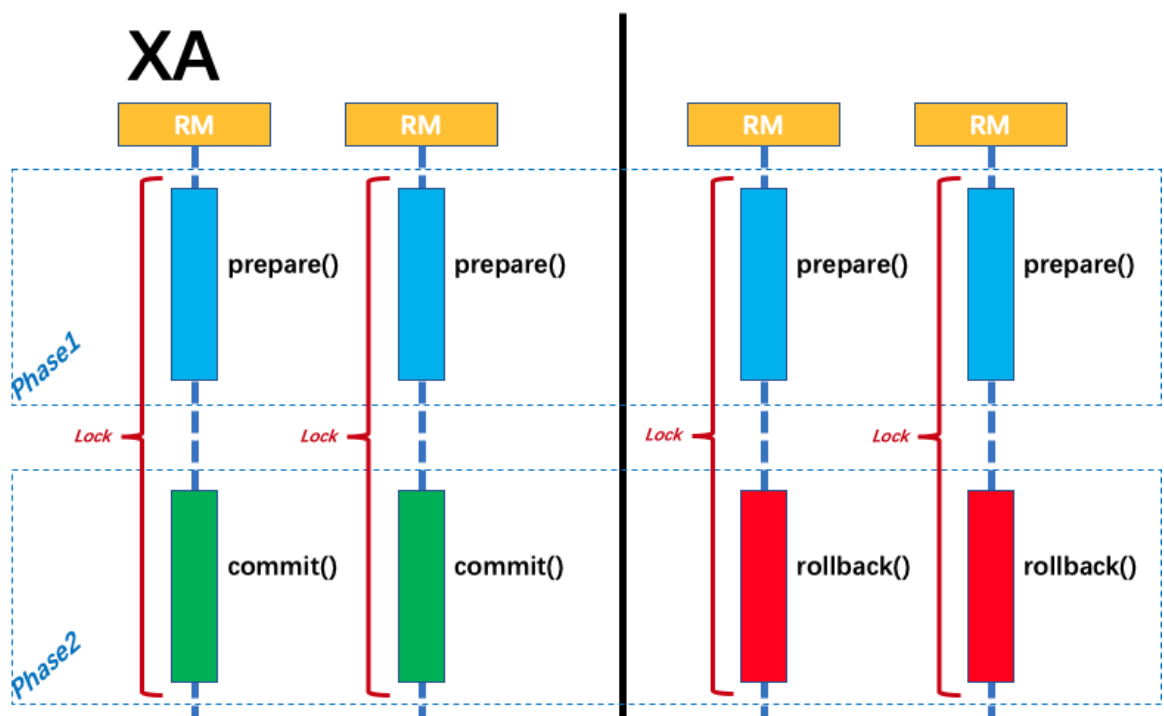


XA 方案的 RM 实际上是在数据库层，RM 本质上就是数据库自身（通过提供支持 XA 的驱动程序来供应使用）。

而 Seata 的 RM 是以二方包的形式作为中间件层部署在应用程序这一侧的，不依赖与数据库本身对协议的支持，当然也不需要数据库支持 XA 协议。这点对于微服务化的架构来说是非常重要的：应用层不需要为本地事务和分布式事务两类不同场景来适配两套不同的数据库驱动。

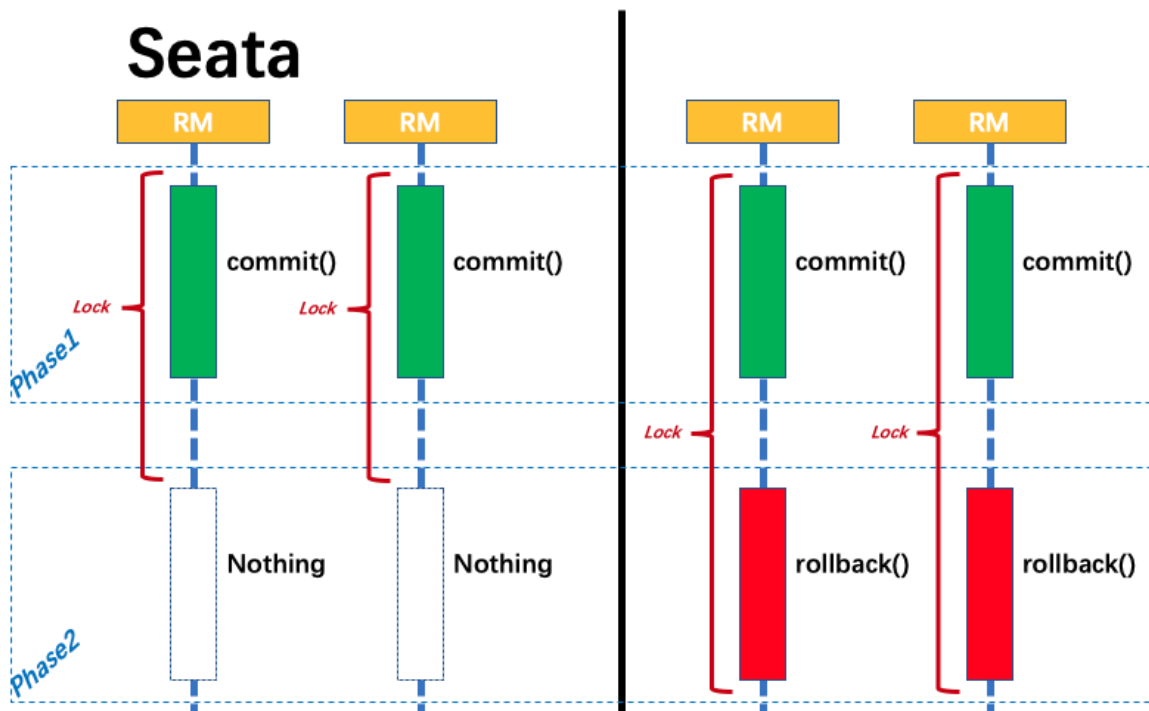
这个设计，剥离了分布式事务方案对数据库在 协议支持 上的要求。

两阶段提交：



无论 Phase2 的决议是 commit 还是 rollback，事务性资源的锁都要保持到 Phase2 完成才释放。

设想一个正常运行的业务，大概率是 90% 以上的事务最终应该是成功提交的，我们是否可以在 Phase1 就将本地事务提交呢？这样 90% 以上的情况下，可以省去 Phase2 持锁的时间，整体提高效率。



- 分支事务中数据的 本地锁 由本地事务管理，在分支事务 Phase1 结束时释放。
- 同时，随着本地事务结束，连接 也得以释放。
- 分支事务中数据的 全局锁 在事务协调器侧管理，在决议 Phase2 全局提交时，全局锁马上可以释放。只有在决议全局回滚的情况下，全局锁 才被持有至分支的 Phase2 结束。

这个设计，极大地减少了分支事务对资源（数据和连接）的锁定时间，给整体并发和吞吐的提升提供了基础。

参考文档：

seata 官方文档地址：

<http://seata.io/zh-cn/docs/overview/what-is-seata.html>

<https://www.cnblogs.com/babycomeon/p/11504210.html>

<https://www.cnblogs.com/javashare/p/12535702.html>

<https://blog.csdn.net/qq853632587/article/details/111356009>

https://blog.csdn.net/qq_35721287/article/details/103573862

<https://www.cnblogs.com/anhaogoon/p/13033986.html>

https://blog.51cto.com/u_15072921/2606182

https://blog.csdn.net/weixin_45661382/article/details/105539999

<https://blog.csdn.net/f4761/article/details/89077400>

https://blog.csdn.net/qq_27834905/article/details/107353159

<https://zhuanlan.zhihu.com/p/266584169>

[论Seata是否适合高并发下的分布式事务解决方案（附测试结果） seata并发量-CSDN博客](#)

